JANUS7

Roadmap & Architektur-Dokumentation

Hybrides Betriebssystem für DSA5 Magierakademien in Foundry VTT

Version 7.0

11. December 2025

Kampagne: Das Geheimnis der Zauberschüler

Akademie der Hohen Magie zu Punin

Inhaltsverzeichnis

[Inhaltsverzeichnis wird beim Öffnen in Word automatisch generiert. Bitte F9-Taste drücken oder Rechtsklick > Felder aktualisieren]

JANUS7 -- Roadmap & Nicht-funktionale Leitplanken

**Stand: Entwurfsfassung\

**

Dieses Dokument fasst die Roadmap von JANUS7 (Phasen 0--8) sowie die ergänzenden, nicht-funktionalen Anforderungen und Wartungsaspekte zusammen. Es dient als Architektur- und Projektleitfaden für die weitere Entwicklung.

Teil I -- Überblick über die Phasen 0--8

Phase 0 -- Leitbild & Governance

Phase 0 definiert das übergreifende Leitbild von JANUS7, die Ziele des Projekts und die Rahmenbedingungen für Weiterentwicklung und Wartung. Hier werden u. a. Versionierungsstrategie, Lizenz- und Nutzungsrahmen, Wartungs- und Upgrade-Strategien sowie grundlegende nicht-funktionale Anforderungen festgelegt.

Kerninhalte Phase 0 (Kurzfassung):

- Leitbild & Projektziele (Magierakademie Punin als Referenzkampagne, aber modularer Kern).
- Abgrenzung Core vs. Setting vs. UI vs. KI-Integration.
- Versioning-Strategy (SemVer) und Umgang mit Breaking Changes.
- Foundry-Upgrade-Strategie (Test-World, Kompatibilität, vorsichtige „maximum"-Versionen).
- Deployment-Prozess DEV → TEST → PROD (inkl. Backups vor Updates).
- Rechtlicher Rahmen (private Nutzung, keine Weitergabe von DSA-IP).
- Non-Functional Requirements (Performance, Sicherheit, Resilienz) als Grundpfeiler.

Phase 1 -- Core & Data Architecture

Phase 1 bildet das technische Fundament von JANUS7: State-Management, Konfiguration, Logging, Hooks und Fehlerbehandlung. Alle höherliegenden Phasen greifen auf diese stabilen Kern-Services zurück.

Kerninhalte Phase 1 (Kurzfassung):

- JanusStateCore: zentraler Kampagnen-State mit Snapshot-, Import-/Export- und Transaktions-API.
- JanusConfig: Welt- und Akademie-spezifische Einstellungen (IDs, Mappings, Feature-Flags).
- Logger-System mit konfigurierbaren Log-Leveln (debug, info, warn, error).
- Hooks-API (janus7StateChanged etc.) als dokumentierte Schnittstelle für Erweiterungen.
- Fehlerbehandlungsstrategie (Error Handling & Recovery, Rollbacks bei Transaktionen).

- State-Backup & Restore (automatische Backups vor kritischen Operationen, Rotation).
- Config-Migration (Schema-Versionen, Migrationspfade für world-config).
- Permissions-Grundlagen (nur GM darf State verändern, Control Panel öffnen etc.).
- Feature Flags / Kill-Switches für große Subsysteme (Atmosphere, KI-Integration ...).

Phase 2 -- Static Academy Data

Phase 2 beschreibt und verwaltet alle statischen Akademiedaten: Lehrpläne, NPC-Datenbanken, Ortsdatenbanken, Bibliotheks- und Regelwerksreferenzen. Diese Daten sind die Grundlage für Simulation, UI und KI-Export.

Kerninhalte Phase 2 (Kurzfassung):

- Strukturierte Datenmodelle für Lektionen, Prüfungen, Fächer, Jahrgänge und Häuser/Zirkel.
- NPC-Datenbank (Lehrkräfte, Schüler, Personal) mit Rollen, Beziehungen und Tags.
- Ortsdatenbank (Akademie-Gebäude, Räume, besondere Schauplätze).
- Bibliotheks- und Regelwerksreferenzen (Verknüpfung zu DSA5-Quellen, ohne IP zu duplizieren).
- Caching-Strategie für häufig genutzte Daten (z. B. Lektionen, NPC-Lookup).
- Trennung von Canon-Lore und Homebrew-Layern (optional, langfristige Erweiterung).

Phase 3 -- DSA5 System Bridge

Phase 3 implementiert die Brücke zwischen JANUS7 und dem DSA5-System in Foundry. Ziel ist eine klare Trennung: JANUS kennt die eigenen Konzepte (Lessons, Exams, Scoring), die DSA5-Bridge übersetzt diese in konkrete Proben, Zauber, Talente und Actor-Manipulationen.

Kerninhalte Phase 3 (Kurzfassung):

- System-Erkennung und Dependency-Checks (DSA5-System vorhanden, Version kompatibel?).
- Adapter-Funktionen für Talent-/Zauberproben, Qualitätsstufen, Vorteile/Nachteile.
- Parser für mechanische Anweisungen („Sinnesschärfe-Probe QS2") → DSA5-interne IDs.
- Fehlerbehandlung bei ungültigen oder veralteten Referenzen (z. B. Zauber nicht gefunden).
- Lesender Zugriff auf DSA5-Compendia (Spells, Talente, Liturgien, Bestiarien).

Phase 4 -- Academy Simulation Logic

Phase 4 enthält die eigentliche „Akademie-Engine": Kalenderlogik, Ereignissystem, Scoring-Mechanik und soziale Dynamiken. Sie arbeitet auf Basis des States (Phase 1) und der statischen Daten (Phase 2) und nutzt die DSA5-Bridge (Phase 3) für Regeln.

Kerninhalte Phase 4 (Kurzfassung):

- Kalender- und Zeitlogik (Jahre, Trimester, Wochen, Tage, Unterrichts-Slots).
- Lessons- und Exams-Engine (Zuweisung von Lektionen zu Zeitslots, Prüfungsabwicklung).
- Event-System (Akademie-Ereignisse, Zufallsereignisse, plotrelevante Events).
- Scoring-System (Haus-/Zirkelpunkte, Belohnungen, Sanktionen, Statistiken).
- Social-System (Beziehungsnetz zwischen NSCs und SCs, Reputations-Tracking).
- Story-Threads und Hooks (offene Stränge, abgeschlossene Ereignislinien).

Phase 5 -- Hybrid & Atmosphere Controller

Phase 5 kümmert sich um die immersive Einbettung am Spieltisch: Audio-/Stimmungssteuerung, optionale Second-Screen-Ansichten und Integration mit bestehenden Foundry-Audio-Tools. Die Funktionalität ist bewusst entkoppelt, damit JANUS bei Problemen auch ohne Atmosphere-Subsystem lauffähig bleibt.

Kerninhalte Phase 5 (Kurzfassung):

- Atmosphere-Controller zur Steuerung von Playlists, Moods und Szenenbindungen.
- Abstrakte Schnittstelle zu Audio-/FX-Modulen, ohne Hard-Dependency.
- Optionale Second-Screen-/Dashboard-Ansichten (read-only) für Spielende.
- Performance- und Fallback-Strategien (Atmosphere kann per Feature Flag deaktiviert werden).

Phase 6 -- User Interfaces

Phase 6 bündelt alle Oberflächen von JANUS7, vor allem das zentrale Control Panel, aber auch Spezialansichten für Kalender, Scoring, Social, KI-Roundtrip und Diagnose. Die UI-Schicht nutzt ApplicationV2 und arbeitet nach dem Prinzip: UI zeigt an, Logik passiert in den Engines der Phasen 1--5.

Kerninhalte Phase 6 (Kurzfassung):

- JANUS Control Panel (Übersicht über Zeit, Events, Punkte, Social, KI-Funktionen).
- Spezielle Views für Kalender, Lektionen, Scoring, Social Network, Atmosphere.
- Permissions-Matrix (welche Rolle darf welche Ansicht öffnen und welche Aktionen ausführen).
- Error-Boundaries in der UI, damit einzelne Apps nicht das gesamte Modul reißen.
- Diagnose-Tab: Anzeige von Versionen, aktiven Modulen, State-Snapshots und Log-Level.
- Grundlegende Accessibility (Keyboard-Navigation, Kontraste) und i18n (DE/EN via Localization).

Phase 7 -- KI-Integration (Roundtrip)

Phase 7 definiert, wie JANUS mit externen KIs (Gemini, ChatGPT, Claude, lokale Modelle) interagiert, ohne von einem konkreten Anbieter abhängig zu sein. Im Fokus stehen robuste Export- und Importpfade, klar definierte JSON-Schemata und eine sichere Einbettung von KI-Vorschlägen in die Kampagne.

Kerninhalte Phase 7 (Kurzfassung):

- JANUS_EXPORT_V2: KI-freundliches Exportformat für State, Akademiedaten und Referenzen.
- JANUS_KI_RESPONSE_V1: standardisiertes Antwortformat für Änderungen (Patch-Plan).
- Exporter: Snapshot-Generation, Download als JSON/ZIP, optional KI-Kit-Struktur.
- Importer: Validierung, Vorschau, Patch-Plan, selektive Übernahme von Änderungen.
- Prompt-Generator: vorbereitete Prompts für typische Aufgaben (Lektionen, Events, Social).
- Sicherheitsmechanismen: Backups vor Import, Import-Recovery, strikte Versionierung.

Phase 8 -- Backlog & Extensions

Phase 8 dient als strukturierter Parkplatz für Erweiterungen, die nicht Teil des JANUS7-Kerns sein müssen, aber langfristig interessant sind. Hier werden Epics wie Multi-Akademie-Support, generisches Schul-Framework, erweiterte Noten-/Zeugnissysteme, Deep-KI-Funktionen oder aufwändige Analytics gebündelt und priorisiert.

Beispiele für Backlog-Epics:

- Multi-Akademie- und Multi-Setting-Unterstützung.
- Generischer School-Framework-Layer (systemagnostisch).
- Noten- und Zeugnis-System, Session-Prep-Assistent, Analytics-Light.
- Second-Screen-/Companion-Views und erweiterte Mobil-Unterstützung.
- Direkte Integrationen mit AI-Modulen (Integrate AI, Phil's AI Assistant, Cibola 8).
- Tooling- und Dev-Workflow-Verbesserungen (Build-Pipeline, Test-Harness, automatisierte Importer).

Teil II -- Nicht-funktionale Anforderungen & Ergänzungen

Die folgenden Punkte ergänzen die Phasen 0--8 um Querschnittsthemen, die für Stabilität, Wartbarkeit und Langfristigkeit des Projekts entscheidend sind. Sie sollten in den jeweiligen Phasen (vor allem 0, 1, 6 und 7) explizit verankert werden.

1. Package, Wartung & Lifecycle

- Manifest- und Release-Strategie (module.json, Compatibility-Felder, SemVer-Regeln).
- Klarer Upgrade-Pfad für neue Foundry-Versionen (Analyse, Test-World, Anpassung, dann PROD).
- Definierter Deployment-Prozess DEV → TEST → PROD (inkl. Backups und Checklisten).
- DSA5-Ökosystem-Bindung: wie JANUS DSA5-System und -Module nutzt, ohne sie zu verändern.
- Rechtlicher Rahmen (private Nutzung, Umgang mit DSA-Inhalten).

2. Resilienz, Fehlerbehandlung & Recovery

- Transaktionsbasierte State-Änderungen mit Rollbacks bei Fehlern.
- Graceful Degradation: JANUS läuft weiter, auch wenn optionale Subsysteme (z. B. Atmosphere) ausfallen.
- Error-Boundaries in UI-Komponenten, um Abstürze zu isolieren.
- Import-Recovery für KI-Änderungen (partielle Anwendung, detaillierte Fehlermeldungen).
- Automatische Backups vor großen Operationen (Wochenwechsel, Mass-Importe).

3. State, Config & Migration

- State-Backup & Restore mit Rotation (z. B. letzte 5 Snapshots).
- Konfigurationsversionierung und Migrationspfade bei Strukturänderungen.
- Klare Trennung von State (dynamische Kampagnendaten) und Config (statische Welt-/Modulsettings).
- Optionale Multi-GM-Strategie (Single-GM empfohlen, Hinweise zu Race Conditions).

4. Logging, Hooks & Erweiterbarkeit

- Konfigurierbare Logging-Level und zentraler Logger (JanusLog).
- Dokumentierte Hooks-API mit Payload-Schemata und Garantien (HOOKS_API.md).
- Stabile, klar benannte Interfaces zwischen Core, System-Bridge, Academy-Engine und UI.
- Explizite Feature Flags (Kill-Switches) für Subsysteme wie Atmosphere, KI-Integration, Scoring.

5. Security, Permissions & Rollen

- Permissions-Matrix für alle Kernfunktionen (Control Panel, KI-Importe, State-Reset etc.).
- Strikte Prüfung von Nutzerrollen bei kritischen Aktionen (nur GM darf State/Import/Reset).
- Bewusste Freigabe von Read-Only-Funktionen (z. B. Leaderboards, Stundenplan) an Spieler.
- Bewertung von KI-Importen als potentiell „unsichere" Quelle -- niemals automatische Ausführung.

6. Performance, Caching & Datenvolumen

- Caching-Strategien für häufig genutzte statische Daten (Lektionen, NPCs, Orte).
- Sparsame Speicherung im State (keine riesigen Textblöcke, stattdessen Referenzen auf Journals).
- KI-Export-Modi (lite, aktuelle Woche, aktuelles Trimester, full) zur Kontrolle des Datenvolumens.
- Bewusster Umgang mit großen Kampagnenwelten (Optionen, nur Teilbereiche zu exportieren).

7. Support-Modus & Diagnose

- Debug-/Support-Mode als Setting (aktiviert detailliertere Logs, zusätzliche Checks, Diagnose-UI).
- Diagnose-Tab im Control Panel mit Informationen zu Versionen, aktiven Modulen, State-Schema.
- Quick-Actions für State-Snapshot, Backup-Anzeige und Export für Fehlersuche.

8. Dokumentation & Onboarding

- Begleitendes Journal-Compendium in Foundry („JANUS Handbuch") mit Einsteiger- und Referenzkapiteln.
- Projektbegleitende Markdown-Dokumente (README, ARCHITECTURE, HOOKS_API, DEPLOYMENT, SECURITY).
- Kurze How-Tos für typische Workflows (Session-Prep, Wochenübergang, KI-Roundtrip).

Fazit

Mit dieser Roadmap und den ergänzenden Leitplanken ist JANUS7 nicht nur funktional, sondern auch langfristig wartbar und erweiterbar angelegt. Die Phasen 0--8 definieren die inhaltlichen und technischen Bausteine, während die nicht-funktionalen Punkte für Resilienz, Sicherheit, Performance und Wartbarkeit sorgen. Dieses Dokument kann sowohl als Architekturgrundlage für die Umsetzung als auch als Referenzleitfaden für spätere Erweiterungen dienen.

Phase 0 – Leitbild & Architektur (Detailliert)

JANUS7 -- Architektur-Leitbild: Eine Vision für das Hybride Rollenspiel

Vision: Ein hybrides Betriebssystem für eine DSA5-Magierakademie am Spieltisch, das die Grenzen zwischen klassischem Pen-&-Paper und digitaler Unterstützung aufhebt.

1. Die drei Säulen der Innovation

A. Der „Hybrid-First" Ansatz: Verschmelzung von Analog und Digital

JANUS7 ist konzeptionell nicht für das reine Online-Spiel über Distanz gedacht, sondern optimiert für das gemeinsame Erlebnis an einem physischen Spieltisch. Das Setup geht von **einem zentralen Bildschirm (Beamer/TV)** aus, der als gemeinsames Fenster in die Spielwelt dient, und **mehreren Tablets oder Laptops (Spieler)**, die als persönliche Interfaces fungieren.

- **Implikation - Audio-Management:** Da sich alle Spieler im selben akustischen Raum befinden, darf Audio (Musik, Soundeffekte) nur auf einem einzigen Client abgespielt werden, um störende Echos und Kakophonie zu vermeiden. Dieser „Master-Client" ist typischerweise der Beamer-Rechner.
- **Implikation - Visuelle Synchronisation:** Würfelwürfe, die auf den individuellen Tablets getätigt werden, müssen visuell und in Echtzeit auf dem Hauptschirm synchronisiert werden. Dies schafft ein gemeinsames Spannungsmoment („Alle schauen auf den Beamer, ob der kritische Treffer gelingt").
- **Implikation - Informationsfluss:** Informationen können gezielt gesteuert werden. Handouts können privat auf ein Tablet geschickt oder öffentlich auf dem Beamer präsentiert werden („Show-to-Screen").

B. „Data-Driven" (Code vs. Content): Flexibilität durch Entkopplung

Eine der wichtigsten architektonischen Entscheidungen ist die strikte Trennung von Programmcode (Logik) und Spielinhalten (Content). Der Code weiß abstrakt, was ein „Zirkel" ist, kennt aber nicht spezifisch „Haus Salamander". Er weiß, wie man Unterricht abbildet, kennt aber nicht den Lehrplan für „Magietheorie I".

- **Implikation - JSON-Datenbanken:** Alle spezifischen Setting-Informationen -- der detaillierte Stundenplan, die Liste der NPCs, die Definitionen der Häuser und deren Wappen -- liegen in externen JSON-Dateien im Verzeichnis data/academy/.
- **Implikation - Laufzeit-Interpretation:** JANUS7 liest diese Daten zur Laufzeit ein. Das bedeutet, dass Anpassungen am Setting (z.B. Umbenennung eines Hauses, Verschiebung einer Prüfung) durch einfaches Editieren einer Textdatei möglich sind, ohne dass eine einzige Zeile JavaScript-Code angefasst werden muss. Dies erhöht die Wartbarkeit und Modifizierbarkeit enorm.

C. Der „Asynchrone KI-Loop": KI als Design-Partner

Die künstliche Intelligenz ist in JANUS7 nicht als Live-Chatbot während des Spiels eingebunden, was den Spielfluss stören könnte. Stattdessen fungiert sie als mächtiges

Planungswerkzeug *zwischen* den Spielsitzungen. Sie ist der „Co-GM", der bei der Vorbereitung hilft.

- **Implikation - Vollständiger State-Export:** Um der KI den nötigen Kontext zu geben, muss der gesamte Spielzustand (State) jederzeit in ein maschinenlesbares Format (JSON) exportierbar sein. Dies umfasst nicht nur Werte, sondern auch narrative Zusammenhänge.
- **Implikation - Patch-Basierte Änderungen:** Die KI liefert keine vagen Textvorschläge, sondern konkrete Daten-Patches (world_build.json), die JANUS7 importieren und anwenden kann (z.B. „Erstelle Szene X", „Setze Beziehung zu NPC Y auf ‚Freundlich'").

2. Technische Prinzipien für Langlebigkeit

1. **Single Source of Truth (SSOT):** Der JanusStateCore ist die einzige Instanz, die den wahren Zustand der Kampagne kennt. UI-Elemente (Fenster, HUDs) halten keine eigenen Daten; sie sind reine Anzeigegeräte, die den Zustand aus dem Core lesen und Änderungen an ihn zurückmelden. Dies verhindert Inkonsistenzen.
2. **Modul-Agnostik und Respekt vor dem Ökosystem:** JANUS7 erfindet das Rad nicht neu. Wir nutzen bestehende, hochwertige Module wie dsa5-core, soms (Abenteuer „Das Geheimnis der Zauberschüler") und magic1-3 als Datenquellen. Wir referenzieren diese Inhalte über UUIDs, kopieren sie aber nicht. Das respektiert Urheberrechte und stellt sicher, dass Updates an den Basis-Modulen auch in JANUS7 ankommen.
3. **IO-Isolation:** Externe Tools (wie KI-Modelle oder Python-Skripte zur Datenanalyse) kommunizieren ausschließlich über das Dateisystem (io/inbox, io/outbox). Es gibt keine direkte API-Abhängigkeit oder fragile Netzwerkverbindungen. Dies macht das System robust und entkoppelt es von spezifischen externen Diensten.

Zielzustand: Ein System, das den Spielleiter von Verwaltungsaufgaben entlastet, die Immersion durch professionelle Präsentation (Beamer/Audio) massiv erhöht und dabei tief in der Welt von DSA5 verwurzelt bleibt, ohne deren Regeln neu erfinden zu müssen.

ChatGTP:

Alles klar, wir steigen in **Phase 0** ein und machen sie so sauber, dass du sie 1:1 als Word-Dokument übernehmen kannst -- inkl. Schnittstellenbild, States-Übersicht, Aufwand, Risiken und Umsetzungsschritten.

Ich habe dein „Phase 0 -- Leitbild"-Dok gelesen und die aktuelle Foundry-VTT-Doku (v13, ApplicationV2, Settings/Flags, Modul-Architektur) gegen geprüft.

Im Folgenden formuliere ich dir **Phase 0** direkt so, wie ich es in ein Word-Dokument schreiben würde.

Phase 0 -- Leitbild & Architektur von JANUS7

0.1 Zweck der Phase

Phase 0 definiert das **architektonische Leitbild** von JANUS7 und setzt den Rahmen für alle folgenden Phasen. Ziel:

- Eine **klare Vision**, was JANUS7 sein und nicht sein soll.
- **Architekturprinzipien**, die in allen Modulen gelten (Core, Academy, DSA5-Bridge, UI, KI, Atmosphere).
- Ein erstes, grobes Bild der **globalen Schnittstellen** und **States**, an denen spätere Phasen anknüpfen.
- Eine Grundlage, um **Scope Creep** zu verhindern und spätere Entscheidungen gegen diese Leitplanken zu prüfen.

Diese Phase erzeugt nur **Konzept & Dokumentation**, noch keinen produktiven Code.

0.2 Vision: JANUS7 als hybrides Betriebssystem für die Akademie

0.2.1 Grundidee

JANUS7 ist kein „nettes Zusatzmakro", sondern ein **hybrides Betriebssystem** für eine DSA5-Magierakademie in Foundry VTT:

- Es **orchestriert** Zeit, Unterricht, Prüfungen, Zirkel-Punkte, NSCs, Orte und Audio/Atmosphäre.
- Es nutzt die bestehenden DSA5-Systeme und Ulisses-Module (z. B. dsa5, dsa5-core, dsa5-soms) als **Single Source of Truth** für Regeln und offiziellen Content.
- Es ist konsequent **datengetrieben**: Lehrplan, Kalender, NPC-Profile etc. liegen als JSON im Modul und können ohne Codeänderung angepasst werden.

0.3 Die drei zentralen Säulen

A. Hybrid-First: Pen-&-Paper + Foundry als Bühne

JANUS7 ist explizit auf ein **physisches, hybrides Setup** optimiert:

- Ein **Leiter-Rechner** mit Foundry (GM-Client).
- Ein **Beamer/TV-Display** als „Bühne" (Player-View).
- Optional: **Spieler-Laptops/Tablets**, aber kein Zwang.

Implikationen:

4. Audio-Management

   - Nur ein „Master-Client" spielt Audio ab (meist der Beamer-Client), alle anderen bleiben leise, um Echo und Chaos zu vermeiden.

5. Visuelle Synchronisation

- Würfelwürfe, Szenenwechsel, Visual-Novel-Overlays etc. sollen auf dem Beamer in einer **gemeinsamen Perspektive** sichtbar sein.

- JANUS7 steuert gezielt, was auf GM-Client vs. Beamer gezeigt wird (z. B. Beamer = nur Szene, GM = Panel + Steuerung).

6. Informationsfluss

- Es gibt drei Informationskanäle:

  - **Nur GM** (z. B. Planungs-Panel, KI-Protokolle).

  - **Nur bestimmte Spieler** (handout-ähnlich, optional später via Player-Clients).

  - **Öffentlich auf dem Beamer** (Handouts, Karten, Lerninhalte, Prüfungsfragen).

Konsequenz: Alle späteren UI-Komponenten (Phase 5 & 6) müssen dieses Hybrid-Modell bewusst berücksichtigen und dürfen **nicht** implizit von einem „jeder-sitzt-alleine-am-Rechner"-Online-Setup ausgehen.

B. Data-Driven: Code vs. Content strikt getrennt

JANUS7 trennt streng zwischen:

- **Code / Logik**\

  -- in modules/janus7/core, academy, systems, ui, ki, atmosphere.

- **Content / Daten**\

  -- in modules/janus7/data/\\ (JSON), plus Foundry-Daten in DSA5- und Ulisses-Modulen.

Grundprinzip:

Code weiß *wie* etwas passiert, Daten bestimmen *was* genau passiert.

Konkrete Prinzipien:

7. JSON als Primärformat für Akademie-Daten

- Kalender, Lehrpläne, Prüfungen, NPC-Katalog, Orte, Bibliothek etc. liegen als JSON-Dateien im Verzeichnis data/academy/.

- Beispiele:

  - calendar.json

  - lessons.json

  - exams.json

    - npcs.json
    - locations.json
    - library.json

8. Laufzeit-Interpretation

- Der Code interpretiert diese JSON-Daten zur Laufzeit:
    - Calendar.getLessonsForToday() liest calendar.json + lessons.json.
    - ScoringEngine benutzt nur IDs/Namen, aber kennt keine Aventurien-Lore.
- Änderungen am Lehrplan oder an NPCs erfolgen durch Anpassen der JSON-Dateien, nicht durch Umbauen des Codes.

9. Modul-Agnostik

- Offizielle Foundry-/Ulisses-Module (DSA5-System, DGdZ-Abenteuer, Bestiarien, Magiebände etc.) sind über **UUIDs & Keys** angebunden.
- JANUS7 speichert nur:
    - z\. B. "rulesRefs": \[{ "moduleId": "dsa5-magic-1", "pack": "magic1spells", "entryName": "Analys Arcanstruktur" }\]
- Die Inhalte selbst bleiben im Originalmodul und werden nicht dupliziert.

C. Asynchroner KI-Loop: KI als Co-GM, nicht als Spieltisch-Chatbot

KI ist in JANUS7 als **Design-Partner** gedacht:

- Sie hilft beim **Vorbereiten**:
    - Ausarbeiten von Unterrichtseinheiten.
    - Generieren von NSC-Details.
    - Erweitern von Zufallstabellen, Szenen, Flavor-Texten.
- Nicht als Chatbot mitten in der Session.

Implikationen:

10. Vollständiger, aber verdichteter State-Export

- JANUS7 kann den aktuellen Zustand (Zeit, wichtige NSCs, offene Plotthreads, Scoring, Bedrohungsgrad u. a.) in eine JSON-Datei schreiben.
- Diese Datei ist so strukturiert, dass verschiedene KIs sie interpretieren können, ohne Foundry-API zu kennen.

11. Patch-basierte Änderungen

- KI liefert Änderungsvorschläge (Patches):
  - „Füge neuen NPC mit ID X hinzu ..."
  - „Erweitere Lesson Y um Abschnitt Z ..."
  - „Setze Beziehung A→B auf 'angespannt' ..."
- JANUS7 validiert diese Patches und spielt sie dann kontrolliert in den State und ggf. in Foundry-Dokumente ein.

12. Dateibasierte IO-Isolation

- Kein direkter HTTP- oder WebSocket-Call zur KI aus Foundry.
- Stattdessen: Export/Import über Dateien (io/outbox, io/inbox), so dass jede beliebige KI / jedes Script dazwischen geschaltet werden kann.

0.4 Technische Leitprinzipien für Langlebigkeit

0.4.1 Single Source of Truth (SSOT) -- JanusStateCore

- Es gibt **einen** globalen, serialisierbaren State namens JanusStateCore, der:
  - die aktuelle Ingame-Zeit,
  - Scoring,
  - Beziehungs-/Story-Informationen,
  - UI-/Hybrid-Modus-Infos (z. B. aktueller Beamer-Modus)\

    verwaltet.
- Technisch wird dieser State über **Game Settings** im world-Scope persistiert (game.settings.register), wie von Foundry für modulbezogene Daten empfohlen.
- Alle Subsysteme (Kalender, Scoring, KI-Export, UI) lesen/schreiben über klar definierte **APIs** auf diesen State. Direkte Schreiberei in game.settings oder Flags durch andere Teile des Codes ist verboten.

0.4.2 Modul-Agnostik & Foundry-Konformität

- JANUS7:
  - nutzt **ApplicationV2** für UIs, da ApplicationV1 ab Foundry v16 entfernt wird.
  - hält sich an die offiziellen Patterns für Module (module.json, Hooks, Settings, Handlebars/HTML-Rendering).
- DSA5 & Ulisses-Module:

- werden über ihre Modul-IDs und Compendium-Packs referenziert, nicht nachgebaut.

0.4.3 IO-Isolation

- Externe Tools (KIs, Python-Skripte, andere LLMs) greifen **ausschließlich** über die Dateien im io-Verzeichnis auf JANUS7-Daten zu.
- Keine direkte Abhängigkeit von bestimmten KI-Anbietern, -APIs oder Cloud-Diensten.

0.5 Globale Schnittstellen & Abhängigkeiten

Ziel von Phase 0 ist nur das „Architektur-Contract", nicht die Implementierung.

0.5.1 Globale Namespaces

- Modul-ID: janus7
- Globaler Namespace in Foundry:
  - game.janus7 (wird in Phase 1 konkretisiert)
- Erwarte Sub-Namespace-Struktur:
  - game.janus7.core
  - game.janus7.academy
  - game.janus7.systems.dsa5
  - game.janus7.ui
  - game.janus7.ki
  - game.janus7.atmosphere

0.5.2 Konzeptionelle State-Bereiche

Diese werden in **Phase 1** technisch konkret:

13. Zeit & Akademiestatus
    - state.time: Jahr, Trimester, Woche, Tag, Phase
    - state.phaseOfStudy: Vorschule, Elevium, Novizium, Studium
14. Scoring & Meta
    - state.scoring.circles: Punkte der einzelnen Zirkel/Häuser
    - state.scoring.students: Punkte einzelner Schüler
    - state.meta: Schema-Version, Timestamps

15. Story & Beziehungen
    - state.story.threads: laufende Plots
    - state.social.relations: Beziehungsnetz
16. Hybrid-/UI-Kontext
    - state.display.beamerMode: aktueller Anzeige-Modus
    - state.display.activeSceneKey
17. Flags
    - state.flags.todayLessonOverrideId, state.flags.todayExamOverrideId

Phase 0 legt nur fest, **dass** diese Kontexte zentral existieren; Phase 1 wird das genaue Schema definieren.

0.5.3 Abhängigkeiten zu späteren Phasen

- **Phase 1**: Implementiert JanusStateCore, Config, Director, IO, Validator exakt entlang dieser Leitlinien.
- **Phase 2**: Füllt data/academy/* mit Lehrplan, Kalender, NPCs usw.; nutzt aber keine eigenen Speicherorte.
- **Phase 3**: Baut DSA5-Adapter auf game.janus7.core auf; keine eigenen globalen States.
- **Phase 5 & 6** (Hybrid & UI): Verwenden ausschließlich JanusStateCore und Director, um Anzeigen & Atmosphären zu steuern.
- **Phase 7** (KI): Verwendet ausschließlich core.io & definierte Export-Schemata, kein direktes Rumfingern in Foundry-Dokumenten ohne Validator.

0.6 Aufwandsschätzung für Phase 0

Annahme: 1 erfahrener Entwickler (du) + KI-Unterstützung.

- **Konzeptverfeinerung & Konsolidierung** (dieses Dokument, Diagramme, Abstimmen mit Roadmap):
  - ca. 0,5--1 Personentage
- **Abstimmung mit Foundry-Doku & API-Konzept** (ApplicationV2, Settings, Module-Pattern):
  - ca. 0,5 PT
- Erstellen finaler Architektur-Dokumentation (Word, ggf. Diagramme getrennt):
  - ca. 0,5 PT

Gesamt für Phase 0: ca. 1,5--2 Personentage (reine Konzeptarbeit, ohne Implementierung).

0.7 Risiken & Gegenmaßnahmen in Phase 0

18. Risiko: Scope Creep / Feature-Wildwuchs

- *Beschreibung:* Es werden immer mehr „coole Ideen" aufgenommen, bis JANUS7 unüberschaubar wird.
- Gegenmaßnahme:
  - Harte Trennung: **MVP vs. Backlog** bereits in der Roadmap festhalten.
  - Jedes Feature bekommt eine Phase/Zugehörigkeit oder fliegt raus.

19. Risiko: Fehlende Alignment mit Foundry-Entwicklung

- *Beschreibung:* Foundry ändert UI-Framework (ApplicationV2), Flags/Settings-Verhalten oder Module-APIs.
- Gegenmaßnahme:
  - Bewusste Entscheidung für **ApplicationV2**-Framework.
  - Nutzung offizieller Mechanismen (game.settings, Hooks), kein Hacken von internen Properties.
  - Phase 0 dokumentiert die Ziel-Version (z. B. Foundry 13.x).

20. Risiko: KI-Abhängigkeit wird zu stark

- *Beschreibung:* JANUS7 verlässt sich zu sehr auf externe KI, ohne dass das System ohne KI benutzbar ist.
- Gegenmaßnahme:
  - KI als **Add-on**, alle Kernfunktionen (Zeit, Lehrplan, Scoring) müssen ohne KI laufen.
  - KI-Loop ist optionaler Power-User-Workflow.

21. Risiko: Datenmodell wird zu starr

- *Beschreibung:* State-/JSON-Schema lässt spontane Anpassungen am Tisch kaum zu.
- Gegenmaßnahme:
  - Phase 1 baut bewusst ein **erweiterbares Schema** (z. B. flags, generische meta-Felder).

- Phase 2--4 behandeln Content als austauschbar und versionierbar (z. B. schemaVersion in JSON).

0.8 Umsetzungsschritte für Phase 0

Schritt 0:\

Roadmap finalisieren (hast du faktisch schon), Phase-Übersicht mit MVP/Backlog-Tagging.

Schritt 1:\

Dieses Leitbild-Dokument (Phase 0) in Word finalisieren:

- Kapitelstruktur (0.1--0.8),
- Referenzen auf spätere Phasen (1--7),
- ggf. ergänzende Diagramme (z. B. Schichtenmodell).

Schritt 2:\

Ein **Architektur-Diagramm** erstellen (z. B. als einfache Grafik für dein Word-Dokument):

- Schichten: core -- academy -- systems -- ui -- ki -- atmosphere.
- Schnittstellen zu Foundry (ApplicationV2, Settings, Documents/Compendia).

Schritt 3:\

Definition eines **„Architecture Contracts"** als kurze Checkliste:

- Darf dieses neue Feature:
  - eigenständig Daten persistieren? → nur über JanusStateCore oder game.settings.
  - direkt Foundry-Dokumente anfassen? → nur via Director & Validator.
  - neue globale Objekte definieren? → nur unter game.janus7.

Schritt 4:\

Phase-1-Dokument öffnen und auf Konsistenz mit Phase 0 prüfen:

- Hat Phase 1 dieselbe State-Idee (JanusStateCore)?
- Hält Phase 1 die Trennung Code vs. Content ein?
- Nutzt Phase 1 nur ApplicationV2 für UI-Komponenten?

Schritt 5:\

Phase 0 als „Done" markieren, sobald:

- Leitbild-Dokument in Word fertig ist.

- Mindestens 1 Diagramm existiert.
- Der Architecture Contract in Stichpunkten vorliegt.
- Alle folgenden Phasen (1--7) sich explizit auf diese Prinzipien beziehen.

Wenn du magst, können wir als nächsten Schritt direkt **Phase 1 -- Core & Data Architecture** nehmen und dort richtig ins Eingemachte gehen: exakte JanusStateCore-Struktur, Schnittstellen (JanusState.get/set), Module/Dateistruktur, sowie Aufwand & Risiken dieser Phase.

Phase 1 – Core & Data Architecture (Detailliert)

Phase 1: Core & Data Architecture -- Erweiterte Dokumentation

Ziel: Aufbau einer robusten, persistenten Datenhaltung und einer internen Kommunikationsinfrastruktur, die als das zentrale Nervensystem für alle nachfolgenden Module dient. Der Core stellt sicher, dass alle Daten konsistent, sicher und für alle Systemteile zugänglich sind.

Module & Funktionen

1.1 State Manager V2 (core/state.js)

Der **State Manager** ist das absolute Herzstück von JANUS7. Er ist der Hüter der „Wahrheit" über den Zustand der Kampagne. In früheren Versionen war dies oft auf verschiedene Skripte verteilt; hier wird es zentralisiert. Er ist nicht nur ein passiver Datenspeicher, sondern eine aktive Komponente, die Änderungen überwacht, validiert und an andere Systemteile (wie UI oder externe Tools) kommuniziert.

- **Funktion:** Speichert und verwaltet zentral alle dynamischen Daten der Kampagne. Dazu gehören die aktuelle Zeitrechnung (das „Wann"), soziale Graphen (das „Wer mit Wem"), ökonomische Daten (das „Wieviel") und der Status der hybriden Anzeige (das „Was auf dem Beamer"). Er garantiert die Datenintegrität und verhindert inkonsistente Zustände (z.B. Zeitreisen in die Vergangenheit).
- **Erweitertes Datenmodell (Schema):** Das Schema ist so konzipiert, dass es flexibel genug für zukünftige Erweiterungen ist (Backlog-Features), aber dennoch eine klare Struktur für die KI und den Validator vorgibt.\

```
{\

meta: {\

version: „7.1.0", // Schema-Version für Migrationen\

lastSave: Timestamp, // Zeitstempel der letzten Speicherung\

campaignId: „unique-id" // Eindeutige ID zur Identifikation der Kampagne\

},\

\

// ZEIT (Die Taktung der Kampagne)\

time: {\

year: 1, // Schuljahr (1-9)\

trimester: 1, // Trimester (1-3)\
```

week: 1, // Woche im Trimester (1-12)\

day: „Praiosstag", // Wochentag (String oder Enum)\

phase: „Morgen", // Tagesphase (Morgen, Vormittag, Mittag, Nachmittag, Abend, Nacht)\

isHoliday: false, // Flag für Feiertage/Ferien\

totalDaysPassed: 0 // Absoluter Zähler für vergangene Tage (für Berechnungen)\

},\

\

// SIMULATION (Daten-Container für Phase 4 - Logik folgt später)\

social: {\

relations: {\

// Struktur: NPC_ID -\> PC_ID -\> {value: -100 bis 100, status: „neutral"\|"friend"\|"rival"}\

// Hier werden Beziehungen zwischen Charakteren und NPCs gespeichert.\

},\

gossip: \[\] // Liste aktiver Gerüchte mit Verbreitungsgrad und Wahrheitsgehalt\

},\

economy: {\

debts: {}, // Schulden-Tracker pro Spieler/Gruppe (Wer schuldet wem was?)\

stash: {}, // Gruppen-Inventar für spezielle Ressourcen (Zutaten, Artefakte) - getrennt vom DSA5-Inventar\

allowance: {} // Taschengeld-Status und nächste Auszahlung\

},\

scoring: {\

circles: { /\ *HouseID* -\> *Points* */ }, // Globale Hauspunkte\

students: { /* *ActorID* -\> *Points* \/ }, // Individuelle Schülerpunkte für Auszeichnungen\

mvp: null // Aktueller „Most Valuable Pupil"\

},\

```
\

// HYBRID (Status für Phase 5 - Beamer-Steuerung)\

display: {\

mode: „dashboard", // Aktueller Modus: cinema, scene, visual_novel, dashboard, show_to_screen\

activeScene: null, // ID der Szene, die auf dem Beamer gezeigt wird\

activeOverlay: null, // ID eines Overlays (z.B. Handout, Bild), das über der Szene liegt\

visualNovelData: { // Daten für den VN-Modus (Portraits, Text, Hintergrund)\

leftPortrait: null,\

rightPortrait: null,\

text: „"\

}\

},\

\

// STORY (Gedächtnis für Phase 7 - KI-Kontext)\

story: {\

activeThreads: \[\], // Liste offener Handlungsstränge (Quests, Mysterien)\

resolvedThreads: \[\], // Liste abgeschlossener Handlungsstränge (Historie für KI)\

npcMemory: {}, // Was wissen spezifische NPCs über die SCs? (Wichtig für KI-Konsistenz)\

chronicle: \[\] // Logbuch wichtiger Ereignisse (Zeitstempel, Beschreibung, Tags)\

}\

}
```

- **Persistenz:** Die Daten werden persistent über game.settings in Foundry VTT gespeichert. Dies ist sicherer und performanter als World Flags für große Datenmengen. Der Speichervorgang sollte optimiert sein (z.B. Debouncing), um die Performance nicht zu beeinträchtigen, wenn sich Daten schnell ändern (z.B. während eines Kampfes).

1.2 Config & Assets (core/config.js)

Die **Config** dient als zentrale Registratur und Abstraktionsschicht für Foundry-IDs. Sie verhindert „Hardcoding" von IDs im Quellcode und ermöglicht so die Portabilität des Moduls zwischen verschiedenen Welten oder Modul-Installationen. Ohne diese Schicht müssten bei jedem neuen Spielstart IDs manuell angepasst werden.

- **Funktion:** Mappt logische, sprechende Schlüssel (z.B. „location.library.main", „npc.teacher.alchemy") auf die tatsächlichen, kryptischen Foundry UUIDs (z.B. „Scene.g73j\...").
- **Asset-Management:** Verwaltet Pfade zu Bildern, Sounds und anderen Assets. Es kann Fallback-Logiken implementieren (z.B. „Wenn kein spezifisches Bild für ‚Labor' existiert, nimm das generische Labor-Bild"). Dies ist entscheidend für die visuelle Konsistenz im Hybrid-Modus.
- **Modul-Support:** Erkennt beim Start automatisch, welche relevanten DSA5-Module (z.B. dsa5-soms, dsa5-magic, dsa5-bestiary) installiert und aktiv sind. Basierend darauf lädt es entsprechende Konfigurations-Presets, um Inhalte aus diesen Modulen nahtlos zu integrieren (z.B. Zaubersprüche aus dsa5-magic für den Indexer).

1.3 Director (core/director.js)

Der **Director** ist die exekutive Gewalt im Core. Er führt Befehle aus, die den Zustand der Foundry-Oberfläche verändern, ohne dabei Geschäftslogik zu enthalten. Er ist das Werkzeug, mit dem andere Module (wie der Atmosphere Controller oder die KI) die Welt „bedienen".

- **Funktion:** Bietet Methoden zum Öffnen und Anzeigen von Szenen, Journalen, Playlists und Compendium-Einträgen. Er abstrahiert die Foundry-API, sodass andere Module nicht wissen müssen, *wie* man eine Szene wechselt (z.B. game.scenes.get(id).view()), sondern nur *dass* sie gewechselt werden soll.
- Methoden:

- openScene(key, options): Wechselt die Szene. Optionen können Übergangseffekte oder Aktivierung für bestimmte User (Beamer) beinhalten.

- showJournal(key, pageId, options): Zeigt ein Journal oder eine spezifische Seite an.

- playPlaylist(key, options): Startet oder stoppt Playlists, steuert Lautstärke und Überblendungen.

- showImage(url, options): Zeigt ein Bild (z.B. Handout) direkt an (Lightbox).

1.4 IO-Layer (core/io.js)

Der **IO-Layer** ist die Schnittstelle zur „Außenwelt", insbesondere für die asynchrone Kommunikation mit KI-Tools oder externen Skripten. Er isoliert das Foundry-System von

direkten API-Zugriffen und nutzt stattdessen ein dateibasiertes Austauschverfahren, was die Sicherheit und Stabilität erhöht.

- **Funktion:** Verwaltet den Datenaustausch über JSON-Dateien in definierten Verzeichnissen.
- **Outbox (io/outbox):** Hier schreibt JANUS7 Daten hinein, die für externe Tools bestimmt sind.
  - state_snapshot.json: Aktueller Kampagnenstatus.
  - context_export.json: Kontext für den KI-Prompt.
  - knowledge_base.json: Exportierte Regeldaten (vom Indexer).
- **Inbox (io/inbox):** Hier überwacht JANUS7 auf neue Dateien, die von externen Tools erstellt wurden.
  - world_build.json: Von der KI generierte Szenen und Daten.
  - npc_update.json: Aktualisierungen für NPCs.
- **Mechanismus:** Nutzt die Foundry FilePicker API zum Schreiben und Lesen. Ein File-Watcher (z.B. via Polling oder Socket, falls möglich) prüft regelmäßig auf Änderungen in der Inbox und triggert entsprechende Events (janus7FileReceived).

1.5 Validator (core/validator.js)

Der **Validator** ist der „Türsteher" für alle Daten, die von außen (via IO-Layer) in das System gelangen. Er stellt sicher, dass importierte Daten die Integrität des Systems nicht gefährden und dem erwarteten Format entsprechen.

- **Funktion:** Validiert eingehende JSON-Daten gegen definierte Schemas (z.B. JSON Schema). Er prüft Struktur (sind alle Felder da?), Datentypen (ist das Jahr eine Zahl?) und logische Konsistenz (z.B. „Existiert die referenzierte NPC-ID wirklich?").
- **Fehlerbehandlung:** Wenn Daten ungültig sind, lehnt der Validator sie ab und generiert detaillierte Fehlermeldungen oder Warnungen, die im Debug-Panel angezeigt werden können. Er verhindert, dass „halluzinierte" IDs oder korrupte Daten den State beschädigen oder das System zum Absturz bringen.

🔗 Abhängigkeiten & Schnittstellen

Abhängigkeiten (Benötigt\...)

Der Core ist die unterste Schicht der Architektur und sollte so wenige Abhängigkeiten wie möglich haben, um Zyklen zu vermeiden und die Stabilität zu gewährleisten.

- **Foundry VTT Core API:** Unerlässlich für die Interaktion mit der Plattform (z.B. game.settings für Persistenz, FilePicker für IO, AudioHelper für Sound, Hooks für Events, foundry.utils für Hilfsfunktionen).

- **Keine:** Der Core darf **keine** harten Abhängigkeiten zu Phase 2 (Academy), Phase 3 (System) oder Phase 4 (Simulation) haben. Er stellt die Infrastruktur bereit, auf der diese Module aufbauen. Kommunikation erfolgt über Events/Hooks oder Dependency Injection (später).

Schnittstellen (Bietet an\...)

Der Core stellt APIs bereit, die von allen anderen Modulen genutzt werden. Dies ist die öffentliche Schnittstelle des JANUS7-Kerns.

- State API (JanusState):
  - get(path): Liest Daten aus dem State (z.B. JanusState.get(„time.year")).
  - set(path, value): Schreibt Daten in den State (z.B. JanusState.set(„scoring.circles.salamander", 150)).
  - update(updateObject): Führt mehrere Änderungen atomar durch.
  - save(): Erzwingt das Speichern in die Datenbank.
- Director API (Director):
  - openScene(key): Öffnet eine Szene basierend auf ihrem logischen Key.
  - showJournal(key): Zeigt ein Journal an.
  - playPlaylist(key): Spielt Musik ab.
- IO API (IO):
  - write(filename, data): Schreibt Daten in eine Datei in der Outbox.
  - read(filename): Liest Daten aus einer Datei in der Inbox.
  - watch(callback): Registriert einen Callback für neue Dateien in der Inbox.
- Hooks:
  - janus7StateChanged: Wird gefeuert, wenn sich der State ändert. UI-Module können darauf hören, um sich zu aktualisieren. Payload: { state, diff }.
  - janus7ConfigLoaded: Wird gefeuert, wenn die Konfiguration vollständig geladen ist.
  - janus7FileReceived: Wird gefeuert, wenn eine neue Datei in der Inbox erkannt wurde.

ChatGTP:

Alles klar, Phase-1-Modus: **Core & Data Architecture**.\

Ich schreibe dir das jetzt so, dass du es direkt als Word-Kapitel übernehmen kannst -- mit:

- Ziel & Scope
- Komponenten (State, Config, Director, IO, Validator)
- Vollständigem JanusStateCore-Schema
- Schnittstellen & Abhängigkeiten
- Aufwandsschätzung
- Risikoanalyse
- Umsetzungsschritten

Ich halte mich dabei bewusst an Foundry-VTT-Best Practices: game.settings für persistente Daten, scope: „world", ApplicationV2-Denke für spätere UIs usw.

Phase 1 -- Core & Data Architecture

1.1 Ziel & Scope der Phase

Ziel von Phase 1:\

Aufbau eines robusten, serialisierbaren **JANUS-Kernzustands** und der dazugehörigen Infrastruktur, damit:

- alle dynamischen Kampagnendaten (Zeit, Scoring, Social, Story, Economy, Display, Flags)
- konsistent, persistent und zentral verwaltet werden,
- von allen anderen Phasen (Kalender, Akademie-Logik, DSA5-Bridge, UI, KI) sauber gelesen/geschrieben werden können
- und sicher in einem Foundry-kompatiblen Speichermechanismus liegen (game.settings mit scope: „world").

Im Scope dieser Phase:

- Definition des **JanusStateCore-Schemas** (Datenmodell)
- Implementierung des **State Managers** (core/state.js)
- Implementierung des **Konfigurations-Layers** (core/config.js)
- Implementierung des **Director** als Orchestrierungs-API (core/director.js)
- Implementierung des **IO-Layers** (core/io.js)
- Implementierung des **Validators** (core/validator.js)
- Einheitliche **Logging-Basis** (core/logger.js, minimal)

Nicht in Phase 1 (kommt später):\

Keine UI (ApplicationV2-Apps), keine DSA5-Systemlogik, keine Akademie-Simulationslogik, keine KI-Pipeline -- die docken nur an die hier definierten APIs an.

1.2 Modulübersicht & Abhängigkeiten

Komponenten dieser Phase

- core/state.js -- **State Manager V2**
- core/config.js -- Config & Mapping Layer
- core/director.js -- Director (Foundry-Orchestrierung)
- core/io.js -- IO-Layer für Datei-Ein/Ausgabe
- core/validator.js -- Validator für State & Import-Formate
- core/logger.js -- Zentrale Logging-Helfer

Abhängigkeiten nach außen

- Foundry-API:
  - game.settings (Persistenz)
  - Hooks (Events wie ready, janus7StateChanged, eigene Hooks)
  - fromUuid, Scene, JournalEntry, Playlist, FilePicker etc.
- Module:
  - Keine harte Abhängigkeit zu DSA5 oder DGdZ -- das kommt erst in Phase 3.
- Andere JANUS-Phasen:
  - Phase 2+ verwenden nur die **öffentlichen Interfaces** von core/state, core/config, core/director, core/io, core/validator.

1.3 State Manager V2 (core/state.js)

1.3.1 Aufgabe & Verantwortung

Der **State Manager** ist das Herz von JANUS7:

- Er hält den vollständigen **Kampagnenzustand** (JanusStateCore) im Speicher.
- Er synchronisiert diesen Zustand mit einer **Foundry-Game-Setting** (z. B. („janus7", „state")) im world-Scope.
- Er bietet eine **typsichere, Pfad-basierte API** (get/set/update) für alle anderen Module.
- Er feuert **Hooks**, wenn sich der State ändert (janus7StateChanged).

1.3.2 Datenmodell: JanusStateCore (Schema V7.1.0)

// Pseudo-Typdefinition: JanusStateCore

{

meta: {

version: „7.1.0", // Schema-Version für Migrationen

lastSave: „2025-12-10T18:34Z", // ISO-Timestamp letzte Persistierung

campaignId: „UUID-STRING", // Eindeutige ID dieser Kampagne

worldId: "your-world-id" // Foundry World-ID (für Sicherheit)

},

// ZEIT (Taktung der Kampagne)

time: {

year: 1, // Schuljahr (1--9)

trimester: 1, // Trimester (1--3)

week: 1, // Woche im Trimester (1--12)

day: „Praiosstag", // Wochentag (Enum/String)

phase: „Morgen", // Tagesphase (Morgen, Vormittag, Mittag, Nachmittag, Abend, Nacht)

isHoliday: false, // Flag: Ferien/Feiertag

totalDaysPassed: 0 // Absoluter Zähler (für Dauer, Lernfortschritt etc.)

},

// STUDIENSTAND / AKADEMIE-KONTEXT

academy: {

phaseOfStudy: „Elevium", // Elevium, Novizium, Studium, etc.

threatLevel: 1, // 0--5; 0 = sicher, 5 = akademischer Untergang

currentCurriculumId: null // Verweis auf aktiven Lehrplan (z. B. „curriculum_default")

},

// SCORING (Zirkel/Hauspunkte & Schülerpunkte)

scoring: {

circles: {

salamander: 0,

staves: 0,

swords: 0,

sickles: 0

},

students: {

// z. B. "pc_ardaion": 25, "npc_jakob": -10

},

lastAwarded: \[\] // Historie: { timestamp, source, amount, targetType, targetId, reason }

},

// SOZIAL (Beziehungen, Reputation, Cliquen)

social: {

relations: {

// Key-Schema: '\${sourceId}::\${targetId}': -100 bis +100

// Beispiel: „pc_ardaion::npc_jakob": 40

},

reputation: {

// z. B. "academy": 10, "faculty": 5, "students": 0

},

tags: {

// z. B. „pc_ardaion": \["vorlaut", „mutig"\]

}

},

// STORY & LORE

story: {

threads: \[

// { id, title, status, priority, tags, lastUpdate, summary }

\],

journalRefs: \[

// { id, journalUuid, label, tags }

\],

chronicle: \[

// Zeitlinie: { dayIndex, title, summary, tags }

\]

},

// ÖKONOMIE (Geld, Schulden, Ressourcen)

economy: {

allowance: {

// Taschengeld pro PC: { \[pcId\]: { amount, currency, nextPayoutDay } }

},

debts: \[

// { fromId, toId, amount, currency, reason, createdAt, resolved }

\],

stash: {

// Gruppen-Schatzkammer (z. B. „coins": 45, „gems": 2)

}

},

// DISPLAY & HYBRID-KONTEXT

display: {

beamerMode: "scene", // scene \| dashboard \| visualNovel \| off

activeSceneKey: null, // z. B. "location.library.main"

activeOverlayId: null, // z. B. eine aktuelle Prüfungsfrage

currentMood: „neutral" // Musik-/Atmosphärenstimmung

},

// FLAGS (Temporäre Steuerbits / Overrides)

```
flags: {
todayLessonOverrideId: null, // Erzwingt eine bestimmte Lesson für heute
todayExamOverrideId: null, // Erzwingt eine bestimmte Prüfung
disableAutoMood: false, // Deaktiviert automatische Musikwechsel
debugMode: false // Zusätzliche Logs
}
}
```

Wichtige Design-Entscheidungen:

- Alles ist **JSON-serialisierbar**, damit es sauber über game.settings gesichert werden kann.
- meta.version erlaubt zukünftige **Migrationen** (z. B. V7.1 → V8).
- flags bietet Flexibilität, ohne das Schema ständig aufblähen zu müssen.

1.3.3 Persistenz in Foundry (Settings)

- Registrierung eines world-Settings im ready-Hook, z. B.:

```
game.settings.register("janus7", "state", {
name: "JANUS7 Kampagnenzustand",
scope: "world",
config: false,
type: Object,
default: DEFAULT_JANUS_STATE, // entspricht obigem Schema
onChange: value =\> game.janus7?.core?.state.\_onExternalChange(value)
});
```

Hinweis: scope: „world" darf erst im ready-Hook geschrieben werden.

1.3.4 Öffentliche API (JanusState)

Geplant als game.janus7.core.state:

- init()\

  Lädt State aus game.settings, migriert falls nötig, initialisiert Caches.

- get(path, defaultValue?)\

  Liest Werte per Pfad, z. B. "time.week", "scoring.circles.salamander".

- set(path, value, { silent = false } = {})\

  Schreibt Werte; optional ohne Hook-Feuerung (für Bulk-Operationen).

- update(patchObject, { silent = false } = {})\

  Flache oder tief verschachtelte Updates, z. B.:

- JanusState.update({
- „time.day": „Rohalstag",
- „display.currentMood": „tense"

```{=html}

<!-- -->

```

- });

```{=html}

<!-- -->

```

- transaction(fn)\

  Führt mehrere set/update-Operationen atomar aus, berechnet ein **Diff** und feuert genau einen janus7StateChanged-Hook:

- JanusState.transaction(state =\> {
- state.time.day = „Peraine";
- state.time.totalDaysPassed += 1;

```{=html}

<!-- -->

```

- });

```{=html}

<!-- -->

```

- save({ force = false } = {})\

Persistiert den aktuellen Zustand mit Debounce (z. B. nur alle X Sekunden/Änderungen).

- exportSnapshot()\

Gibt eine tief kopierte, read-only Version des States für KI-Export, Debug-UI usw. zurück.

- reset()\

Setzt den State auf DEFAULT_JANUS_STATE zurück (nur GM, mit Bestätigung).

1.3.5 Ereignisse / Hooks

- janus7StateChanged\

Payload: { state, diff, reason }\

Wird nach jeder erfolgreichen Änderung gefeuert (außer silent).\

→ Listener: UI-Module, Atmosphere, Academy-Logic, KI-Export.

- janus7StateLoaded\

Wird nach init() gefeuert, wenn der State aus game.settings geladen wurde.

1.4 Config & Mapping Layer (core/config.js)

1.4.1 Aufgabe

Die Config-Schicht verwaltet **welt-spezifische Mappings & Optionen**, z. B.:

- logische Keys → Foundry-UUIDs (Scenes, JournalEntries, Playlists)
- Aktivierung/Deaktivierung von JANUS-Features
- Defaultpfade für IO und Assets

Ziel: Kein anderer Teil von JANUS7 muss konkrete IDs kennen.

1.4.2 Datenquelle & Struktur

- **Modul-Config-Datei:** modules/janus7/config/world-config.json (optional)
- **World Setting:** („janus7", „config"), das Mappings übersteuern kann.

Beispiel:

{

"scenes": {

"location.library.main": "Scene.zxv123",

"location.dormitory.pc_room": "Scene.abcd67"

},

"journals": {

"handout.academy.rules": "JournalEntry.xyz890",

"lesson.template.generic": "JournalEntry.def345"

},

"playlists": {

"mood.neutral": "Playlist.aa11bb22",

"mood.tension": "Playlist.33cc44dd"

},

"io": {

"outboxPath": "worlds/\<world-id\>/janus/io/outbox",

"inboxPath": "worlds/\<world-id\>/janus/io/inbox"

},

"features": {

"enableScoring": true,

"enableEconomy": false,

„enableSocialEngine": false

}

}

1.4.3 Öffentliche API (JanusConfig)

- load()\

  Lädt Modul-Defaults + world-spezifische Overrides (Settings/JSON).

- get(path, defaultValue?)\

  z. B. JanusConfig.get("scenes.location.library.main").

- getSceneKey(key) / getJournalKey(key) / getPlaylistKey(key)\

  Convenience-Methoden für Director.

- resolveUUID(keyOrUuid)\

Nimmt entweder direkten UUID-String oder Config-Key und gibt eine gültige Foundry-UUID zurück.

- getFeatureFlag(name)\

  z. B. "enableScoring".

- set(path, value)\

  Ermöglicht weltweite Anpassungen durch GM (wird im Setting gespeichert).

1.4.4 Hooks

- janus7ConfigLoaded -- gefeuert, wenn Konfiguration initial geladen wurde.

1.5 Director (core/director.js)

1.5.1 Aufgabe

Der **Director** ist die Abstraktionsebene zwischen JANUS-Logik und Foundry-Dokumenten:

- Er kennt **Config-Keys** und kann sie in konkrete Befehle an die Foundry-API übersetzen.
- Er bietet **hochlevelige Aktionen**, die UI, KI-Importer oder Simulation einfach nutzen können.

Wichtig: Der Director enthält **keine DSA5-Logik** und keine Akademie-Regeln -- nur „Knöpfe", die man drücken kann.

1.5.2 Öffentliche API (JanusDirector)

Geplant als game.janus7.core.director:

- openScene(key, { pan = true, elevate = true, userScope = „all" } = {})\

  Sucht über JanusConfig die UUID und aktiviert die Szene.\

  userScope: „all" \| „gmOnly" \| „playersOnly" \| „beamer".

- showJournal(key, pageId = null, { asPopup = true } = {})\

  Öffnet Journal/Seite beim GM oder optional für Spieler.

- showHandoutImage(keyOrUrl, options)\

  Entweder über Config-Key (Journal mit Bild oder Scene-BG) oder über direkten URL/Pfad.

- playPlaylist(key, { fade = 500, volume = 1.0, loop = true } = {})\

  Startet/stoppt Playlists, delegiert technische Details an Atmosphere-Layer (Phase 5).

- setMood(moodKey)\

  Z. B. "mood.neutral", "mood.tension", "mood.exam".\

  Aktualisiert state.display.currentMood und ruft playPlaylist.

- focusOnToken(tokenIdOrName, options) **(Backlog innerhalb Phase 1, aber vorbereitbar)**\

  Zentriert Szene auf Token, ggf. Zoom/Highlight.

1.5.3 Integration & Abhängigkeiten

- Nutzt JanusConfig für UUID-Auflösung.
- Nutzt Foundry-Klassen (Scene, JournalEntry, Playlist, fromUuid).
- Wird von:
  - UI (Phase 6),
  - KI-Importer (Phase 7),
  - Academy Logic (Phase 4) genutzt.

1.6 IO-Layer (core/io.js)

1.6.1 Aufgabe

Der IO-Layer ist die **einzige offizielle Schnittstelle** von JANUS7 zur Außenwelt (KI-Tools, Python-Scripts etc.):

- Schreibt **Exports** in io/outbox.
- Liest **Imports** aus io/inbox.
- Kapselt Dateipfade, Namenskonventionen und eventuelles Security-Handling.

1.6.2 Verzeichnisstruktur (Standard)

- modules/janus7/io/outbox/
  - state_snapshot.json
  - context_export.json
  - dsa5_index.json (aus Phase 3)
  - knowledge_*.json (z. B. Bibliothek, Lektionen)
- modules/janus7/io/inbox/
  - world_build.json
  - npc_update.json
  - schedule_update.json

- job_*.json (generischer Job-Mechanismus)

Paths können über JanusConfig.io überschrieben werden.

1.6.3 Öffentliche API (JanusIO)

- getOutboxPath() / getInboxPath()
- writeOutbox(filename, data)\

  Serialisiert data als JSON und speichert es in der Outbox.

- readInbox(filename)\

  Liest und parsed JSON aus der Inbox.

- deleteInboxFile(filename)\

  Säubert verarbeitete Dateien.

- listInbox(pattern?)\

  Listet Dateien (z. B. job_*.json).

- watchInbox(callback)\

  Optionale Polling-Logik (oder Hook via externem Script):\

  callback(filename, data) → importierende Module können darauf reagieren.

1.6.4 Hooks

- janus7FileReceived\

  Wird gefeuert, wenn watchInbox eine neue Datei erkennt.\

  Payload: { filename, data }.

1.7 Validator (core/validator.js)

1.7.1 Aufgabe

Der Validator ist der **Türsteher** für:

- Importierte KI-Daten (world_build.json, npc_update.json, ...)
- Patch-Beschreibungen (z. B. Änderungen am State)
- Optionale Validierung des State selbst (JanusStateCore)

Ziel: Keine KI- oder Tool-Daten dürfen den World-State zerstören oder inkonsistent machen.

1.7.2 Validierungsziele

- **Strukturvalidierung**\

-- Pflichtfelder, Typen, erlaubte Werte (z. B. Tag-Enums, ID-Formate).

- **Referenzvalidierung**\

  -- existieren NPC-IDs, Szenen-Keys, Journale etc.?

- **Semantische Logik**\

  -- z. B. time.year in 1--9, threatLevel in 0--5, Scoring-Werte sind Zahlen.

1.7.3 Öffentliche API (JanusValidator)

- validateState(state) → { valid, errors }
- validateImport(importData, schemaType)\

  z. B. schemaType = „JANUS_IMPORT_V1".

- assertValidState(state)\

  Wirft Exception und loggt Fehler, wenn State ungültig ist.

- registerSchema(name, schemaDefinition)\

  erlaubt zukünftige Erweiterung ohne Codeänderung.

1.7.4 Implementierungsoption

- Entweder leichtgewichtige, eigene Checks
- oder Integration eines JSON-Schema-Validators (z. B. ajv), falls du Lust auf mehr Formalität hast.

1.8 Logging & Debugging (core/logger.js)

Kleines, aber wichtiges Stück Infrastruktur:

- Level: debug \| info \| warn \| error
- Ausgabe über console + optional Foundry-Benachrichtigungen (ui.notifications).
- API z. B.:

```
JanusLog.debug("State updated", diff);

JanusLog.warn("Missing config for scene key", { key });

JanusLog.error("Invalid import file", { filename, errors });
```

Später kann ein Debug-Panel (ApplicationV2) diese Logs anzeigen.

1.9 Aufwandsschätzung Phase 1

Grober Rahmen, 1 erfahrener Dev (du) + gelegentliche KI-Unterstützung:

22. **Datenmodell finalisieren & dokumentieren**\

    -- JanusStateCore + Config-Schema\

→ 1,0--1,5 PT

23. State Manager implementieren (inkl. Migration & Hooks)\

→ 2,0--3,0 PT

24. Config-Layer implementieren (inkl. Settings & JSON)\

→ 1,0--1,5 PT

25. Director-API (Basisfunktionen)\

→ 1,0 PT

26. IO-Layer (lesen/schreiben, Basis-Watcher)\

→ 0,5--1,0 PT

27. Validator (Basis-Schemas, State + 1--2 Import-Typen)\

→ 1,5--2,0 PT

28. Tests & Feinschliff (Unit-ähnlich + 1--2 Live-Tests im DGdZ-World)\

→ 1,0--1,5 PT

Gesamt: ca. **8--11 Personentage**\

(je nach Strenge bei Tests & Validator-Formalisierung).

1.10 Risiken & Gegenmaßnahmen

29. Risiko: Schema ändert sich später häufig

- *Folge:* Viele Migrationsroutinen, kaputte alte Saves.
- Gegenmaßnahme:
  - Phase 1 bewusst „überdimensionieren": lieber ein paar meta-/flags-Felder mehr einplanen.
  - meta.version konsequent nutzen, Migrationen zentral halten.

30. Risiko: Doppel-Persistenz (State vs. andere Module)

- *Folge:* Inkonsistenzen, wenn z. B. DSA5-Module auch eigenen State speichern.
- Gegenmaßnahme:
  - Klare Regel: JANUS7 speichert **nur seinen eigenen** State.
  - DSA5-Module werden gelesen, aber ihre Daten nicht dupliziert (nur referenziert).

31. Risiko: Performance bei großen States

- *Folge:* UI-Lag, Speichern dauert, Hooks feuern zu oft.
- Gegenmaßnahme:
  - Debounce in save() (z. B. max. alle X Sekunden).
  - transaction() nutzen, nicht zig einzelne set-Aufrufe.
  - State bewusst kompakt halten, große Textblöcke nicht hier speichern (sondern in Journals).

32. Risiko: Validator zu lax oder zu streng

- *Folge:* Entweder gehen fehlerhafte Daten durch, oder sinnvolle Änderungen werden geblockt.
- Gegenmaßnahme:
  - Start mit Minimal-Schemas + Logging der tatsächlichen Importdaten.
  - Nachschärfen, wenn du reale Nutzungsfälle gesehen hast.

1.11 Umsetzungsschritte (strukturierter Plan)

Schritt 1 -- Datenmodell & Contracts finalisieren

- JanusStateCore-Schema in einem separaten SCHEMA.md festhalten.
- State-Pfad-Konventionen definieren (z. B. time.phase, scoring.circles.*).

Schritt 2 -- State Manager (core/state.js)

- DEFAULT_JANUS_STATE implementieren.
- Laden/Speichern via game.settings (world-Scope) umsetzen.
- API get/set/update/transaction/save/exportSnapshot/reset implementieren.
- Hooks janus7StateLoaded, janus7StateChanged definieren.

Schritt 3 -- Config Layer (core/config.js)

- Default-Config-JSON anlegen (z. B. config/world-config-default.json).
- Lesen + Zusammenführen mit world-Settings.
- APIs get, getSceneKey, getPlaylistKey, getFeatureFlag, resolveUUID implementieren.

Schritt 4 -- Director (core/director.js)

- Grundfunktionen openScene, showJournal, playPlaylist, setMood implementieren.
- Nur JanusConfig + Foundry-API nutzen, **kein** Zugriff auf DSA5-Spezifika.

Schritt 5 -- IO-Layer (core/io.js)

- Pfade aus Config lesen.

- writeOutbox, readInbox, listInbox, deleteInboxFile implementieren.
- Einfaches watchInbox-Polling + Hook janus7FileReceived bauen.

Schritt 6 -- Validator (core/validator.js)

- Mini-Schemas für:
  - JanusStateCore (nur Grundchecks),
  - JANUS_IMPORT_V1 (wenn du es schon definierst),
  - 1--2 einfache Patch-Formate.
- API validateState, validateImport, assertValidState implementieren.

Schritt 7 -- Logging-Helfer (core/logger.js)

- Basic-Wrapper um console + ui.notifications.
- Im State/Config/IO/Validator einbinden.

Schritt 8 -- Integrationstest in deiner DGdZ-World

- Modul aktivieren, JanusStateCore initialisieren.
- Zeit verändern, speichern, reloaden, prüfen.
- Erste einfache openScene/showJournal-Calls über die Konsole testen.

Damit ist **Phase 1** so detailliert beschrieben, dass du:

- ein Kapitel in dein Roadmap-Word-Dokument übernehmen kannst
- und direkt mit der Implementierung starten kannst, ohne dass wir später an den Grundpfeilern rütteln müssen.

Im nächsten Schritt können wir -- wenn du magst -- **Phase 2 (Static Academy Data)** genauso tief zerlegen: JSON-Schemas für Kalender, Lessons, Exams, NPC-Datenbanken, Bibliothek etc.

Phase 2 – Static Academy Data (Detailliert)

Phase 2: Static Academy Data -- Die Welt als Daten -- Erweiterte Dokumentation

Ziel: Die strukturelle Definition der Spielwelt (Zeit, Ort, Lehrplan) als reine Daten, getrennt von der Programmlogik. Dies ermöglicht maximale Flexibilität, einfache Anpassbarkeit und eine klare Trennung zwischen dem „Was" (Inhalt) und dem „Wie" (Code).

Module & Funktionen

2.1 Calendar Definition (data/academy/calendar.json)

Der Kalender bildet das zeitliche Rückgrat der Kampagne. Er definiert nicht nur die Abfolge von Tagen, sondern auch die akademische Struktur des Schuljahres.

- Inhalt:

  - **Wochenstruktur:** Definition der Wochentage (z.B. Praiosstag bis Rahjatag) und deren Bedeutung im akademischen Kontext (z.B. Praiosstag als Ruhetag oder Hauptunterrichtstag).

  - **Tagesphasen:** Unterteilung des Tages in relevante Abschnitte (Morgen, Vormittag, Mittag, Nachmittag, Abend, Nacht) für die Planung von Unterricht, Freizeit und Ereignissen.

  - **Semestereinteilung:** Definition von Trimestern oder Semestern, inklusive Start- und Endwochen.

  - **Feiertage & Ferien:** Liste fester Feiertage (z.B. Namenlose Tage, religiöse Feste) und Ferienzeiten, in denen kein regulärer Unterricht stattfindet. Diese können auch spezielle Ereignisse triggern.

  - **Jahreszählung:** Startjahr der Kampagne und aktuelle Jahreszählung.

2.2 Lesson Database (data/academy/lessons.json)

Diese Datenbank enthält den gesamten Pool aller möglichen Unterrichtseinheiten. Sie ist das Herzstück des akademischen Alltags.

- **Inhalt:** Jede Lektion ist ein strukturierter Datensatz mit folgenden Informationen:

  - **ID:** Eindeutige Kennung der Lektion.

  - **Titel & Thema:** Name der Lektion und spezifisches Thema (z.B. „Magietheorie: Die 7 Sphären").

  - **Lehrer:** Referenz auf den unterrichtenden NPC (NPC-ID).

  - **Ort:** Referenz auf den Ort des Unterrichts (Location-ID, z.B. „Hörsaal Ost").

- **Anforderungen:** Bedingungen für die Teilnahme (z.B. „Nur für Jahrgang 1", „Benötigt Talent ‚Magiekunde' 4+").
- **Dauer:** Anzahl der benötigten Tagesphasen.

- **Referenzen:** Enthält Verweise auf DSA-Module (z.B. „Siehe Buch ‚Aventurische Magie', Seite X", „Nutzt Regelmechanik aus ‚Wege der Zauberei'"). Dies ermöglicht dem Spielleiter schnellen Zugriff auf relevante Regeltexte.

2.3 Exams (data/academy/exams.json)

Prüfungen sind Meilensteine im Leben eines Schülers. Diese Datei definiert, wann und wie sie stattfinden.

- Inhalt:
  - **Prüfungsart:** Schriftlich, mündlich oder praktisch.
  - **Anforderungen:** Welche Talente oder Zauber werden geprüft? Welche Mindestwerte sind erforderlich?
  - **Schwierigkeitsgrad:** Modifikatoren für Proben.
  - **Belohnungen/Konsequenzen:** Was passiert bei Erfolg (z.B. Hauspunkte, AP, Freischaltung neuer Kurse) oder Misserfolg (z.B. Nachsitzen, Punktabzug, Wiederholung)?
  - **Terminierung:** Wann im Semester findet die Prüfung statt (z.B. „Letzte Woche des 1. Trimesters")?

2.4 Calendar Logic (academy/calendar.js)

Die Calendar Logic ist die Business-Logik, die die statischen Daten aus den JSON-Dateien zum Leben erweckt und mit dem aktuellen Spielstatus verknüpft.

- Funktion:
  - **Zeitberechnung:** Berechnet das aktuelle Datum und die Uhrzeit basierend auf dem State.
  - **Stundenplan-Generierung:** Ermittelt für einen gegebenen Zeitpunkt (z.B. „Heute Vormittag"), welcher Unterricht oder welches Ereignis stattfindet. Dabei werden Feiertage, Ferien und individuelle Stundenpläne der Schüler berücksichtigt.
  - **Übersetzung:** Rechnet abstrakte Zeitangaben wie „Jahr 1, Woche 3, Dienstag, Vormittag" in konkrete Spielsituationen um (z.B. „Unterricht bei Magister X im Raum Y").
  - **Fortschrittskontrolle:** Überwacht den Ablauf von Zeiträumen (z.B. Dauer eines Trimesters) und triggert entsprechende Ereignisse (z.B. „Prüfungswoche beginnt").

🔗 Abhängigkeiten & Schnittstellen

Abhängigkeiten (Benötigt\...)

Damit die Akademie-Daten sinnvoll genutzt werden können, ist eine enge Verzahnung mit dem Core-System notwendig.

- Phase 1 (Core):

  - Nutzt State für das aktuelle Datum, um den richtigen Kontext für Kalenderabfragen zu haben.

  - Nutzt Config, um logische Orte (z.B. „Raum Y") in konkrete Foundry-Szenen (UUIDs) aufzulösen, damit bei Unterrichtsbeginn die richtige Szene geladen werden kann.

Schnittstellen (Bietet an\...)

Die Akademie-Daten stellen Informationen und Funktionen für andere Module bereit.

- Calendar API:

  - Calendar.getLessonsForToday(): Liefert den kompletten Stundenplan für den aktuellen Tag, gefiltert nach relevanten Gruppen oder Spielern.

  - Calendar.isHoliday(): Prüft, ob der aktuelle Tag ein Feiertag oder ferienfrei ist.

  - Calendar.advanceTime(delta): Spult die Zeit in der Kampagne vor (z.B. „+1 Tag", „+1 Woche") und aktualisiert den State entsprechend.

  - Calendar.getCurrentPhase(): Gibt die aktuelle Tagesphase zurück.

- Data Access:

  - AcademyData.getLesson(id): Liefert die Details einer spezifischen Lektion anhand ihrer ID.

  - AcademyData.getExam(id): Liefert die Details einer spezifischen Prüfung anhand ihrer ID.

  - AcademyData.getNPC(id): Liefert Informationen zu einem Lehrer oder Mitschüler (falls in den Akademie-Daten definiert).

- Hooks:

  - janus7DateChanged: Wird gefeuert, wenn das Datum oder die Uhrzeit im Spiel voranschreitet. UI-Elemente und andere Logik können darauf reagieren.

  - janus7LessonStarted: Wird gefeuert, wenn eine Unterrichtseinheit beginnt. Kann genutzt werden, um Szenen zu wechseln, Musik zu starten oder Nachrichten anzuzeigen.

- janus7ExamStarted: Wird gefeuert, wenn eine Prüfung beginnt.

ChatGTP:

Phase-2-Zeit. Jetzt wird aus deinem ganzen Lore-Berg endlich eine saubere Daten-Schicht.

Ich schreibe dir Phase 2 wieder so, dass du das direkt als Word-Kapitel übernehmen kannst -- konsistent mit Phase 0 und 1, und anschlussfähig an Foundry-Best Practices (statische JSON im Modul, Persistenz über Settings/Flags, keine wilden Hacks in .db-Dateien).

Phase 2 -- Static Academy Data

Die Akademie als Datenuniversum

2.1 Ziel & Scope der Phase

Ziel von Phase 2:\

Alle **statischen Inhalte** der Akademie (Stundenplan, Lektionen, Prüfungen, Orte, NPCs, Bibliothek etc.) werden in **klar definierten JSON-Strukturen** abgelegt und über eine konsistente Loader-Schicht (AcademyData) bereitgestellt.

- Die Daten sind:
  - **System-agnostisch** (noch keine DSA5-Regelmechanik),
  - **Code-agnostisch** (keine Logik in den JSONs),
  - **editierbar**, damit du Lehrpläne, Stundenpläne, NPC-Beziehungen etc. ohne Code umbauen kannst.
- Phase 2 definiert auch die **Schnittstellen**, über die spätere Phasen (Kalender-Logik, DSA5-Bridge, KI-Import, UI) auf diese Daten zugreifen.

Im Scope:

- Verzeichnisstruktur modules/janus7/data/academy/\\
- JSON-Schemas für:
  - calendar.json
  - lessons.json
  - exams.json
  - npcs.json
  - locations.json
  - library.json

- optional: events.json (Feiertage / besondere Akademie-Ereignisse)

- Loader-/Service-Modul: academy/data.js (bzw. academy/academy-data.js) mit der API AcademyData.*

Nicht im Scope von Phase 2:

- Keine Mutationen am **State** (das ist Phase 1 & 4).
- Keine Entscheidungslogik (z. B. *„welche Stunde findet heute statt?"*) → Phase 4 (Academy Simulation Logic).
- Keine DSA5-Regelanwendung (Talente, Proben) → Phase 3 (DSA5 System Bridge).
- Keine UI (ApplicationV2) → Phase 6.
- Keine KI-Exporte/Importe → Phase 7 (nutzt die Daten aber).

2.2 Modul- und Verzeichnisstruktur

Vorgeschlagene Struktur innerhalb des Moduls:

modules/janus7/

data/

academy/

calendar.json

lessons.json

exams.json

npcs.json

locations.json

library.json

events.json (optional, aber sinnvoll)

scripts/

academy/

academy-data.js (Loader & Query-API)

Statische Daten liegen **als Dateien im Modul** (nicht als Compendium-DB) -- das ist stabiler und einfacher mit Git/ZIP/KI-Export zu versionieren. Compendien nutzt du weiterhin dort, wo Foundry Dokumente zwingend will (z. B. fertige Journals).

2.3 AcademyData -- Rolle & Schnittstellen

AcademyData ist dein Read-only Katalog-Service:

- Lädt beim Start alle JSON-Dateien aus data/academy/.

- Validiert die Daten gegen einfache Schemas (Validator aus Phase 1).
- Bietet eine **stabile API**, um Kalender, Lektionen, Prüfungen, NPCs, Orte und Bibliothek abzufragen.

Grundprinzip:
Phase 2 beschreibt *was existiert*. Phase 4 entscheidet *was wann passiert*.

2.3.1 Öffentlicher Namespace
Geplant als game.janus7.academy.data mit z. B. folgenden Methoden:

- Kalender:
  - AcademyData.getCalendarEntryByDay(dayRef)
  - AcademyData.findCalendarEntries(query)
- Lektionen:
  - AcademyData.getLesson(id)
  - AcademyData.listLessonsByTag(tag)
  - AcademyData.listLessonsByTeacher(npcId)
- Prüfungen:
  - AcademyData.getExam(id)
  - AcademyData.listExamsByLesson(lessonId)
- NPCs:
  - AcademyData.getNPC(id)
  - AcademyData.listNPCs(filter)
- Orte:
  - AcademyData.getLocation(id)
  - AcademyData.listLocations(filter)
- Bibliothek:
  - AcademyData.getLibraryItem(id)
  - AcademyData.searchLibrary(query)
- Events:
  - AcademyData.listEventsForDay(dayRef)

- AcademyData.getEvent(id)

Die **Day-Referenz** (dayRef) wird an das Zeitmodell aus Phase 1 angelehnt (z. B. { year, trimester, week, day, phase } oder eine abgeleitete dayIndex-Notation).

2.4 Datenmodelle im Detail

####### 2.4.1 calendar.json -- Akademiekalender

Der Kalender beschreibt, **was an welchem Tag/Slot geplant ist**, ohne Logik.

Grundstruktur:

{

„meta": {

„version": „1.0.0",

„description": „Kalender der Akademie Punin für Jahre 1-3",

"generatedFrom": \[

"SCHEDULE.md",

"SCHEDULE_V2_ERWEITERT.md"

\]

},

"entries": \[

{

"id": "y1_t1_w1_praios_morning",

"year": 1,

"trimester": 1,

"week": 1,

"day": "Praiosstag",

"phase": "Morgen",

"type": "lesson-slot", // lesson-slot \| exam-slot \| event-only \| holiday

"lessonId": "LES_MAG_BASICS_01",// optional

"examId": null, // optional

"eventId": null, // optional

"holidayKey": null, // z. B. "Perainefest"

„notes": „Erste Einführungsstunde in Magiekunde"

}

\]

}

Designpunkte:

- Kalender **beherrscht**:
  - reguläre Stundenblöcke,
  - Prüfungsblöcke,
  - Ferien/Feiertage,
  - besondere Ereignisse (z. B. Schulball, Akademie-Krise).
- type ist wichtig für die spätere Logik in Phase 4:
  - z\. B. holiday → Calendar.isHoliday() = true.
- meta.generatedFrom kann auf deine Referenzdokumente verweisen:
  - SCHEDULE.md, SCHEDULE_V2_ERWEITERT.md, ZAUBER_LEHRPLAN.md etc.

API-Kopplung an Phase 4:

- Phase 4 implementiert:
  - Calendar.getLessonsForToday() → nutzt AcademyData.getCalendarEntryByDay() + getLesson().
  - Calendar.isHoliday() → prüft type === "holiday" oder holidayKey != null.
  - Calendar.advanceTime(delta) → nutzt das Zeitmodell aus Phase 1, nicht den Kalender selbst.

####### 2.4.2 lessons.json -- Lektionenkatalog

Lektionen sind **wiederverwendbare Unterrichtseinheiten**, die der Kalender verlinkt.

Grundstruktur:

{

„meta": {

„version": „1.0.0",

„source": \[
„ZAUBER_LEHRPLAN.md",
„LESSON_GENERATOR.md",
„DAEMONOLOGIE_UNTERRICHT.md",
„ELEMENTAR_VERWANDLUNG.md"
\]
},
„lessons": \[
{
„id": „LES_MAG_BASICS_01",
„name": „Grundlagen der Magietheorie I",
"subject": "Magiekunde",
"category": "lecture", // lecture \| practice \| seminar \| lab
"yearRange": \[1, 1\],
"phaseOfStudy": "Elevium",
"teacherId": "NPC_PROF_XYZ", // Referenz auf npcs.json
"locationId": "LOC_CLASSROOM_A", // Referenz auf locations.json
„tags": \["magie", „theorie", „einstieg"\],
„summary": „Einführung in die grundlegenden Konzepte der arkanen Theorie.",
„learningGoals": \[
„Schüler kennen die Begriffe Astralkraft und Strukturgeber.",
„Schüler verstehen die fünf Antimagie-Paradigmen grob."
\],
„references": {
„aventurica": \["REGELWERKE_REFERENZ#MagiekundeBasis"\],
"libraryItems": \["LIB_BOOK_ARCANA_01"\],

"dsA5Rules": \[

{

"moduleId": "dsa5-magic-1",

"pack": "magic1spells",

„entryName": „Analys Arcanstruktur"

}

\]

},

„difficulty": 1, // 1-5

„defaultDurationSlots": 1, // Anzahl Phasen (Morgen/Nachmittag)

„examIds": \["EX_MAG_BASICS_01"\], // passende Prüfungen

„flags": {

„optional": false

}

}

\]

}

Wichtige Aspekte:

- teacherId, locationId, examIds verknüpfen Lektionen mit anderen Daten-JSONs.
- references.dsA5Rules ist bewusst **nur referenziell** -- DSA5-Bridge (Phase 3) löst diese in echte Items/Kompendien auf.
- learningGoals und summary sind super Futter für KI und Handouts.

####### 2.4.3 exams.json -- Prüfungskatalog

Prüfungen sind strukturierte Bausteine, die:

- im Kalender (calendar.json) referenziert werden,
- von der Exam-Logik (Phase 4) ausgelesen,
- von DSA5-Bridge (Phase 3) in Proben übersetzt,
- von KI (Phase 7) als Template erweitert werden können.

Grundstruktur:

{

„meta": {

„version": „1.0.0",

„source": \["DAEMONOLOGIE_UNTERRICHT.md", „ZAUBER_LEHRPLAN.md"\]

},

"exams": \[

{

"id": "EX_MAG_BASICS_01",

„name": „Zwischenprüfung Magietheorie I",

„lessonId": „LES_MAG_BASICS_01",

"type": "written", // written \| oral \| practical \| mixed

„durationSlots": 2,

„difficulty": 2, // 1-5 (pädagogische Schwierigkeit)

„gradingScheme": {

"scale": "QS", // QS \| Prozent \| Schulnoten

"passThreshold": 2,

"excellentThreshold": 4

},

„mechanics": {

„skills": \[

{

„label": „Magiekunde",

„systemSkillId": „TALENT_MAGIEKUNDE", // DSA5-Bridge löst das

"weight": 0.6

},

{

"label": "Klugheit",

"systemSkillId": "ATTRIBUTE_KLUGHEIT",

"weight": 0.4

}

\]

},

"scoringImpact": {

"onPass": { "circlePoints": 5, "studentPoints": 2 },

"onFail": { "circlePoints": -2, "studentPoints": -1 }

},

„notes": „Erste ernsthafte Prüfung im Elevium."

}

\]

}

Kopplung zu späteren Phasen:

- Phase 4: Exams.startExam(id), Exams.evaluateResult(result) nutzt diese Definition.
- Phase 3: mappt mechanics.skills\[*\].systemSkillId auf konkrete DSA5-Talente/Attribute.
- Phase 2: bleibt komplett datengetrieben, keine Logik.

####### 2.4.4 npcs.json -- NPC-Datenbank der Akademie

Diese Datei legt **kanonische Akademie-NSCs** fest, getrennt von Foundry-Actors (die können via UUID referenziert werden).

Grundstruktur:

{

„meta": {

„version": „1.0.0",

„source": \[

„NPC_DATENBANK_AKADEMIE_V5_KANONISCH.md",

„NPC_PROFILES.md",

„NPC_PROFILE_WELTENWISSEN_LEHRER.md"

\]

},

"npcs": \[

{

"id": "NPC_PROF_ARCANA",

"name": "Meisterin Salpicon Savertin",

"role": "teacher", // teacher \| student \| staff \| guest

„subjectSpecialties": \["Magietheorie", „Sphärenkunde"\],

„phaseOfStudy": null, // nur für Schüler

"circleAffiliation": null, // z. B. "salamander"

„tags": \["strenge Lehrkraft", „loewe"\],

„shortDescription": „Strenge, aber faire Dozentin mit Fokus auf theoretische Grundlagen.",

"relationships": \[

{

"targetId": "NPC_REKTOR",

"type": "colleague",

"intensity": 60

}

\],

"foundry": {

"actorUuid": "Compendium.dsa5-soms.actors.XYZ123",

"tokenPrototypeUuid": "Compendium.dsa5-soms.actors.XYZ123#Token.ABC"

},

„flags": {

„plotCritical": true

}

}

\]

}

Hinweise:

- foundry.actorUuid bindet an das offizielle DGdZ-Modul bzw. andere DSA5-Module.
- Beziehungen sind bewusst leichtgewichtig, die eigentliche Social-Engine sitzt in Phase 4/State.

####### 2.4.5 locations.json -- Akademie-Orte

Definiert Räume, Gebäude, Außenbereiche, jeweils mit Foundry-Anbindung.

{

„meta": {

„version": „1.0.0",

„source": \["ORTSDATENBANK_AKADEMIE_PUNIN_V1.md"\]

},

"locations": \[

{

"id": "LOC_CLASSROOM_A",

"name": "Hörsaal A",

"building": "Hauptgebäude",

"floor": 2,

"type": "classroom", // classroom \| lab \| library \| dorm \| outdoor \| special

„tags": \["theorie", „standard"\],

„description": „Ein mittelgroßer Hörsaal mit schwerem Holzgestühl.",

"foundry": {

"sceneKey": "location.classroom.a", // in Config/Scenes gemappt

"sceneUuid": null,

"journalKey": "handout.location.classroom.a"

},

"defaultMoodKey": "mood.neutral",

"travelTimeMinutes": 5

}

\]

}

Kopplung:

- Lektionen nutzen locationId.
- Director/Atmosphere (Phase 5) nutzt defaultMoodKey, foundry.sceneKey.

####### 2.4.6 library.json -- Bibliothek & Wissensquellen

Hier fließen BIBLIOTHEK_KATALOG.md, GEOGRAPHIE_AVENTURIEN.md, GESCHICHTE_AVENTURIENS.md, KOSMOLOGIE_SPHAEREN.md, REGELWERKE_REFERENZ.md zusammen -- aber modular.

{

"meta": {

"version": "1.0.0",

"source": \[

"BIBLIOTHEK_KATALOG.md",

„GEOGRAPHIE_AVENTURIEN.md",

„GESCHICHTE_AVENTURIENS.md",

„KOSMOLOGIE_SPHAEREN.md",

„REGELWERKE_REFERENZ.md"

\]

},

"items": \[

{

"id": "LIB_BOOK_ARCANA_01",

"title": "Grundlagen der Sphärenkunde",

"type": "book", // book \| scroll \| map \| compendium \| lecture-notes

"category": "Sphärenkunde",

"tags": \["kosmologie", "theorie"\],

„difficulty": 2, // wie schwer zu verstehen/auszuwerten

„summary": „Ein Überblick über die Sphärenstruktur Aventuriens.",

"knowledgeHooks": \[

{

"topic": "Sphärenstruktur",

"relatedSkillId": "TALENT_SPHAERENKUNDE",

"synergyLessons": \["LES_SPH_BASIC_01"\]

}

\],

"foundry": {

"journalKey": "library.sphaerenkunde.01",

"journalUuid": null

}

}

\]

}

Geplanter Nutzen:

- Akademische Szenen, in denen Schüler in der Bibliothek recherchieren.
- KI-Erweiterungen (neue Bücher, Randnotizen).
- DSA5-Probenlogik (Phase ¾) kann hier Referenzboni etc. ziehen.

####### 2.4.7 events.json -- Besondere Ereignisse (optional, sehr nützlich)

{

"meta": {

"version": "1.0.0"

},

"events": \[

{

"id": "EVT_SCHOOL_FEST_01",

"name": "Akademiefest des 1. Jahres",

"type": "social", // social \| crisis \| exam-related \| holiday

„tags": \["feier", „schüler"\],

„summary": „Ein Fest zu Ehren der neuen Elevi.",

"locationId": "LOC_COURTYARD",

"relatedStoryThreads": \["THREAD_ELEVES_INTRO"\],

"calendarRefs": \[

{

"year": 1,

"trimester": 1,

"week": 4,

"day": "Firunstag",

„phase": „Abend"

}

\]

}

\]

}

Kalender kann eventId referenzieren; alternativ können Events eine Liste geplanter Zeitpunkte enthalten.

2.5 Schnittstellen zu anderen Phasen

Phase 1 (Core & State)

- Phase 2 liest nur (JanusStateCore.time → DayRef) aber modifiziert den State nicht.

- AcademyData.getCalendarEntryByDay() nutzt Zeit aus State und liefert statische Infos.

Phase 3 (DSA5 System Bridge)

- Nutzt Felder wie mechanics.skills\[*\].systemSkillId, references.dsA5Rules aus Lessons/Exams, um DSA-Talente/Zauber etc. zu finden.

Phase 4 (Academy Simulation Logic)

- Implementiert Calendar.getLessonsForToday, Exams.startExam, SocialEngine.applyLessonEffects etc.
- Verwendet ausschließlich AcademyData.* + JanusStateCore, ändert aber weder JSON noch Config.

Phase 5 (Hybrid & Atmosphere Controller)

- Nutzt locations.json (defaultMoodKey, foundry.sceneKey) und evtl. events.json als Trigger für Musikwechsel.

Phase 6 (User Interfaces)

- UIs bauen sich Listen/Dropdowns etc. über AcademyData (z. B. Lesson-Browser, NPC-Kartei).

Phase 7 (KI-Integration)

- Exporte können academyDataSnapshot enthalten (teilweise oder vollständig).
- KI-Patches für neue Lektionen, Prüfungen, NPCs etc. werden über Validator + IO wieder in diese JSON-Strukturen eingespielt (oder in separate „Custom"-Dateien, z. B. lessons_custom.json).

2.6 Aufwandsschätzung Phase 2

Richtgröße (du + KI, Content teilweise schon in MD-Form vorhanden):

33. Schema-Design & Dokumentation

    - Ausarbeiten der finalen JSON-Schemas (wie oben)\

      → 1--2 PT

34. Umsetzung academy-data.js (Loader & Query-API)

    - Dateien laden, mergen (z. B. *_custom.json), simple Query-Funktionen\

      → 1,5--2,5 PT

35. Migration/Überführung der bestehenden Referenztexte

- Mapping von SCHEDULE*.md, ZAUBER_LEHRPLAN.md, NPC-Profile etc. in JSON
- Kann stark von KI unterstützt werden, aber Nacharbeit händisch\

  → 3--5 PT (abhängig davon, wie detailliert du alles einpflegen willst)

36. Validator-Integration (Phase-1-Validator auf diese JSONs anwenden)\

→ 1--1,5 PT

37. Tests in einer Beispiel-World

- Konsistenzchecks (alle referenzierten IDs vorhanden, keine Waisen)
- Erste Abfragen über Konsole (z. B. AcademyData.getLesson(„...”))\

  → 1 PT

Gesamt: ca. **7,5--12 PT**, je nachdem, wie viel Lore du direkt in JSON gießt und wie sehr du dich auf KI-Vorarbeit stützt.

2.7 Risiken & Gegenmaßnahmen

38. Risiko: Datenexplosion / Unübersichtlichkeit

- *Problem:* Zu viele Dateien, zu viele Felder; niemand blickt durch.
- Gegenmaßnahmen:
  - Strikte Trennung nach Domänen (calendar, lessons, exams, npcs, locations, library, events).
  - Kurze, kommentierte SCHEMA-Sektion in einem separaten Dev-Dokument (oder in den JSON-Metadaten).

39. Risiko: Inkonsistente IDs & Referenzen

- *Problem:* Tippfehler in lessonId, npcId, locationId etc. → Laufzeitfehler.
- Gegenmaßnahmen:
  - Validator prüft referenzielle Integrität (z. B. „Existiert teacherId in npcs.json?”).
  - Build-/Test-Script („Consistency Check”) vor Sessionstart.

40. Risiko: Bindung an konkrete DSA5-Version/Module

- *Problem:* systemSkillId oder moduleId veraltet, wenn sich Module ändern.
- Gegenmaßnahmen:

- DSA5-spezifische Hinweise in references.dsA5Rules & mechanics nur referenziell, Auflösung durch Phase 3.
- Optionales Mapping in REGELWERKE_REFERENZ.md konsistent halten.

41. Risiko: KI-Updates zerstören manuell gepflegte Daten

- *Problem:* KI schreibt generisch über deine handverlesenen Lektionen drüber.
- Gegenmaßnahmen:
  - Trennung: lessons_core.json (kanonisch) vs. lessons_custom.json (KI-/Homebrew).
  - Loader merged, aber überschreibt „core" nur, wenn du es bewusst willst.

2.8 Umsetzungsschritte (strukturierter Plan)

Schritt 1 -- Schemas finalisieren

Aus diesem Dokument und deinen Referenz-MDs (SCHEDULE, ZAUBER_LEHRPLAN, NPC-DB etc.) die finalen JSON-Schemas in einem ACADEMY_DATA_SCHEMA.md festhalten.

IDs und Namenskonventionen (z. B. LES_\, *EX*_\, NPC_\, *LOC*_\, LIB_\, *EVT*_\) definieren.

Schritt 2 -- Leere JSON-Skeletons anlegen

calendar.json, lessons.json, exams.json, npcs.json, locations.json, library.json, events.json mit:

- meta-Block,
- leeren Arrays (entries, lessons, ...),
- 1--2 Beispiel-Objekten als Template.

Schritt 3 -- academy-data.js implementieren

JSON-Load (über fetch aus modules/janus7/data/\... oder via game.modules.get(„janus7").path).

AcademyData-API (get/list/search).

Fehlerhandling + Logging (JanusLog aus Phase 1).

Schritt 4 -- Validator-Regeln ergänzen

In core/validator.js:

- Schemas für jede Datei registrieren (academy.calendar, academy.lessons, ...).
- Konsistenzchecks implementieren (z. B. referenzielle Integrität).

Schritt 5 -- Datenmigration aus deinen MD-Referenzen

Per KI-Unterstützung JSON-Chunks erzeugen (z. B. Lektionen aus LESSON_GENERATOR.md, ZAUBER_LEHRPLAN.md).

Manuell stichprobenartig gegenchecken (Inhalt, IDs, Verlinkung).

Schrittweise in die JSONs einpflegen.

Schritt 6 -- Integrations-Smoke-Test

Foundry-World starten, janus7 aktivieren.

In der Browserkonsole:

- game.janus7.academy.data.getLesson("LES_MAG_BASICS_01")
- game.janus7.academy.data.getCalendarEntryByDay(\...)

Prüfen, ob:

- JSON sauber geladen,
- Validator keine Fehler wirft,
- Referenzen (teacherId/locationId) auf existierende Einträge zeigen.

Schritt 7 -- Übergabe an Phase 3 & 4

Kurze Schnittstellenbeschreibung:

- Welche Felder erwartet die DSA5-Bridge (Phase 3)?
- Welche Felder erwartet die Academy-Logik (Phase 4)?

Noch fehlende Felder, die Phase ¾ brauchen, ergänzen (lieber jetzt als später).

Wenn du magst, können wir als Nächstes genau so in **Phase 3 -- DSA5 System Bridge** eintauchen: welche Felder aus lessons/exams/npcs wie auf DSA5-Talente, Zauber, Actors und Compendia gemappt werden, und wie das sauber entkoppelt von Janus-Core bleibt.

Phase 3 – DSA5 System Bridge (Detailliert)

Phase 3: DSA5 System Bridge -- Die Verbindung zur Regelmechanik -- Erweiterte Dokumentation

Ziel: Eine saubere, robuste und flexible Integration der DSA5-Spielmechaniken in JANUS7. Das Ziel ist es, die volle Tiefe des DSA5-Systems zu nutzen, ohne dass der Core-Code von JANUS7 direkt von den spezifischen Implementierungsdetails des DSA5-Systems abhängig wird. JANUS7 bleibt system-agnostisch in seiner Architektur, nutzt aber über diese Brücke die volle Power von DSA5.

Module & Funktionen

3.1 Integration (systems/dsa5/integration.js)

Dieses Modul ist der erste Berührungspunkt mit dem DSA5-System und stellt sicher, dass die Umgebung korrekt konfiguriert ist.

- **Funktion:**

  - **System-Check:** Prüft beim Start von Foundry VTT, ob das dsa5 System tatsächlich installiert und aktiv ist.

  - **Versions-Check:** Überprüft, ob die installierte Version des DSA5-Systems mit JANUS7 kompatibel ist. Dies verhindert Abstürze durch API-Änderungen im System.

  - **Modul-Erkennung:** Scannt nach optionalen, aber empfohlenen Modulen wie dsa5-core, dsa5-bestiary oder dsa5-magic.

  - **Warnsystem:** Gibt detaillierte Warnungen in der Konsole oder im UI aus, wenn kritische Abhängigkeiten fehlen oder Versionen inkompatibel sind.

3.2 Bridge (systems/dsa5/bridge.js)

Die Bridge ist das Herzstück der Integration. Sie fungiert als Übersetzer zwischen den generischen Befehlen von JANUS7 und den spezifischen API-Aufrufen von DSA5.

- **Funktion:** Stellt abstrakte Methoden bereit, die JANUS-Logik in DSA5-Aktionen übersetzen.

  - **findActor(criteria):** Sucht nach NPCs oder Charakteren in der Welt oder in Kompendien. Die Suche kann nach Name, Typ oder Tags erfolgen. Sie ist intelligent genug, um z.B. \"Rahjada\" auch zu finden, wenn der Actor \"Meisterin Rahjada\" heißt (Fuzzy Matching).

  - **rollCheck(actor, skill, mod):** Führt einen Würfelwurf für ein Talent, einen Zauber oder eine Eigenschaft aus.

    - **Input:** Actor-Objekt, Name des Talents (z.B. \"Sinnesschärfe\"), Modifikator (z.B. -2).

- **Prozess:** Findet das entsprechende Item auf dem Charakterbogen, berechnet die Probe unter Berücksichtigung von DSA5-Regeln (Eigenschaften, FW, Modifikatoren, Zustände) und führt den Wurf aus.

- **Output:** Liefert ein standardisiertes Ergebnis-Objekt zurück (z.B. { success: true, qs: 2, resultString: \"Gelungen (QS 2)\" }), das JANUS weiterverarbeiten kann (z.B. für Hauspunkte).

- **getInventory(actor):** Liest das Inventar eines Charakters aus und gibt es in einer vereinfachten JSON-Struktur zurück (z.B. für den Schwarzmarkt oder Zutaten-Tracker).

- **addCondition(actor, conditionId, level):** Fügt einem Charakter einen Zustand (z.B. \"Betäubung\", \"Schmerz\") hinzu oder entfernt ihn.

3.3 Parser (systems/dsa5/parser.js)

Der Parser ist der Dolmetscher für Text-Inputs, die oft von der KI oder aus manuell geschriebenen Abenteuern kommen.

- **Funktion:** Wandelt menschenlesbare, unstrukturierte Anweisungen in maschinenlesbare Datenstrukturen um, die die Bridge verstehen kann.

  - **Skill-Parsing:**

    - Input: \"Probe auf Sinnesschärfe -2\" oder \"Sinnesschärfe (erschwert um 2)\"

    - Output: { skill: \"Sinnesschärfe\", modifier: -2, type: \"skill\" }

  - **Damage-Parsing:**

    - Input: \"2W6+4 Schaden\"

    - Output: { formula: \"2d6+4\", type: \"damage\" }

  - **Attribute-Parsing:**

    - Input: \"Mut-Probe\"

    - Output: { attribute: \"mu\", type: \"attribute\" }

3.4 Indexer (systems/dsa5/indexer.js)

Der Indexer ist das Langzeitgedächtnis für die KI. Er löst das Problem, dass Sprachmodelle oft Dinge erfinden (\"Halluzinationen\"), die es in den Regeln gar nicht gibt.

- **Funktion:** Scannt einmalig (oder auf Knopfdruck) alle verfügbaren und relevanten Kompendien in der Foundry-Welt (Zauber, Items, Bestiarium, Sonderfertigkeiten).
- **Output:** Erstellt eine große, strukturierte Index-Datei (dsa5_index.json) im io/outbox/knowledge/ Ordner.
- **Inhalt:** Diese Datei enthält Listen von validen Namen und IDs.

- Beispiel: spells: \[\"Ignifaxius\", \"Balsam Salabunde\", \...\], items: \[\"Schwert\", \"Heiltrank\", \...\].

- **Zweck:** Diese Datei wird der KI als Kontext zur Verfügung gestellt. Wenn die KI dann eine Szene generiert, kann sie *echte* Zaubernamen und Gegenstände verwenden, anstatt welche zu erfinden.

🔗 Abhängigkeiten & Schnittstellen

Abhängigkeiten (Benötigt\...)

Damit die Bridge funktionieren kann, ist sie auf das DSA5-System angewiesen.

- **System DSA5:** Das dsa5 System muss in Foundry VTT installiert und als aktives Spielsystem für die Welt ausgewählt sein. Ohne dieses System kann die Bridge keine DSA-spezifischen Datenstrukturen (wie Charakterbögen) lesen oder manipulieren.
- **Optionale Module:** Module wie dsa5-core (Regelwerk), dsa5-bestiary (Monster), dsa5-magic (Zauber) sind nicht zwingend erforderlich, werden aber vom Indexer gescannt, um den Wissensschatz der KI zu erweitern.

Schnittstellen (Bietet an\...)

Die Bridge stellt eine definierte API bereit, die vom Core (insbesondere dem Director) und den Simulations-Modulen genutzt wird.

- **Bridge API:**

  - Bridge.rollCheck(actor, skill, mod): Für automatische Proben im Unterricht oder bei Events.

  - Bridge.findItem(name): Um Gegenstände in Kompendien zu finden (z.B. für Belohnungen).

  - Bridge.findActor(name): Um NPCs für Szenen zu identifizieren.

- **Parser API:**

  - Parser.parseCheckString(string): Um Textanweisungen aus KI-generierten Szenen in ausführbare Logik umzuwandeln.

- **Output:**

  - Generiert io/outbox/knowledge/dsa5_index.json: Dies ist die Schnittstelle zur KI (Phase 7), damit diese \"regelkonforme\" Vorschläge machen kann.

ChatGPT:

Alles klar, Phase-3-Zeit -- jetzt wird aus „JANUS weiß irgendwas über Proben" ein sauberes Interface zu „game.dsa5". Wir bauen quasi den diplomatischen Dienst zwischen Akademie und Regelsystem.

Unten ist der Text so formatiert, dass du ihn fast 1:1 in dein Word-Kapitel „Phase 3 -- DSA5 System Bridge" übernehmen kannst.

Phase 3 -- DSA5 System Bridge

3.1 Ziel, Scope & Nicht-Ziele

Ziel von Phase 3

Die DSA5 System Bridge stellt eine saubere, gekapselte Schicht zwischen:

- dem **systemagnostischen JANUS-Kern** (State, Director, Academy-Logik)
- und dem **Foundry-System „dsa5"** (Charakterbögen, Proben, Items etc.)

Sie sorgt dafür, dass:

- der Director **niemals direkt** game.dsa5.* aufruft, sondern immer via game.janus7.system
- DSA-Spezifika (Talent-IDs, Roll-APIs, Compendium-Namen) **nur hier** konzentriert sind
- die Bridge **deaktivierbar/ersetzbar** ist (z. B. später systems/otherSystem/)

Scope

- Zugriff auf **Talente, Zauber, Liturgien, Rituale, Waffenproben, Attribute** via DSA5-API
- Standardisierte **Roll-Requests** (z. B. requestSkillCheck, requestSpellCast)
- Lookup-Funktionen für **Actors & Items** (Studenten, Dozenten, Zauber, Ausrüstung)
- Leichte Nutzung von DSA-Compendia (Core, Kompendium, Armory, Magic 1--3, Herbarium etc.)
- Konfiguration über game.settings (z. B. welche Packs als Quellen dienen)

Nicht-Ziele

- Kein eigenes Kampfsystem, keine Alternativ-Würfelengine. DSA5 bleibt „Rules Authority".
- Keine tiefen Eingriffe in das DSA5-System (keine Monkey-Patches, keine Rewrites).
- Keine UI-Logik (die gehört in Phase 6 / Anwendungen auf ApplicationV2).

3.2 Modulstruktur & Namespaces

Dateistruktur (Vorschlag)

modules/janus7/

systems/

dsa5/

index.js // Entry Point, Auto-Detection, Export der Public API

rolls.js // Würfel-Schnittstelle: Skills, Zauber, Liturgien, Angriffe

actors.js // Lookups & Hilfsfunktionen rund um Actors

items.js // Lookup & Matching von Items (Zauber, Liturgien, Ausrüstung)

packs.js // Dynamischer Index über DSA5-Compendia

config.js // Konfigurierbare Mappings (Talent-Keys, Pack-Namen)

diagnostics.js // Selbsttest & Integritätschecks

Globaler Access-Point

```
// Beim Ready-Hook eingerichtet
game.janus7 = game.janus7 ?? {};
game.janus7.system = {
id: \"dsa5\",
version: \"7.3.5\", // ermittelt aus game.system.data.version
rolls: JanusDsa5Rolls, // aus rolls.js
actors: JanusDsa5Actors, // aus actors.js
items: JanusDsa5Items, // aus items.js
packs: JanusDsa5Packs, // aus packs.js
diagnostics: JanusDsa5Diagnostics
};
```

Compatibility-Checks

- Aktivierung **nur**, wenn game.system.id === \"dsa5\" ist.
- Falls nicht, wird eine „No-Op-Bridge" registriert (z. B. für später andere Systeme).

3.3 Zentrale Datenstrukturen

3.3.1 Skill- & Spell-IDs (systemSkillId)

Der Academy-Layer arbeitet mit abstrakten IDs wie:

- TALENT_SINNESSCHAERFE
- SPELL_CORPOFRIGO

- LITURGY_BANNSEGEN

Die DSA5-Bridge mappt diese auf echte System-Keys und Items.

Mapping in config.js:

```
export const JANUS_DSA5_CONFIG = {

skills: {

\"TALENT_SINNESSCHAERFE\": \"SIN\", // DSA5 interner Talent-Key

\"TALENT_MAGIEKUNDE\": \"MAG\"

},

spells: {

\"SPELL_CORPOFRIGO\": {

name: \"Corpofrigo\",

fallbackPack: \"dsa5-magic-1.spells\"

}

},

liturgies: {

\"LITURGY_BANNSEGEN\": {

name: \"Bannsegen\",

fallbackPack: \"dsa5-gods.liturgies\"

}

}

};
```

Damit bleiben Lehrplan/Schedule (Phase 2/4) später änderbar; nur die Bridge weiß, wo im System die passenden Talente/Zauber liegen.

3.3.2 Compendium-Index

packs.js hält einen **Index aller relevanten DSA5-Packs** im aktuellen World-Setup:

- erkennt automatisch alle Packs, deren system === \"dsa5\" oder deren Modul-ID mit dsa5- beginnt
- cacht für schnelle Lookups (spellByName, itemById etc.)

Beispielstruktur:

```
{
systemId: \"dsa5\",
version: \"7.3.5\",
packs: {
spells: \[
\"dsa5-core.spells\",
\"dsa5-magic-1.spells\",
\"dsa5-magic-2.spells\",
\"dsa5-magic-3.spells\"
\],
talents: \[
\"dsa5-core.skills\"
\],
creatures: \[
\"dsa5-bestiary.creatures\",
\"dsa5-bestiary2.creatures\"
\]
}
}
```

3.4 Öffentliche API der DSA5-Bridge

Alle anderen Phasen sprechen **nur** mit dieser API -- nie direkt mit game.dsa5.

3.4.1 Roll-API (rolls.js)

```
// Pseudo-TypeScript zur Dokumentation
interface SkillCheckOptions {
systemSkillId: string; // z.B. \"TALENT_SINNESSCHAERFE\"
```

```
modifier?: number; // Pauschalmodifikator

difficultyLabel?: string;

actor?: Actor; // Standard: selektierter Actor

reason?: string; // Beschreibung für den Chat

}

interface RollResultRef {

messageId?: string;

resultSummary?: string;

}

interface JanusDsa5Rolls {

requestSkillCheck(options: SkillCheckOptions): Promise\<RollResultRef\>;

requestAttributeCheck(attributeKey: string, options?: any): Promise\<RollResultRef\>;

requestSpellCast(spellId: string, options?: any): Promise\<RollResultRef\>;

requestAttack(options: any): Promise\<RollResultRef\>;

}
```

Interne Umsetzung

- nutzt die offiziell dokumentierte Macro-API:\

  game.dsa5.macro.requestRoll(\"TALENT\", MODIFICATOR)

- mappt systemSkillId → DSA5-Talent-Key über JANUS_DSA5_CONFIG.skills
- prüft, ob actor gesetzt ist, sonst nimmt es den aktuell selektierten (oder wirft einen Fehler, wenn keiner vorhanden)

Damit bleibt der Director-Code extrem simpel:

```
await game.janus7.system.rolls.requestSkillCheck({

systemSkillId: \"TALENT_SINNESSCHAERFE\",

modifier: -1,

reason: \"Heimliches Tuscheln im Schlafsaal\"

});
```

3.4.2 Actor-API (actors.js)

Typische Funktionen:

- getStudentActorByKey(studentKey: string)
- getTeacherActorByKey(teacherKey: string)
- ensureActorForNpcProfile(profileKey: string) -- nutzt Phase-2/Academy-Datenbanken
- syncHouseAssignment(actor, houseId) -- House/Zirkel als Actor-Flag setzen

Intern:

- nutzt game.actors und ggf. DSA5-Aktortypen (\"character\", \"npc\")
- optional: Erzeugt neue Actors aus Template-Journals (DGdZ-Modul)

3.4.3 Item-API (items.js)

Funktionen:

- findSpellByConfigId(\"SPELL_CORPOFRIGO\") → Item \| null
- findLiturgieByConfigId(\"LITURGY_BANNSEGEN\")
- ensureItemOnActor(actor, itemConfigId) -- z. B. im Rahmen von Lehrplanfortschritt
- grantSpecialAbility(actor, saConfigId)

Nutzt:

- Compendium-Index aus packs.js (siehe 3.3.2)
- Standard-Compendium-API von Foundry (game.packs.get, pack.getDocuments())

3.5 Integration mit Foundry & DSA5

3.5.1 Aktivierung & Lifecycle

Im ready-Hook von JANUS7:

42. Prüfen, ob game.system.id === \"dsa5\" ist.
43. Falls nein → No-Op-Bridge registrieren (game.janus7.system = JanusNullSystem).
44. Falls ja:

    - packs.js scannt alle game.packs nach DSA5-Inhalten.
    - config.js und User-Settings werden geladen.
    - diagnostics.js führt einen kurzen Selbsttest aus (z. B. „Core-Talente vorhanden?").

3.5.2 Settings & Konfiguration

Moduleinstellungen (game.settings.register im Namespace janus7):

- janus7.dsa5.enabled -- Bridge ein/aus
- janus7.dsa5.preferredSpellPacks -- Liste der Spell-Packs, die zuerst durchsucht werden
- janus7.dsa5.strictConfigMode -- ob ein systemSkillId ohne Mapping einen Fehler werfen soll

Scopes:

- Konfig-Settings: scope: \"world\" (übergreifend für die Kampagne)

3.6 States, Flags & Persistenz

Die DSA5-Bridge selbst hält **möglichst wenig eigenen State**; State gehört in Phase-1-JanusStateCore.

Trotzdem gibt es ein paar Dinge, die die Bridge zwischenspeichern darf:

- **Cache für Compendium-Lookups** (z. B. Map: spellName → Item.id)
- **Diagnoseergebnisse**, z. B. welche Module/Compendia gefunden wurden

Persistenzoption:

- Leichte Caches als client-Setting (für den GM), da sie bei Änderung neu erzeugt werden können.

3.7 Abhängigkeiten zu anderen Phasen

Hängt ab von

- **Phase 1 -- Core & Data Architecture**
  - game.janus7.state → um Actors, Zirkel, Zeitkontext zu kennen
  - Logger & Fehlerbehandlung
- **Phase 2 -- Static Academy Data**
  - Keys wie TALENT_SINNESSCHAERFE, SPELL_CORPOFRIGO stammen hierher
  - NPC-/Ort-IDs für Actor-Erzeugung oder Probenkontext

Wird genutzt von

- **Phase 4 -- Academy Simulation Logic**
  - Szenen wie „Prüfung in Magiekunde" rufen direkt game.janus7.system.rolls.*
- **Phase 5 -- Hybrid & Atmosphere Controller**
  - Eventual: DSA5-spezifische Trigger für Musik (z. B. bei Patzer)

- **Phase 6 -- User Interfaces**
  - Buttons im Control Panel: „Probe Sinnesschärfe" → Bridge

So bleibt der Datenfluss sauber:\

Academy/Director → DSA5-Bridge → DSA-System → Chat/Resultat.

3.8 Aufwandsschätzung (Phase 3)

Grob für eine Person, mit deinem Erfahrungslevel und vorhandenen Code als Referenz:

45. **Grundgerüst & Detection (index.js, config.js)**
    - 0,5 -- 1 PT
46. **Roll-API (rolls.js) inkl. Skill/Spell-Requests über game.dsa5.macro.requestRoll**
    - 1,5 -- 2 PT
47. **Compendium-Index (packs.js) + Item-Lookups (items.js)**
    - 1,5 -- 2,5 PT (stark abhängig davon, wie „smart" das Matching sein soll)
48. **Actor-API (actors.js) inkl. Hook in deine NPC-Datenbanken**
    - 1 -- 2 PT
49. **Diagnostics & Settings (inkl. Test & Logging)**
    - 1 PT
50. **Integration mit Phase-1/2 + erste End-to-End-Tests**
    - 1 -- 2 PT

Summe: 6 -- 10 Personentage, je nach Tiefe der Diagnostik und Matching-Logik.

3.9 Risiken & Gegenmaßnahmen

51. **Änderungen am DSA5-System (API-Brüche)**
    - *Risiko:* game.dsa5.macro.requestRoll oder interne Keys ändern sich.
    - *Gegenmaßnahme:*
      - Alle DSA5-APIs zentral in rolls.js kapseln.
      - diagnostics.js prüft bei Start, ob game.dsa5.macro.requestRoll existiert.
      - Bei Fehlern: Deaktivieren mit klarer Notification („DSA5-API inkompatibel mit JANUS7").

52. **Nicht alle DSA-Module sind installiert**

- *Risiko:* Config verweist auf Packs, die nicht existieren.
- *Gegenmaßnahme:*
  - packs.js arbeitet tolerant („Pack nicht da? → Überspringen, Warnlog").
  - Fallback-Packs konfigurierbar über game.settings.

53. **Performance bei großen Compendia**

- *Risiko:* Vollscan aller Compendia beim Ready-Hook kann zäh werden.
- *Gegenmaßnahme:*
  - Lazy-Loading: Items erst bei Bedarf laden und cachen.
  - Optionaler Pre-Index („Build Index Now"-Button in Phase-6-UI).

54. **Zu starke Kopplung an Punin-Akademie**

- *Risiko:* DSA5-Bridge ist mit Academy-Konzepten verfilzt.
- *Gegenmaßnahme:*
  - Bridge denkt nur in DSA-Begriffen („Actor", „Skill", „Spell").
  - Academy-Sicht (Schüler, Zirkel, Stundenplan) bleibt in Phase 2/4.

3.10 Umsetzungsschritte / Plan

In der Praxis würde ich Phase 3 so bauen:

55. **Skeleton & Detection**

- systems/dsa5/index.js mit:
  - Check game.system.id === \"dsa5\"
  - Registrierung von game.janus7.system = JanusDsa5System
  - Fallback bei Nicht-DSA5

56. **Minimal-Roll-API**

- rolls.js mit einer Funktion:\

  requestSkillCheck({ systemSkillId, modifier }) → ruft game.dsa5.macro.requestRoll(\"TALENT\", modifier)

- Ein hartcodierter Test-Skill (TALENT_SINNESSCHAERFE) über config.js

57. **Packs & Items**

- packs.js erkennt alle DSA5-Packs und loggt sie.
- items.js bekommt findSpellByName(name) und findItemByName(name) mit einfacher Suche.

58. **Config & Settings**

- JANUS_DSA5_CONFIG aus Phase-2-IDs füllen.
- Settings für bevorzugte Spell-Packs & Strict-Mode registrieren.

59. **Actors & NPC-Integration**

- actors.js implementiert getStudentActorByKey anhand deiner NPC-Datenbanken (Phase 2).
- Optional: ensureActorForNpcProfile.

60. **Diagnostics**

- diagnostics.js prüft:
  - game.dsa5 vorhanden?
  - macro.requestRoll vorhanden?
  - Mindestens ein Spell-Pack gefunden?

61. **Abschluss dieser Phase**

- Smoketest: Eine einfache Lektion in der Akademie triggert eine Sinnesschärfe-Probe über den Director → DSA5-Roll erscheint im Chat.
- Dokumentation (diese Phase) finalisieren und an Phase-4-Spezifikation übergeben.

Wenn du magst, können wir als Nächstes einen konkreten **API-Entwurf für rolls.js + config.js** ausformulieren, den du direkt in ein Modul kippen kannst -- dann wird Phase 3 von „Konzepttext" zu „halbem Implementierungsplan".

Phase 4 – Academy Simulation Logic (Detailliert)

Phase 4: Academy Simulation Logic -- Die lebendige Welt -- Erweiterte Dokumentation

Ziel: Implementierung der Spielmechaniken, die das Schulleben jenseits von Kämpfen und Abenteuern simulieren (Social, Economy, Scoring). Diese Module füllen die Lücke zwischen statischen Daten und dynamischem Spielerlebnis.

Module & Funktionen

4.1 Scoring Engine (academy/scoring.js)

Das Punktesystem ist der zentrale Motivationsfaktor für die Schüler. Es verwaltet sowohl den Wettbewerb der Häuser als auch individuelle Leistungen.

- **Funktion:**

  - **Hauspunkte:** Verwaltet den Punktestand der vier Häuser (Salamander, Stäbe, Schwerter, Sicheln). Punkte können durch Unterrichtsbeiträge, Quests oder Disziplinarmaßnahmen gewonnen oder verloren werden.

  - **Schülerpunkte:** Trackt individuelle Leistungspunkte, die für den Jahresbesten-Titel oder spezielle Belohnungen relevant sind.

  - **Rankings:** Berechnet live Bestenlisten für Häuser und Schüler.

  - **MVP-Tracking:** Ermittelt den aktuellen \"Most Valuable Pupil\" (MVP) basierend auf recent performance.

4.2 Social Engine (academy/social.js)

\$\$V7\$\$

Die \"Persona\"-Mechanik bringt soziale Tiefe in die Akademie. Sie macht NPCs zu dynamischen Akteuren, die auf das Verhalten der Spieler reagieren.

- **Funktion:**

  - **Relations-Matrix:** Speichert für jedes Paar aus NPC und SC einen Beziehungswert (-100 bis +100) und einen Status (z.B. \"Neutral\", \"Freund\", \"Rivale\", \"Erzfeind\").

  - **Gossip-System:** Simuliert die Verbreitung von Gerüchten. Ein Gerücht (z.B. \"Alrik hat geschummelt\") hat einen Ursprung, einen Wahrheitsgehalt und eine Verbreitungsrate. NPCs reagieren auf Gerüchte, was ihre Beziehung zu den SCs beeinflusst.

- **Cliquen-Dynamik:** Gruppen von NPCs (z.B. \"Die Adligen\") teilen Meinungen. Eine Änderung der Beziehung zu einem Mitglied kann sich auf die ganze Clique auswirken.

4.3 Economy System (academy/economy.js)

\$\$V7\$\$

Die alternative Schul-Ökonomie simuliert den Mangel und die Tauschgeschäfte, die für Schüler typisch sind.

- **Funktion:**

  - **Taschengeld:** Verwaltet regelmäßige Einkünfte (Dukaten/Silber) basierend auf dem Sozialstatus.

  - **Schulden-Tracker:** Wer schuldet wem was? Schulden können bei NPCs oder anderen Spielern gemacht werden und haben Fälligkeitsdaten.

  - **Schwarzmarkt:** Generiert wöchentliche Angebote für illegale oder seltene Waren (z.B. verbotene Bücher, seltene Zutaten). Der Kauf ist risikobehaftet (Codex-Verstoß).

  - **Gruppen-Lager (Stash):** Ein virtuelles Inventar für gemeinsame Ressourcen wie Alchemiezutaten, die für Klassenprojekte benötigt werden.

4.4 Learning Engine (academy/learning.js)

\$\$V7\$\$

Dieses Modul überwacht den akademischen Fortschritt und die Disziplin.

- **Funktion:**

  - **Time Management:** Spieler haben begrenzte Zeit-Slots pro Woche (z.B. \"3 Nachmittage zur freien Verfügung\"). Sie müssen entscheiden, ob sie lernen, forschen oder soziale Kontakte pflegen.

  - **Thesis-Tracker:** Ein Fortschrittsbalken (0-100%) für die Abschlussarbeit. Fortschritt wird durch erfolgreiche Proben (Forschung) und gefundene Hinweise (Items) generiert.

  - **Disziplinar-System:** Überwacht Verstöße gegen den Codex Albyricus. Verwaltet \"Strikes\" (Verwarnungen) und triggert bei Überschreitung Konsequenzen (Nachsitzen, Strafarbeiten).

🔗 Abhängigkeiten & Schnittstellen

Abhängigkeiten (Benötigt\...)

Die Simulation ist stark vernetzt und benötigt Daten und Ereignisse aus anderen Phasen.

- **Phase 1 (Core):** Speichert alle Simulationsdaten persistent im State. Nutzt Config für Einstellungen.
- **Phase 2 (Academy Data):** Braucht den Kalender für zeitabhängige Events (z.B. Zinszahlung, Taschengeld, Unterrichtszeiten für Lern-Slots).
- **Phase 3 (Bridge):** Braucht DSA5-Würfe für Lernerfolge, Crafting-Proben, soziale Interaktionen (z.B. \"Überreden\") und Schwarzmarkt-Deals.

Schnittstellen (Bietet an\...)

Die Simulation stellt APIs bereit, die von der UI (Phase 6) und der KI (Phase 7) genutzt werden.

- **Scoring API:**
  - Scoring.addPoints(house, amount, reason, source): Fügt Punkte hinzu und loggt den Grund.
  - Scoring.getRanking(): Gibt die aktuelle Rangliste zurück.
- **Social API:**
  - Social.modifyRelation(npc, pc, value, reason): Ändert den Beziehungswert.
  - Social.getRelation(npc, pc): Fragt den aktuellen Status ab.
  - Social.spreadGossip(rumor): Startet ein neues Gerücht.
- **Economy API:**
  - Economy.addDebt(pc, amount, creditor, dueDate): Registriert eine neue Schuld.
  - Economy.checkIngredients(recipe, stash): Prüft, ob genug Zutaten im Lager sind.
  - Economy.getBlackMarketOffers(): Liefert die aktuellen Angebote.
- **Hooks:**
  - janus7ScoreChanged: Bei Punkteänderung (für UI-Updates).
  - janus7RelationChanged: Bei Beziehungsänderung.
  - janus7ThesisUpdated: Bei Fortschritt in der Thesis.
  - janus7DebtDue: Wenn eine Schuld fällig wird.

ChatGPT:

Jetzt kommt die Phase, in der die Akademie wirklich „lebt" -- Stundenplan, Prüfungen, Punkte, Beziehungen. Das ist der Game-Loop deines Schuljahrsimulators.

Ich schreibe dir Phase 4 wieder so, dass du sie direkt als Kapitel ins Word-Dokument kleben kannst.

Phase 4 -- Academy Simulation Logic

Der Schuljahrs-Motor von JANUS7

4.1 Ziel & Scope der Phase

Ziel von Phase 4

Diese Phase implementiert die **Simulationslogik** der Akademie -- also alles, was aus:

- dem **Zeit-State** (Phase 1, JanusStateCore.time),
- den **statischen Akademie-Daten** (Phase 2, AcademyData.*),
- und der **DSA5-Bridge** (Phase 3, game.janus7.system.*)

einen **spielbaren Schulalltag** macht.

Konkret:

- Zeitfortschritt im Akademie-Kalender (Tage, Phasen, Wochen).
- Bestimmung der aktuellen Unterrichtseinheiten (Lessons) und Prüfungen.
- Berechnung von Haus-/Zirkelpunkten und Schülerpunkten.
- Ableitung sozialer Effekte (Beziehungen, Reputation).
- (Einfach) ökonomische Effekte (Taschengeld, Schulden, Ressourcen).

Im Scope:

- Logikmodule unter academy/:
  - academy/calendar.js -- Zeit & Tagesstruktur
  - academy/lessons.js -- Unterrichtslogik
  - academy/exams.js -- Prüfungslogik
  - academy/scoring.js -- Punkte- & Gamification-Engine
  - academy/social.js -- einfache Beziehungs-Engine
  - academy/events.js -- (optionale) Spezialereignis-Logik
- Keine UI (das kommt in Phase 6).
- Keine direkte DSA5-API-Nutzung ohne die Bridge (Phase 3).
- Keine Datei-Ein/Ausgabe (läuft über Phase 1/IO & Phase 7).

Nicht im Scope:

- Kein eigener Kalender UI -- Integration mit Foundry-Zeit bleibt minimal (Hooks, game.time).
- Keine KI-Interaktion (nur vorbereitend, wird in Phase 7 genutzt).

- Keine „Intelligenz" im Sinne von KI -- alles deterministische/regelbasierte Logik.

4.2 Modul- & Verzeichnisstruktur

Vorgeschlagene Struktur:

modules/janus7/

academy/

calendar.js // Zeit- & Slot-Logik

lessons.js // Auswahl & Auswertung von Unterricht

exams.js // Start, Ablauf, Auswertung von Prüfungen

scoring.js // Zirkel- und Schülerpunkte

social.js // Beziehungs- & Reputations-Effekte

events.js // optionale Ereignis-Engine (Feiertage, Feste, Krisen)

Globaler Zugriff:

```
game.janus7.academy = {
calendar: JanusCalendarEngine,
lessons: JanusLessonEngine,
exams: JanusExamEngine,
scoring: JanusScoringEngine,
social: JanusSocialEngine,
events: JanusEventEngine
};
```

4.3 Kalender & Zeitlogik (academy/calendar.js)

4.3.1 Aufgabe

Der Kalender-Motor ist die Schicht zwischen:

- JanusStateCore.time (Phase 1)
- AcademyData.calendar & AcademyData.events (Phase 2)
- optional: Foundry-Weltzeit (game.time, updateWorldTime-Hook)

Er beantwortet Fragen wie:

- „Welcher Unterricht/Prüfung/Ereignis liegt **jetzt** an?"
- „Wie sieht die **Woche** an Unterricht aus?"
- „Ist heute ein Feiertag oder Akademie-Event?"
- „Was passiert beim **Fortschritt** auf die nächste Phase / den nächsten Tag?"

4.3.2 State & Modelle

Nutzen:

- JanusStateCore.time:
  - year, trimester, week, day, phase, totalDaysPassed, isHoliday
- Phase-2-Daten:
  - calendar.entries\[\]
  - events.events\[\]

Zur Arbeit wird ein gemeinsames **DayRef/SlotRef** benutzt:

type DayRef = {

year: number;

trimester: number;

week: number;

day: string; // \"Praiosstag\" etc.

};

type SlotRef = DayRef & {

phase: string; // \"Morgen\" \| \"Vormittag\" \| etc.

};

4.3.3 Öffentliche API JanusCalendarEngine

- getCurrentSlotRef(): SlotRef\

  Wandelt JanusStateCore.time in eine SlotRef-Struktur.

- getCalendarEntryForCurrentSlot()\

  Sucht im calendar.json den Eintrag (type, lessonId, examId, eventId, holidayKey) für die aktuelle SlotRef.

- getCalendarEntriesForDay(dayRef: DayRef)\

Liefert alle Slots eines Tages (für Wochen-/Tagesansichten, Phase 6).

- isHoliday(dayRef?: DayRef): boolean\

  Prüft, ob der Tag einen type: \"holiday\" oder holidayKey hat.

- advancePhase(options?)

  - Aktualisiert JanusStateCore.time.phase logikgerecht.
  - Wenn Phase überschritten → Tag/Woche/Trimester/Year hochzählen.
  - Aktualisiert totalDaysPassed bei Tageswechsel.
  - Optional mit Kopplung an game.time.advance() (Foundry-Weltzeit), falls gewünscht.

- advanceDay(options?)

  - Springt direkt zum nächsten Tag (z. B. zwischen Sitzungen).

- setTime(dayRef: DayRef, phase: string)

  - Direkter Hard-Jump (für SL-Steuerung).

Die **eigentliche Persistenz** (Speichern des neuen Zeit-States) läuft immer über JanusStateCore (Phase 1). Kalender-Engine ruft dazu z. B. JanusState.transaction(\...) auf.

4.4 Unterrichtslogik (academy/lessons.js)

4.4.1 Aufgabe

Der Lesson-Motor entscheidet:

- welche **Lektion(en)** im aktuellen Slot stattfinden,
- welche **Effekte** eine abgehaltene Lektion auf:

  - Wissensstand (später),
  - Social/Scoring,
  - Story/Chronik\

    hat.

Er ist das Bindeglied zwischen:

- Kalender (welcher Slot, welche lessonId),
- statischen Lektionen (lessons.json),
- State (Scoring, Social, Story),

- DSA5-Bridge (wenn die Lektion z. B. zu einer Probe führen soll -- aber diese Ansteuerung passiert nur über Phase 3).

4.4.2 Öffentliche API JanusLessonEngine

- getLessonsForCurrentSlot(): Lesson\[\]
  - Sucht calendarEntry zur current SlotRef.
  - Löst lessonId → AcademyData.getLesson(id) auf.
- runLesson(lessonId: string, options?)
  - Führt die „Standardlogik" für eine Lektion aus:
    - optional: Social-/Scoring-Basiseinfluss (z. B. Fleiß im Unterricht → +Punkte).
    - Einträge in story.chronicle.
    - optional: erzeugt ToDos für DSA5-Proben (aber ruft sie nicht zwingend sofort).
  - Rückgabe: LessonResult mit Infos, die UI/SL nutzen können.
- applyLessonOutcome(lessonId, outcome)
  - Fasst die Ergebnisse (z. B. aus einer Session, Proben) zusammen und wendet sie auf:
    - JanusStateCore.scoring,
    - JanusStateCore.social,
    - JanusStateCore.story an.

Wichtig:\

LessonEngine bleibt **mechanik-neutral**. Ob eine bestimmte Lektion im Rahmen der Session nun wirklich mit 3 DSA-Proben gespielt oder nur „erzählt" wird, entscheidest du am Tisch -- die Engine stellt nur Hooks und Standardvorschläge bereit.

4.5 Prüfungslogik (academy/exams.js)

4.5.1 Aufgabe

Der Exam-Motor wickelt **strukturierte Prüfungen** ab auf Basis von:

- exams.json (Phase 2),
- Kalender (wann ist die Prüfung),
- DSA5-Bridge (Phase 3, Proben),
- State/Scoring (Phase 1).

4.5.2 Öffentliche API JanusExamEngine

- getExamsForCurrentSlot(): Exam\[\]
  - Analog zu Lessons: Kalender → examId → AcademyData.getExam(id).
- startExam(examId: string, options?)\

  Typischer Ablauf:

  1. Prüfen, ob examId existiert, ob heute der passende Slot ist (optional).
  2. Internen **ExamRun** erzeugen:
  3. type ExamRun = {
  4. id: string;
  5. examId: string;
  6. startedAtDayIndex: number;
  7. participants: string\[\]; // Student-IDs
  8. status: \"ongoing\" \| \"completed\" \| \"aborted\";
  9. results: Record\<string, any\>; // pro Schüler
  10. };
  11. Optional: Trigger für UI, um Proben auszulösen.
- requestExamChecks(examId: string, participants: Actor\[\] \| string\[\])
  - Delegiert für jede relevante Skill/Komponente an:
    - game.janus7.system.rolls.requestSkillCheck(\...)
    - game.janus7.system.rolls.requestSpellCast(\...) (bei Praxisprüfungen)
- finalizeExam(examId: string, resultSummary: ExamResultPayload)
  - Interpretiert resultSummary (z. B. QS pro Schüler, Bonus/Malus).
  - Wendet gradingScheme (aus exams.json) an.
  - Aktualisiert:
    - JanusStateCore.scoring (House/Student),
    - JanusStateCore.social.reputation (z. B. „Fachschaft Magie"),

    - JanusStateCore.story.chronicle.

- cancelExam(examId: string, reason?)
  - Setzt Status → \"aborted\", loggt in story.chronicle.

States & Flags:

- Exam-spezifischer Zwischenstate kann entweder:
  - im JanusStateCore.flags gehalten werden (flags.currentExamRun), oder
  - in einem eigenen, kleinen Sub-State im exam-Engine (nicht persistent, nur Session).

Empfehlung: Persistente Infos (z. B. „Prüfung abgeschlossen, Noten vergeben") im Haupt-State, laufende technische Details im RAM.

4.6 Scoring & Gamification (academy/scoring.js)

4.6.1 Aufgabe

Der Scoring-Motor ist die **einzige Instanz**, die JanusStateCore.scoring mutieren darf:

- Zirkel/Hauspunkte verwalten.
- Individuelle Schülerpunkte verwalten.
- Historie führen.

4.6.2 Datenbasis

Verwendet:

- JanusStateCore.scoring:
  - circles: { salamander, staves, swords, sickles }
  - students: { \[studentId\]: number }
  - lastAwarded\[\]: Historie

4.6.3 Öffentliche API JanusScoringEngine

- addCirclePoints(circleId, amount, reason, options?)
  - Aktualisiert scoring.circles\[circleId\] += amount.
  - Fügt Eintrag in lastAwarded ein:
  - {
  - timestamp,

  - source: \"lesson\" \| \"exam\" \| \"event\" \| \"manual\",
  - amount,
  - targetType: \"circle\",
  - targetId: circleId,
  - reason
  - }
- addStudentPoints(studentId, amount, reason, options?)
- getCircleScore(circleId) / getStudentScore(studentId)
- getLeaderboard({ type: \"circle\" \| \"student\", topN? })
  - Für UI oder Narration (z. B. Hauspokal-Szenen).
- applyExamImpact(examDef, examResults)
  - Convenience: nutz examDef.scoringImpact, verteilt Punkte an Circle/Studenten.

Schnittreferenzen:

- Wird massiv von lessons.js und exams.js verwendet.
- UI (Phase 6) ruft nur Scoring-API, fasst selbst nichts am State an.

4.7 Social Engine (academy/social.js)

4.7.1 Aufgabe

Die Social-Engine pflegt die sozialen Werte in JanusStateCore.social:

- Beziehungen zwischen Figuren (relations),
- Reputationswerte („Allgemeiner Ruf", „Ruf bei Lehrkörper/Schülern"),
- Tags (z. B. „Streber", „Rowdy").

Ziel: einfache, **regelbasierte** Veränderungen, keine überfrachtete Soziologie-Sim.

4.7.2 Datenbasis

- JanusStateCore.social:
  - relations: { \"sourceId::targetId\": number } (--100 bis +100)
  - reputation: { \[scopeId\]: number }
  - tags: { \[actorId\]: string\[\] }

4.7.3 Öffentliche API JanusSocialEngine

- adjustRelation(sourceId, targetId, delta, reason?)
  - Clamped auf --100...+100.
  - Nutzt Schlüsselschema \${sourceId}::\${targetId}.
- setRelation(sourceId, targetId, value)
- adjustReputation(scopeId, delta, reason?)
  - scopeId z. B. \"academy\", \"faculty\", \"students\".
- addTag(actorId, tag) / removeTag(actorId, tag)
- applyLessonSocialEffects(lessonDef, context)
  - Bsp.: Gruppenarbeit stärkt Beziehungen in der Gruppe, etc.

Kopplungen:

- Lessons (Phase 4) entscheiden, *wann* Social-Effekte ausgelöst werden.
- Künftig könnten KI-Tools diese State-Daten nutzen, um z. B. Drama-Hooks zu bauen.

4.8 Events & Hooks (academy/events.js)

4.8.1 Aufgabe

Events sind spezielle, vorkonfigurierte **Akademie-Ereignisse**, die:

- über events.json (Phase 2) definiert sind,
- über calendar.json oder Zeitbedingungen getriggert werden,
- kombinierte Effekte auf Story/Scoring/Social haben können.

4.8.2 Öffentliche API JanusEventEngine

- listEventsForCurrentDay()
- triggerEvent(eventId, options?)
  - Schreibt in story.chronicle.
  - Kann scoring und social beeinflussen.
  - Kann Hooks an Director/Atmosphere vorbereiten (z. B. Schulball → Musikwechsel).

4.9 Integration in Foundry-Zeit & Hooks

Du hast zwei Zeitachsen:

62. **JANUS-Zeit** (State) -- was die Akademie denkt.
63. **Foundry-Weltzeit** (game.time.worldTime, updateWorldTime-Hook).

Designempfehlung:

- JANUS ist führend für die **Ingame-Schulstruktur**.
- Foundry-Weltzeit kann optional mitlaufen, aber ist nicht zwingend.

Mögliche Integration:

- Hooks.on(\"updateWorldTime\", \...):
  - Prüft, ob sich genug Zeit bewegt hat, um z. B. eine Phase weiterzuschalten.
- Umgekehrt: JanusCalendarEngine.advancePhase() kann optional game.time.advance(delta) aufrufen.

Hook-Verwendung ist die empfohlene Art, sich an Foundry-Ereignisse zu hängen, statt Core-Methoden zu patchen.

4.10 Abhängigkeiten zu anderen Phasen

Hängt ab von:

- **Phase 1 -- Core & State**
  - Lese/Schreibzugriff auf JanusStateCore.time, scoring, social, story, flags.
  - Nutzung von JanusState.transaction() + Hooks.
- **Phase 2 -- Static Academy Data**
  - calendar.json, lessons.json, exams.json, npcs.json, locations.json, events.json.
- **Phase 3 -- DSA5 System Bridge**
  - Prüfungslogik ruft **nur** über game.janus7.system.rolls.* Proben an.

Wird genutzt von:

- **Phase 5 -- Hybrid & Atmosphere**
  - Reagiert auf Zeit/Events/Lessons/Exams (z. B. Musikwechsel bei Prüfungen).
- **Phase 6 -- UI**
  - Baut Control Panels („Heute", „Woche", „Zirkelstand", „Soziales") auf Basis der Engines.
- **Phase 7 -- KI**
  - Liest Akademie-State & Kalendereinträge, um Folgewochen/Events vorzuschlagen.

4.11 Aufwandsschätzung Phase 4

Grobe Schätzung:

64. **CalendarEngine** (calendar.js)

- SlotRef-Logik, Advance-Funktionen, Holiday-Erkennung\

  → 1,5--2,5 PT

65. **LessonEngine** (lessons.js)

- Lookup & Standard-Hooks (ohne komplexe Mechanik)\

  → 1--2 PT

66. **ExamEngine** (exams.js)

- Start/Finalize-Flow, Integration mit DSA5-Bridge (ohne fancy UI)\

  → 2--3 PT

67. **ScoringEngine** (scoring.js)

- Punkte-API + Historie\

  → 1--1,5 PT

68. **SocialEngine** (social.js)

- einfache Relation/Reputation/Tags-Operationen\

  → 1--1,5 PT

69. **EventEngine** (events.js)

- Basis: Trigger + simple Effekte\

  → 1--1,5 PT

70. **Tests, Debugging & Feinschliff**

- Integrationstests mit Phase-1/2/3 in einer Test-World\

  → 1,5--2,5 PT

Summe: ca. **9--14 Personentage**, je nachdem, wie komplex du Exams & Social wirklich ausbaust.

4.12 Risiken & Gegenmaßnahmen

71. **Risiko: Logik bläht sich auf („Sims-Syndrom")**

- *Problem:* Du baust immer mehr Spezialfälle in die Engines.

- *Gegenmaßnahmen:*
  - Strikt bleiben: Phase 4 = **Engine + Hooks**, keine große KI im Code.
  - Für Sonderfälle lieber KI-Pipeline (Phase 7) oder SL-Eingriffe nutzen.

72. **Risiko: Enge Kopplung UI ↔ Logik**

- *Problem:* Buttons rufen direkt State-Mutation statt Engine-APIs.
- *Gegenmaßnahmen:*
  - UI (Phase 6) muss ausschließlich academy.*-APIs nutzen.
  - Keine State-Manipulation direkt aus ApplicationV2.

73. **Risiko: Inkonsistenz zwischen Kalender und Zeit**

- *Problem:* calendar.json und JanusStateCore.time laufen auseinander.
- *Gegenmaßnahmen:*
  - Alle Zeitänderungen **nur** über JanusCalendarEngine.advance*() durchführen.
  - Validator/Diagnostics können regelmäßig prüfen, ob eine SlotRef existiert.

74. **Risiko: Overkill bei Prüfungslogik**

- *Problem:* Komplettmodell von Noten/Boni/Proben wird zu kompliziert.
- *Gegenmaßnahmen:*
  - Start mit basalen „Pass/Fail + Bonus/Malus" (wie in exams.json vorgesehen).
  - Erweiterte Notenmodelle als spätere Backlog-Items.

4.13 Umsetzungsschritte (strukturierter Plan)

Schritt 1 -- Kalenderschnittstelle definieren

- SlotRef-/DayRef-Typen in einem kleinen types.ts oder Kommentar definieren.
- calendar.js mit:
  - getCurrentSlotRef
  - getCalendarEntryForCurrentSlot
  - advancePhase, advanceDay\

    ausstatten und an JanusStateCore.time anbinden.

Schritt 2 -- Lessons-Grundlogik

- lessons.js implementieren:
  - getLessonsForCurrentSlot
  - runLesson (mit minimalen Effekten)
  - applyLessonOutcome (Platzhalter, später erweitert)

Schritt 3 -- ScoringEngine isolieren

- scoring.js schreiben:
  - addCirclePoints, addStudentPoints
  - getLeaderboard
- Direkt in ein/zwei Testmacro-Szenarien einbinden („+10 Punkte an Zirkel Salamander").

Schritt 4 -- ExamEngine integrieren

- exams.js mit:
  - getExamsForCurrentSlot
  - startExam, finalizeExam
  - minimaler Integration zur DSA5-Bridge:
    - z\. B. nur eine zentrale requestExamChecks-Funktion, die 1--2 Proben auslöst.

Schritt 5 -- SocialEngine & EventEngine

- social.js:
  - adjustRelation, adjustReputation, addTag/removeTag.
- events.js:
  - triggerEvent, das v. a. story.chronicle beschreibt und optional Scoring/Social nutzt.

Schritt 6 -- Hook-Integration & einfache Tests

- Ein/zweimal advancePhase() aus einem Testmacro rufen.
- Im Log prüfen:
  - Zeit korrekt weitergeschaltet?
  - Lessons/Exams für Slot korrekt gefunden?
  - Punkte/Beziehungen aktualisiert?

Schritt 7 -- Dokumentation / Übergabe an Phase 5 & 6

- Kurze Developer-Doku:

  - „Wenn du die Zeit weiterstellen willst → benutze academy.calendar.*."

  - „Wenn du Punkte vergeben willst → benutze academy.scoring.*."

  - „Wenn du eine Prüfung mit UI steuern willst → kombiniere academy.exams.\ + *system.rolls.*\."

Damit ist Phase 4 als „Akademie-Motor" ausreichend detailliert beschrieben, um:

- im Word-Dokument als vollwertiges Kapitel zu landen,
- und als Blaupause für deine Implementierung zu dienen, ohne dass du später alles umwerfen musst.

Als Nächstes können wir, wenn du bereit bist, Phase 5 (Hybrid & Atmosphere Controller) in derselben Tiefe spezifizieren -- inkl. Integration mit Playlists, Foundry-Audio und ggf. Modulen wie Simple Calendar oder Seasons & Stars, falls du da koppeln willst.

Phase 5 – Hybrid & Atmosphere Controller (Detailliert)

Phase 5: Hybrid & Atmosphere Controller -- Der Wow-Faktor -- Erweiterte Dokumentation

Ziel: Steuerung des Erlebnisses am Spieltisch, insbesondere des \"Second Screen\" (Beamer/TV) und der audiovisuellen Atmosphäre. Diese Module verwandeln Foundry VTT von einem reinen Regel-Tool in ein immersives Präsentationswerkzeug für eine hybride Runde.

Module & Funktionen

5.1 Beamer Controller (hybrid/display.js)

\$\$V7\$\$

Dies ist die Fernbedienung für den Beamer-Client. Der Beamer-Rechner ist in Foundry als spezieller User (\"Beamer\") eingeloggt. Dieses Modul sorgt dafür, dass dieser Client genau das anzeigt, was der GM will, ohne dass der GM ständig die Maus auf dem zweiten Bildschirm bewegen muss.

- **Funktion:**

  - **Socket-Kommunikation:** Der GM-Client sendet Befehle über Foundry-Sockets an den Beamer-Client. Der Beamer-Client lauscht auf diese Befehle und führt sie aus (z.B. \"Blende UI aus\", \"Zeige Bild X\").

  - **View-Modes:** Schaltet den Beamer in verschiedene Darstellungsmodi:

    - Scene: Zeigt die aktuelle Karte/Szene maximiert an. Die UI-Elemente (Sidebar, Hotbar) werden ausgeblendet (\"Cinema Mode\").

    - Dashboard: Zeigt ein \"Idle\"-Overlay an, wenn gerade keine Szene aktiv ist (z.B. in Pausen). Das Dashboard enthält Informationen wie das Akademiewappen, aktuelle News (Gossip), den Punktestand der Häuser oder den Kalender.

    - VisualNovel: Zeigt einen statischen oder animierten Hintergrund und blendet NPC-Portraits im Vordergrund ein, wenn Dialoge stattfinden. Dies simuliert den Stil von Visual Novels oder JRPGs und fokussiert die Aufmerksamkeit auf die Charaktere.

    - ShowToScreen: Zeigt ein Handout (Bild, Text) sofort im Vollbildmodus (Lightbox) auf dem Beamer an, ideal um Spielern Hinweise zu präsentieren.

5.2 Atmosphere Controller (atmosphere/controller.js)

Der Regisseur für Licht und Ton. Dieses Modul verknüpft die logische Spielwelt mit den audiovisuellen Assets in Foundry.

- **Funktion:**

- **Location-Mapping:** Verknüpft logische Orte (z.B. \"Bibliothek\") mit konkreten Playlists und Beleuchtungseinstellungen.

- **Mood-Management:** Setzt globale Stimmungen (Moods), die unabhängig vom Ort sein können (z.B. \"Kampf\", \"Spannung\", \"Gemütlich\"). Ein Wechsel der Mood kann die Playlist ändern, das Licht dimmen oder Wettereffekte (FXMaster) aktivieren.

- **Automatisierung:** Kann automatisch beim Szenenwechsel die passende Stimmung aktivieren, basierend auf Tags in der Szene oder im State.

5.3 Audio Master (hybrid/audio.js)

\$\$V7\$\$

Zentrales Audio-Routing für den Spieltisch. Da alle Spieler im selben Raum sind, wäre es chaotisch, wenn jeder Laptop Musik abspielt.

- **Funktion:**

- **Client-Filter:** Stellt sicher, dass Audio (Musik, Ambiance) *ausschließlich* auf dem Client abgespielt wird, der als \"Audio Master\" (meist der Beamer-Rechner) konfiguriert ist.

- **Mute-Logik:** Schaltet automatisch alle anderen Clients (Tablets der Spieler) stumm, um Echos und Überlagerungen zu verhindern.

- **Volume-Control:** Ermöglicht dem GM, die Lautstärke des Beamers direkt von seinem Interface aus zu steuern.

5.4 Visualizer (hybrid/visualizer.js)

\$\$V7\$\$

Feedback und \"Eye Candy\" für den Tisch. Verwandelt abstrakte Zahlen in gemeinsame Erlebnisse.

- **Funktion:**

- **Dice Sync:** Fängt Würfel-Events der Spieler-Tablets ab und rendert sie synchron als 3D-Dice auf dem Beamer. Alle sehen denselben Wurf zur selben Zeit rollen.

- **Erfolgs-Animationen:** Zeigt bei kritischen Erfolgen oder Patzern spezielle Animationen oder Popups auf dem Beamer an, um den Moment zu feiern.

- **Feedback-Overlays:** Kann kurze Nachrichten oder Symbole einblenden (z.B. \"Hauspunkte +10\", \"Neues Gerücht!\"), um Änderungen im Spielstatus visuell zu bestätigen.

🔗 Abhängigkeiten & Schnittstellen

Abhängigkeiten (Benötigt\...)

Der Hybrid-Layer baut auf der Infrastruktur des Cores auf.

- **Phase 1 (Core):**
  - Nutzt Config, um Szenen-IDs und Playlists aufzulösen.
  - Nutzt State, um den aktuellen Display-Status (welcher Modus ist aktiv?) und Audio-Status zu speichern und zu lesen.
- **Phase 1 (Director):** Wird vom Director aufgerufen, um Änderungen umzusetzen (z.B. \"Director: Öffne Szene X\" -\> \"Beamer Controller: Schalte auf Scene-Mode und zeige X\").
- **Foundry Sockets:** Essenziell für die Echtzeit-Kommunikation zwischen GM-Client und Beamer-Client, da normale Foundry-Aktionen (wie canvas.pan) normalerweise nur lokal wirken.

Schnittstellen (Bietet an\...)

Der Hybrid-Layer bietet High-Level-APIs für die UI und Skripte.

- **Display API:**
  - Display.setMode(mode, data): Wechselt den Anzeigemodus des Beamers.
  - Display.showImage(url): Zwingt den Beamer, ein bestimmtes Bild anzuzeigen.
  - Display.showOverlay(html): Blendet custom HTML-Content über der Szene ein.
- **Atmosphere API:**
  - Atmosphere.setMood(moodKey): Setzt eine definierte Stimmung (Audio + Licht + FX).
  - Atmosphere.setLocation(locationId): Setzt Stimmung passend zum Ort.
- **Hooks:**
  - Lauscht auf createChatMessage (für Dice Sync), um Würfelwürfe abzufangen.
  - janus7MoodChanged: Wenn sich die Stimmung ändert.

ChatGPT:

Jetzt sind wir bei Phase 5 -- also dem Teil, wo JANUS aufhört, nur clever zu sein, und endlich auch **stilvoll klingt**. 😄\

Ich formuliere dir das wieder so, dass du es direkt als Kapitel ins Word-Dokument übernehmen kannst.

Phase 5 -- Hybrid & Atmosphere Controller

Der Sound- und Stimmungs-Motor von JANUS7

5.1 Ziel & Scope der Phase

Ziel von Phase 5

Diese Phase bündelt alles, was mit **Stimmung & Atmosphäre** zu tun hat, in einem eigenen, klar gekapselten Layer:

- Steuerung von **Musik-Playlists** und **Ambience** über die Foundry-Audio-API (Playlists & Ambient Sounds).
- Zuweisung von **„Moods"** (Stimmungen) zu:
  - Szenen (Bibliothek, Schlafsaal, Mensa, Prüfungsraum),
  - Kalender-Slots (Unterricht, Prüfungen, Feste),
  - Events (Schulball, Disput, Alarm).
- Berücksichtigung des **Spielmodus** („nur Tisch, nur VTT, Hybrid"), ohne separate Tablet-Steuerung zu bauen.
- Optionale, saubere **Integrationspunkte** zu bestehenden Audio-/Zeit-Modulen (Simple Calendar, Simple Timekeeping, Wetter-/Soundscape-Module).

Nicht-Ziele (bewusst gestrichen)

- **Keine** eigene Remote-/Tablet-UI mehr, die Foundry fernsteuert.
- **Keine** tiefe Kopplung an ein bestimmtes Drittmodul (z. B. „nur Simple Calendar Pflicht"). Alles optional.
- **Keine** eigene Audio-Engine -- JANUS benutzt die Foundry-Mechanismen (Playlists, AmbientSound, AudioHelper).

5.2 Modulstruktur & Namespaces

Vorgeschlagene Struktur:

modules/janus7/

atmosphere/

controller.js // Zentrale Steuerung, Public API

profiles.js // Definition von Moods & Profilen

bindings.js // Mapping: Szene / Slot / Event → Mood

integrations.js // Optionale Brücken zu Simple Calendar, Timekeeping, Soundscape etc.

Globale Registrierung:

game.janus7.atmosphere = {

controller: JanusAtmosphereController,

profiles: JanusAtmosphereProfiles,

bindings: JanusAtmosphereBindings,

integrations: JanusAtmosphereIntegrations

};

5.3 Datenmodell & State

5.3.1 State im Janus-Core

Phase 5 nutzt einen eigenen Bereich in JanusStateCore, z. B.:

JanusStateCore.atmosphere = {

currentMoodId: string \| null,

lastSceneId: string \| null,

lastPlaylistKey: string \| null,

mode: \"table\" \| \"vtt\" \| \"hybrid\",

autoSceneBindingEnabled: boolean,

autoCalendarBindingEnabled: boolean,

volumePreferences: {

music: number; // 0--1

ambience: number;

}

};

Der State wird -- wie in Phase 1 vorgesehen -- nur über JanusStateCore-Transaktionen verändert (z. B. JanusState.updateAtmosphere(\...)).

5.3.2 Moods & Profile (profiles.js)

Ein **Mood** beschreibt, welche Atmosphäre aktiv sein soll, z. B.:

type MoodProfile = {

id: string; // \"LIBRARY_STUDY\", \"EXAM_TENSION\", \...

label: string; // Für UI / Logs

playlistKey?: string; // interner Name oder Playlist-ID in Foundry

playlistMode?: \"loop\" \| \"shuffle\" \| \"sequential\";

ambientTags?: string\[\]; // z.B. \[\"rain\", \"fireplace\"\]

ambientSceneBindings?: string\[\]; // optionale Referenzen auf AmbientSound-IDs

fadeInMs?: number;

fadeOutMs?: number;

volumeOverrides?: {

music?: number;

ambience?: number;

};

};

profiles.js stellt typischerweise eine Kombination aus:

- **statisch definierten Standardmoods** (z. B. Bibliothek, Mensa, Nacht im Schlafsaal, Sturm in Punin),
- optional konfigurierbaren oder im World-Config hinterlegten Profilen bereit.

5.4 Atmosphere Controller (controller.js)

5.4.1 Aufgabe

Der Controller ist die \\ einzige\\ Stelle, die:

- tatsächlich game.playlists, Playlist.playSound(), Playlist.stopAll() etc. aufruft,
- Ambient-Sounds der Szene ein-/ausblendet,
- JanusStateCore.atmosphere aktualisiert.

Er kapselt damit die gesamte Audio-Logik, sodass andere Phasen nur sagen müssen:\

„Gib mir **Mood X**" -- und nicht „Spiel Playlist Y, stoppe Z, setze Lautstärke".

5.4.2 Öffentliche API

Zentrale Methoden (High-Level)

- setMood(moodId: string, options?)

- Lookup des MoodProfile via profiles.js.
- Stop/Fade-out der alten Playlist (abhängig von Options).
- Start/Fade-in der neuen Playlist.
- ggf. Ambient-Sounds umschalten.
- Update von JanusStateCore.atmosphere.currentMoodId.

- fadeToMood(moodId: string, options?)
  - Komfortmethode, um sanft zu wechseln (z. B. 2--3 Sekunden Crossfade).
- applySceneMood(sceneId: string \| Scene)
  - Prüft bindings.js, ob für diese Szene ein moodId hinterlegt ist → setMood(moodId).
- applySlotMood(slotRef: SlotRef)
  - Prüft Bindings für Kalender-Slot (z. B. „Unterricht in Dämonologie" → Mood „Arcane Tension").
- stopAllAudio(options?)
  - Stoppt Musik und optional Ambience (z. B. für Pausen).

Low-Level / Utility

- playPlaylist(playlistKey: string, options?)
  - Sucht in game.playlists das entsprechende Playlist-Dokument und steuert es (start, loop, stop).
- setVolume(type: \"music\" \| \"ambience\", value: number)
  - Aktualisiert volumePreferences im State; konkrete Umsetzung kann via Playlist-/Sound-Volume erfolgen.

5.5 Bindings (bindings.js)

5.5.1 Aufgabe

bindings.js definiert, **wo** welche Moods greifen:

- Szene → Mood (z. B. Bibliothek → „LIBRARY_STUDY").
- Kalender-Slot → Mood (z. B. Praiosstag, Vormittag, Zauberkunde → „CLASSROOM_ARCANE").
- Ereignis → Mood (z. B. Schulball → „BALLROOM_FESTIVE").

5.5.2 Datenmodell

Beispiel:

```
type SceneBinding = {
sceneId: string;
moodId: string;
};
type SlotBinding = {
year?: number;
trimester?: number;
week?: number;
day?: string;
phase?: string;
lessonId?: string;
eventId?: string;
moodId: string;
};
```

bindings.js stellt Funktionen wie:

- getMoodForScene(sceneId): string \| null
- getMoodForSlot(slotRef, calendarEntry): string \| null
- registerSceneBinding(sceneId, moodId)
- registerSlotBinding(criteria, moodId)

Damit bleibt die Zuordnung **konfigurierbar** und kann auch später durch KI (Phase 7) oder Tools angepasst/generiert werden.

5.6 Integration mit Foundry-Audio

5.6.1 Playlists-API

Die Steuerung von Playlists erfolgt über game.playlists und Playlist-Methoden wie:

- playlist.playSound(sound)
- playlist.stopSound(sound)
- playlist.stopAll()

Grundidee:

- profiles.playlistKey ist entweder:
  - der **Name** eines Playlist-Dokuments, oder
  - ein eigener Key, der in einer Mapping-Tabelle auf eine Playlist-ID verweist.

Controller-Schritte z. B.:

75. Finde Playlist-Dokument:\

    const pl = game.playlists.find(p =\> p.name === \"Akademie -- Bibliothek\");

76. Stoppe ggf. andere Musik:\

    await pl.stopAll();

77. Spiele gewünschten Track oder Random:
    - Entweder pl.playSound(pl.sounds.contents\[0\]),
    - oder via playbackOrder und mode eine eigene Logik.

5.6.2 Ambient Sounds / Soundbits

Für Umgebungssounds nutzt JANUS:

- **AmbientSound-Dokumente** auf Szenen (z. B. Kaminfeuer, Regen, Markt).
- Optional kompatible Module wie **Ambient Soundbits**, das speziell aktivierbare Ambient-Sound-Objekte anbietet.

Der Controller muss nicht alle Ambients selbst erstellen, sondern:

- kennt die IDs/Names der relevanten AmbientSounds,
- kann sie aktivieren/deaktivieren (z. B. „aktiviere Regengeräusch für Mood STORM_NIGHT").

5.7 Optionale Modul-Integrationen (integrations.js)

Diese Datei enthält **optionale** Brücken zu verbreiteten Zeit-/Wetter-/Audio-Modulen:

- **Simple Calendar** -- Zeit/Datum-Integration; viele Module hängen sich an „World Time" drange.
- **Simple Timekeeping** -- Zeit, Kalender & Wetter mit Foundry-Weltzeit-Kopplung.
- **Weather Control / Simple Weather / Calendar-Weather** -- dynamisches Wetter, optional gekoppelt mit FXMaster für Wettereffekte.
- **Soundscape / Soundscape Adventure** -- komplexe Soundmischungen und automatisierte Soundscapes.

Wichtiges Designprinzip:

- JANUS7 bleibt **voll funktionsfähig ohne** eines dieser Module.
- Wenn ein Modul erkannt wird, kann integrations.js:
  - bei Zeit-/Wetterwechseln passende Moods vorschlagen oder automatisch schalten,
  - bei Soundscape/Soundscape Adventure vorhandene Profile ansteuern (z. B. „nutze dieses Soundscape-Profil statt Foundry-Playlist").

5.8 Hybrid-Aspekt (ohne Tablet-Steuerung)

„Hybrid" bedeutet hier:

- du kannst am echten Tisch spielen,
- gleichzeitig aber Foundry als **Atmosphäre- und Anzeige-Backend** nutzen (Musik, Handouts, Karten).

Phase 5 trägt dazu bei, indem sie:

- **Spielmodi** im State verwaltet (mode: \"table\" \| \"vtt\" \| \"hybrid\"),
- ermöglicht, dass du im table-Modus vielleicht weniger laute/komplexe Moods nutzt,
- automatisch alle Audioeffekte über Foundry laufen lässt -- die Spieler, ob remote oder am Tisch, hören **dieselbe** Szene.

Keine direkte Netzwerk-/Tablet-Fernsteuerung, sondern:

- klare API für Moods,
- UI-Buttons in Phase 6, die du z. B. auf dem Laptop bedienst,
- Foundry selbst sorgt für Verteilung an alle Clients.

5.9 Abhängigkeiten zu anderen Phasen

Hängt ab von:

- **Phase 1 -- Core & State**
  - JanusStateCore.atmosphere (State)
  - Logger / Fehlerbehandlung
- **Phase 2 -- Static Academy Data**
  - Szenen-IDs, Location-IDs (Bibliothek, Mensa, Schlafsaal, Hörsäle)
  - Event-/Kalenderinformationen (Feste, Prüfungen, besondere Tage)
- **Phase 4 -- Academy Simulation**
  - Zeit-/Slot-Informationen (calendar.getCurrentSlotRef())

- Hooks aus Lessons/Exams/Events, die Moods triggern („Prüfung beginnt" → „EXAM_TENSION").

Wird genutzt von:

- **Phase 6 -- User Interfaces**

  - Control Panel bekommt Buttons wie „Aktuelle Szene-Mood anwenden", „Nachtmodus", „Prüfungsatmosphäre".

- **Phase 7 -- KI-Integration**

  - KI kann anhand von Kalender/State Vorschläge für neue Moods & Bindings generieren oder Moods automatisch setzen.

5.10 Aufwandsschätzung Phase 5

Grober Rahmen:

78. **State & Grundgerüst (controller.js, State-Felder)**

    - 1--1,5 PT

79. **Moods & Profiles (profiles.js)**

    - 1--2 PT (inkl. 8--15 Basisprofile für die Akademie)

80. **Bindings (bindings.js)**

    - 1--1,5 PT (inkl. Szenen- und Slot-Bindings API)

81. **Playlist-Steuerung & Ambient-Kopplung**

    - 2--3 PT (inkl. Tests mit Foundry-API)

82. **Optionale Integrationen (integrations.js)**

    - 1,5--3 PT (je nach Anzahl angebundener Module)

83. **Tests & Feintuning (Crossfade, Fallbacks, Logging)**

    - 1,5--2 PT

Summe: ca. **8--13 Personentage**, abhängig davon, wie umfangreich du die Modul-Integrationen tatsächlich umsetzt.

5.11 Risiken & Gegenmaßnahmen

84. **Risiko: Audio-API-/Version-Änderungen in Foundry**

    - *Problem:* Zwischen v10--v13 können sich Details an Playlist/AmbientSound ändern.

- *Gegenmaßnahmen:*
  - Alle direkten API-Aufrufe in controller.js bündeln.
  - Ein kleiner diagnostics-Check (z. B. AudioHelper, game.playlists) beim Start.

85. **Risiko: Abhängigkeit von Drittmodulen**

- *Problem:* Simple Calendar, Wetter- oder Soundscape-Module sind optional, Versionen divergieren.
- *Gegenmaßnahmen:*
  - Integrationen strikt optional halten.
  - Existenz & Version der Module prüfen; bei Problemen sauber deaktivieren.

86. **Risiko: Überladene Konfiguration**

- *Problem:* Zu viele Moods/Bindings → unübersichtliche Konfig.
- *Gegenmaßnahmen:*
  - Start mit kleinem Set von Basis-Moods.
  - Backlog-Tickets für einen späteren „Mood Manager" in Phase 6/8.

87. **Risiko: Performance/Audio-Lags bei vielen Sounds**

- *Problem:* Zu viele Playlists/Ambients gleichzeitig.
- *Gegenmaßnahmen:*
  - Controller sorgt für klare Regeln (z. B. maximal 1 Musik-Playlist + X Ambients).
  - Option im State/Settings für „Light Mode" mit weniger Ambience.

5.12 Umsetzungsschritte (strukturierter Plan)

Schritt 1 -- State & Skeleton

- JanusStateCore.atmosphere-Struktur definieren.
- atmosphere/controller.js mit Platzhalterfunktionen (setMood, applySceneMood, stopAllAudio).

Schritt 2 -- Moods & Profiles

- profiles.js mit 8--15 Basisprofilen für:
  - Bibliothek, Mensa, Schlafsaal, Tagesunterricht, Nacht, Sturm, Prüfung, Fest.

- Mapping zu existierenden Playlists (z. B. aus Game Audio Bundle / eigenen Modulen).

Schritt 3 -- Playlists & Ambience

- Implementierung der Playlist-Steuerung mit game.playlists + Playlist.playSound/stopAll.
- Einfache Ambient-Toggles für wichtige Sounds in Schlüsselszenen.

Schritt 4 -- Bindings & Kalender-Anbindung

- bindings.js:
  - Szene → Mood-Bindings für alle Kernorte der Akademie.
  - Slot-Bindings für Standardstunden (z. B. „Praiosstag Vormittag: Unterricht Rakorium → MOOD_CLASSROOM_ARCANE").
- academy.calendar (Phase 4) ruft bei Zeitwechsel optional atmosphere.controller.applySlotMood(\...) auf.

Schritt 5 -- Optional Module Hooks

- In integrations.js:
  - Check auf Simple Calendar / Simple Timekeeping / Weather Control / Soundscape-Module.
  - Wenn vorhanden:
    - bei Wetterwechsel → ggf. Mood ändern („Regen in Punin" → „STORMY_STUDY").

Schritt 6 -- Tests im echten Akademie-Szenario

- Testworld mit:
  - Bibliothek, Schlafsaal, Hörsaal, Hof, Prüfungsraum.
  - Mindestens einer Woche Stundenplan (Phase 4).
- Durchklicken:
  - Szenenwechsel → applySceneMood
  - Zeitfortschritt → applySlotMood
  - manuelle Mood-Wechsel über Konsolenbefehle/Macros (bis UI in Phase 6 da ist).

Schritt 7 -- Dokumentation & Übergabe

- Kurz-Doku für dich als SL/Dev:
  - „Wie lege ich neue Moods an?"
  - „Wie mappe ich eine Szene oder ein Event auf ein Mood?"
  - „Wie deaktiviere ich Auto-Moods, wenn ich manuell DJ sein will?"

Damit ist Phase 5 so ausgearbeitet, dass du:

- sie als eigenes, vollständiges Kapitel in deiner Roadmap führen kannst,
- später beim Implementieren weißt, **wo** du Foundry-API, State & Akademie-Logik sauber entkoppelt hältst.

Als nächstes wartet dann Phase 6 (User Interfaces) -- da können wir die ganzen Knöpfe, Slider und Ansichtspanels auf ApplicationV2-Ebene entwerfen, die genau diese Atmosphere-/Calendar-/Scoring-Engines ansteuern.

Phase 6 – User Interfaces (Detailliert)

Phase 6: User Interfaces -- Die Schnittstelle zum Menschen -- Erweiterte Dokumentation

Ziel: Entwicklung intuitiver, rollenspezifischer Bedienoberflächen, die die komplexe Logik des Backends (Phasen 1-5) für Menschen nutzbar machen. Das Design-Ziel ist „Information at a Glance" für den GM und „Focus & Immersion" für die Spieler. Die UIs sollen nicht nur funktional sein, sondern das Spielgefühl der Akademie unterstützen.

Module & Funktionen

6.1 GM Control Panel (ui/apps/control-panel.js)

Die Kommandozentrale für den Spielleiter. Dieses Fenster ist das „Cockpit", aus dem die gesamte Akademie-Simulation gesteuert wird. Es ist als ApplicationV2 implementiert, um modernste Foundry-Features (wie CSS-Variablen und verbesserte Event-Handler) zu nutzen.

- **Architektur:** Das Panel ist modular aufgebaut und lädt Inhalte dynamisch basierend auf dem aktiven Tab. Es lauscht auf Hooks (janus7StateChanged), um sich in Echtzeit zu aktualisieren, ohne die gesamte Anwendung neu zu rendern (Reactivity).
- Tabs & Funktionen:

  - Übersicht (Dashboard):

    - **Zeitsteuerung:** Große Buttons für „+1 Phase", „+1 Tag", „Nächste Stunde". Visuelle Anzeige der aktuellen Zeit (z.B. „Praiosstag, Nachmittag - Woche 3, Herbsttrimester"). Ein Fortschrittsbalken visualisiert die Zeit bis zur nächsten wichtigen Deadline (z.B. Prüfung).

    - **Aktueller Kontext:** Zeigt an, welche Lektion oder welches Event *jetzt gerade* stattfindet (Titel, Lehrer, Ort). Ein Klick öffnet das Journal oder die Szene.

    - **Quick Actions:** Schnellzugriff auf häufige Funktionen wie „Punkte vergeben", „Musik stoppen" oder „Beamer schwärzen".

  - Akademie (Simulation):

    - **Stundenplan:** Eine Wochenansicht des Kalenders mit Drag-and-Drop-Funktionalität, um Lektionen spontan zu verschieben. Farbcodierung zeigt den Status (geplant, abgeschlossen, ausgefallen).

    - **Scoring-Matrix:** Tabellarische Ansicht der Hauspunkte und Schüler-Rankings mit direkter Editiermöglichkeit (+/- Buttons). Warnungen bei kritischen Werten (z.B. Disziplinarverfahren droht).

    - **Ökonomie & Social:** Ein Editor für die Beziehungsmatrix und den Schuldenstand, um manuelle Korrekturen an der Simulation vorzunehmen.

- Regie (Hybrid & Atmosphere):
    - **Beamer-Steuerung:** Umschalter für die Modi (Scene, Dashboard, VN). Dropzones, um Bilder oder Journale direkt auf den Beamer zu „werfen". Vorschau-Fenster zeigt, was der Beamer gerade anzeigt.
    - **Mood-Board:** Grid mit Buttons für verschiedene Stimmungen (z.B. „Spannung", „Unterricht", „Taverne"), die Licht und Ton gleichzeitig ändern. Preset-Management für eigene Szenarien.
- KI-Labor:
    - Statusanzeige laufender Generierungs-Jobs.
    - Drag-and-Drop-Zone für den Import von world_build.json Dateien.
    - Export-Buttons für den aktuellen State.
    - Log-Anzeige für KI-Interaktionen.

6.2 Student Dashboard (ui/apps/student-dashboard.js)

\$\$V7\$\$

Eine spezialisierte Web-App-Ansicht für Tablets (Mobile-First Design). Sie ersetzt den überladenen Standard-Charakterbogen von Foundry durch eine fokussierte, fingerfreundliche Oberfläche, die speziell auf Touch-Bedienung optimiert ist.

- **Design-Philosophie:** „Weniger ist mehr". Auf einem kleinen Tablet-Screen sollen nur die Informationen stehen, die *jetzt* relevant sind. Große Buttons, klare Typografie.
- Modi:
    - **Stundenplan-Ansicht:** Zeigt, wo der Schüler sein muss. Tippen auf eine Lektion zeigt Details (Lehrer, benötigtes Buch, Hausaufgaben).
    - **Fokus-Modus (Unterricht/Kampf):** Wenn eine Probe verlangt wird (z.B. durch den GM getriggert), blendet das Dashboard alles aus außer einem riesigen „Würfeln"-Button für das relevante Talent (z.B. „Magiekunde"). Das Ergebnis wird groß angezeigt und an den Beamer gesendet.
    - **Inventar & Aufgaben:** Vereinfachte Listenansicht für Quest-Items („Zutaten für Freitag") und Hausaufgaben. Checkboxen zum Abhaken erledigter Aufgaben.
    - **Digitales Notizbuch:** Ein Textfeld, das automatisch synchronisiert wird. Die Eingaben werden exportiert, damit die KI später weiß, was der Schüler sich notiert hat („Verständnis-Check").

- **Messenger:** Ein privater Kanal für Zettelchen an andere Schüler oder geheime Nachrichten vom GM („Du spürst ein Kribbeln\..."). Unterstützt auch das Senden von Items oder Geld.

6.3 Meta-Game UI (ui/apps/meta.js)

\$\$V7\$\$

Gamification-Elemente, die die Langzeitmotivation fördern und den „Schul-Charakter" unterstreichen.

- Jahresrückblick (The ‚Wrapped' Feature):

  - Generiert am Ende eines Jahres (oder auf Knopfdruck) eine statische HTML-Seite mit Statistiken: „Du hast 145 Mal Magiekunde gewürfelt", „Dein Lieblings-NPC war Sequin", „Du hast 50 Hauspunkte verloren", „Deine häufigste Aktion war ‚Schlafen'".

- Meta-Shop:

  - Ein Interface, in dem Spieler ihre „Fleißpunkte" (Meta-Währung für gutes Rollenspiel) ausgeben können.

  - **Kategorien:** Kosmetik (neue Würfelfarben, Avatar-Rahmen für das Dashboard), Komfort (einmaliger Reroll-Gutschein), In-Game-Vorteile (Tipp vom Hausmeister, Zugang zur Lehrer-Lounge).

- Sammelalbum (Das Grimoire):

  - Eine visuelle Sammlung (Grid-View) von freigeschaltetem Wissen. Wenn ein Schüler einen Dämonen oder eine Pflanze erfolgreich identifiziert, wird die entsprechende „Karte" im Album farbig und enthüllt Lore-Texte und Bilder. Motiviert zum Entdecken („Sammle sie alle!").

🔗 Abhängigkeiten & Schnittstellen

Abhängigkeiten (Benötigt\...)

Das UI ist die Spitze des Eisbergs und hängt von fast allen anderen Systemen ab.

- **Phase 1 (Core - State):** Liest alle Daten (Zeit, Punkte, Display-Modus) aus dem State. Ohne State ist das UI leer.
- **Phase 2 (Academy Data):** Benötigt Kalender- und Lektionsdaten für die Stundenplan-Anzeige.
- **Phase 4 (Simulation):** Ruft APIs auf, um Punkte zu ändern oder Ökonomie-Daten anzuzeigen.
- **Phase 5 (Hybrid):** Sendet Befehle an den Beamer-Controller.
- **Foundry ApplicationV2:** Das technische Framework für die Fenster-Erstellung.

Schnittstellen (Bietet an\...)

- **Keine Logik-API:** Das UI enthält *keine* Geschäftslogik. Es berechnet keine Punkte und würfelt keine Proben selbst. Es dient nur der Anzeige und Eingabe.
- **User Input Events:** Es transformiert Benutzeraktionen in API-Aufrufe an die unteren Schichten:

  - Klick auf „+1 Tag" -\> ruft Calendar.advanceTime() auf.
  - Klick auf „Show on Beamer" -\> ruft Display.setMode() auf.
  - Drag auf Stundenplan -\> ruft Calendar.moveLesson() auf.

ChatGPT:

Phase-6-Zeit, jetzt kriegt JANUS endlich Knöpfe, Slider und fancy Panels. 😄 \

Ich schreibe dir das wieder so, dass du es direkt als Kapitel ins Word-Dokument kleben kannst.

Phase 6 -- User Interfaces (ApplicationV2)

Das Bedienpult von JANUS7

6.1 Ziel & Scope der Phase

Ziel von Phase 6\

In dieser Phase entsteht die komplette **Benutzeroberfläche** von JANUS7 auf Basis des neuen **ApplicationV2-Frameworks** von Foundry v13.

Die UIs sollen:

- die Engines aus den Phasen 1--5 (State, Academy, DSA5-Bridge, Atmosphere) **sichtbar und bedienbar** machen,
- **keine** eigene Business-Logik enthalten, sondern nur die vorhandenen Public APIs nutzen,
- konsequent auf **ApplicationV2** basieren, ohne neue FormApplication-Altlasten (FormApplication ist in v13 deprecatet).

Im Scope:

- Basisklasse für JANUS-UIs (JanusBaseApp, ApplicationV2).
- **JANUS Control Panel** (zentrales GM-Dashboard).
- **Academy Overview** (Kalender/Wochenplan).
- **Scoring & Social View** (Hauspunkte, Schüler, Reputation).
- **Atmosphere DJ** (Mood-Auswahl, Lautstärke).
- Einfache **Konfig-Dialoge** (z. B. für Integrations- und Komfort-Settings).
- Integration mit Foundry-Settings (game.settings, SettingsConfig).

Nicht im Scope (bewusst gestrichen / Backlog):

- Keine externe Tablet-WebApp, kein separater Remote-Client.

- Kein vollwertiger „World-Builder" im Browser -- Lehrpläne, Kalender etc. bleiben **einfach editierbar**, aber nicht als gigantischer Editor-Monolith.
- Keine direkten Manipulationen an DSA5-Interna aus den UIs (immer über die Bridge, Phase 3).

6.2 UI-Architektur & Verzeichnisstruktur

Vorschlag:

modules/janus7/

ui/

base/

JanusBaseApp.js // gemeinsame Basis für alle UIs (ApplicationV2)

apps/

JanusControlPanel.js // zentrales GM-Dashboard

JanusAcademyOverview.js // Kalender/Wochenplan

JanusScoringView.js // Hauspunkte & Leaderboard

JanusSocialView.js // Soziale Beziehungen (optional kombiniert)

JanusAtmosphereDJ.js // Atmosphäre/Moods

JanusStateInspector.js // Dev-/Debug-Ansicht (optional)

JanusConfigPanel.js // erweiterte Moduleinstellungen (optional)

templates/ // Handlebars/HTML-Templates für ApplicationV2

control-panel.hbs

academy-overview.hbs

scoring-view.hbs

social-view.hbs

atmosphere-dj.hbs

state-inspector.hbs

config-panel.hbs

Alle Apps basieren auf **ApplicationV2** (statt Application/FormApplication). Foundry hat v13 konsequent auf ApplicationV2 umgestellt, u. a. für eigene UIs (Compendiums, Playlists, Settings etc.), daher ist das der zukunftssichere Weg.

6.3 JanusBaseApp (UI-Basisklasse)

6.3.1 Aufgabe

JanusBaseApp kapselt alles, was alle JANUS-Apps gemeinsam haben:

- Module-ID, CSS-Klassen, i18n-Prefix,
- Standard-Fensteroptionen (Breite, Höhe, Position),
- einheitliche Header-Buttons (z. B. „Pin", „Refresh", „Dokumentation"),
- Logging & Fehlerhandling (Anbindung an Phase 1-Logger),
- Hooks wie getHeaderControlsApplicationV2 für zusätzliche Controls.

6.3.2 Konfiguration

Typische static DEFAULT_OPTIONS:

- id: "janus-control-panel" (per App unterschiedlich)
- classes: \["janus", „janus-app", „theme-dark"\]
- position: { width: 800, height: 600 }

JanusBaseApp kann auf **HandlebarsApplicationMixin** aufbauen, wie im ApplicationV2-Guide gezeigt, damit du Template-Rendering komfortabel handhabst.

6.3.3 Öffentliche API

- render(force?: boolean) -- inherited, ggf. Convenience-Wrapper.
- refresh() -- Shortcut, der render(false) auslöst.
- open() / close() -- syntactic sugar für render(true) und close().
- Optional: static showSingleton() -- sorgt dafür, dass pro App nur eine Instanz geöffnet ist.

6.4 UI-State & Settings

UI-Zustand ist primär **clientbezogen** (welche Panels offen, welche Ansicht gewählt) und sollte nicht in den globalen Kampagnen-State (Phase 1) gemischt werden.

6.4.1 JanusStateCore.ui (nur das Nötigste global)

Im globalen State (Phase 1) ist nur sinnvoll:

JanusStateCore.ui = {

// z.B. globale Sichtbarkeit bestimmter Panels:

controlPanelPinned: boolean;

showDebugTools: boolean;

};

Alles andere -- Fensterpositionen, Client-Präferenzen -- sollte als **Client-Setting** gespeichert werden.

6.4.2 Client-Settings für UI

Über game.settings.register kannst du pro Client UI-Settings speichern, z. B.:

- janus7.ui.controlPanelOpen (bool)
- janus7.ui.controlPanelPosition (JSON)
- janus7.ui.preferredTheme (light/dark)
- janus7.ui.showAtmosphereDock

Die Anzeige im Foundry-Settings-Panel läuft wie gewohnt über die SettingsConfig-App, die alle config: true-Settings rendert.

6.5 Haupt-UI 1: JANUS Control Panel

6.5.1 Aufgabe

Das **JANUS Control Panel** ist das zentrale GM-Dashboard:

- Überblick über **Zeit/Kalender**,
- Schnellaktionen (Zeit vor/ zurück, Szene öffnen, Mood wechseln),
- Access zu Sub-Apps (Academy, Scoring, Atmosphere etc.),
- ein „Now"-Bereich: „Was ist in diesem Slot wichtig?"

6.5.2 Layout (High-Level)

Tabs oder Sektionen:

88. Overview

    - aktueller Slot (academy.calendar.getCurrentSlotRef()),
    - heutige Lessons/Exams/Events,
    - Buttons: „Nächste Phase", „Nächster Tag", „Wochenübersicht".

89. Scenes & Locations

    - Liste wichtiger Akademie-Szenen (Phase 2 Ortsdatenbank),
    - Buttons „Szene aktivieren", optional Direktlink zu Foundry-Scene-Config.

90. Quick Actions

    - Buttons für häufige Proben (via Phase 3-Bridge),

- Buttons für Standard-Moods (via Phase 5).

91. Diagnostics / Tools

- Links zu StateInspector, Config, ggf. KI-Export (Phase 7).

6.5.3 Schnittstellen & Abhängigkeiten

ControlPanel greift zu auf:

- game.janus7.state (Phase 1)
  - Zeit, Akademie-State, Scoring-Kurzinfo.
- game.janus7.academy.calendar (Phase 4)
  - getCurrentSlotRef, advancePhase, advanceDay.
- game.janus7.academy.lessons / .exams / .events (Phase 4)
  - getLessonsForCurrentSlot(), getExamsForCurrentSlot().
- game.janus7.atmosphere.controller (Phase 5)
  - applySlotMood, setMood.
- game.janus7.system.rolls (Phase 3)
  - für „Quick Probes" (z. B. Sinnesschärfe, Mut, Magiekunde).

Wichtig:\

Das Control Panel ruft **nur** Methoden der Engines auf, keine direkte Manipulation an JanusStateCore und kein direkter Zugriff auf DSA5-Systemobjekte.

6.6 Haupt-UI 2: Academy Overview (Kalender/Wochenplan)

6.6.1 Aufgabe

Die Academy Overview zeigt den **Stundenplan** und die akademischen Strukturen:

- Wochen-/Tagesansicht mit Slots,
- welche Lektion/Prüfung/Event in welchem Slot liegt,
- evtl. Markierung, was bereits „durch" ist.

6.6.2 UI-Features

- Grid-Ansicht: Tage (Praiosstag ...) vs. Phasen (Morgen, Vormittag ...).
- Click auf einen Slot:
  - zeigt Details zur Lesson/Exam/Event,
  - Buttons:

- „Slot im Control Panel fokussieren",
- optional: „Lesson im Detail anzeigen".

6.6.3 Schnittstellen

- academy.calendar:
  - getCalendarEntriesForDay(dayRef),
  - getCalendarEntryForCurrentSlot.
- AcademyData (Phase 2):
  - getLesson(lessonId),
  - getExam(examId),
  - getEvent(eventId).
- optional academy.events:
  - für Folgeaktionen, z. B. automatische Eventtrigger.

Zusätzlich kann die Overview über Header-Buttons (ApplicationV2-Hook getHeaderControlsApplicationV2) z. B. eine „Heute"-Schaltfläche bereitstellen.

6.7 Haupt-UI 3: Scoring & Social Views

6.7.1 Scoring View

Zweck: Einsicht und Steuerung von:

- Haus-/Zirkelpunkten,
- individuellen Schülerpunkten,
- Punktvergabe-Historie.

Schnittstellen:

- academy.scoring (Phase 4):
  - getCircleScore, getStudentScore, getLeaderboard,
  - addCirclePoints, addStudentPoints.
- JanusStateCore.scoring:
  - zur Anzeige, nicht zur direkten Manipulation.

UI-Features:

- Haus-Leaderboard (Balkendiagramm/Ranking).

- Liste der Schüler mit Punkten.
- Buttons „+5 Punkte Zirkel Salamander" etc. (natürlich Engine-Calls).

6.7.2 Social View

Zweck: Einsicht in:

- Relations zwischen NPCs/SCs,
- Reputation pro „Scope" (Akademie, Lehrkörper, Mitschüler),
- Tags/Labels.

Schnittstellen:

- academy.social (Phase 4):
  - adjustRelation, adjustReputation, addTag/removeTag.
- JanusStateCore.social:
  - Anzeige der Werte.

UI-Features:

- Auswahl eines Charakters → Anzeige relevanter Beziehungen.
- Einfache Buttons:
  - „Beziehung verbessern/verschlechtern",
  - „Tag hinzufügen/entfernen".

6.8 Haupt-UI 4: Atmosphere DJ

6.8.1 Aufgabe

Eine klar fokussierte UI, um:

- aktuelle **Mood** zu sehen,
- Moods zu wechseln,
- Lautstärke zu kontrollieren,
- ggf. Auto-Bindings (Kalender/Scenes) ein-/auszuschalten.

6.8.2 Schnittstellen

- atmosphere.controller (Phase 5):
  - setMood, fadeToMood, applySceneMood, applySlotMood,
  - setVolume.
- atmosphere.profiles:
  - listMoods(), getMood(moodId).

- atmosphere.bindings:
  - getMoodForScene, getMoodForSlot.

UI-Features:

- Liste aller Moods inkl. kurzer Beschreibung.
- Toggle für:
  - „Automatische Moods nach Szene",
  - „Automatische Moods nach Kalender".
- Slider für Musik-/Ambience-Lautstärke (speichern in Client-Settings).

Die tatsächliche Playlist- & Ambient-Steuerung erfolgt weiterhin nur über den Controller, der die Foundry-Audio-API nutzt.

6.9 Zusatz-UI: State Inspector & Config Panel (Dev/Power-User)

6.9.1 State Inspector

Kleine App, die:

- den aktuellen JanusStateCore strukturiert anzeigt (read-only),
- ggf. JSON-Export/Copy-Button anbietet,
- Filter (z. B. nur time, nur scoring, nur story).

Das ist Gold wert beim Debuggen oder beim manuellen „Stachel"-Anpassen vor einer Session.

6.9.2 Config Panel

Optionales Panel für komplexere Konfigs, die du nicht in zig Settings-Einträgen vergraben willst, z. B.:

- Mappings von Scenes ↔ Moods,
- globale Switches (z. B. „Exams nur als Story, keine Proben erzwingen").

Wichtig:\

Auch dieses Panel schreibt **nur** in:

- game.settings (für Konfigurationen),
- oder in definierte Bereiche von JanusStateCore über die Engine-APIs.

6.10 ApplicationV2-Details & Hooks

6.10.1 ApplicationV2-Basics

- Jede UI-App ist eine Klasse, die ApplicationV2 erweitert.
- Rendering-Logik:

- _prepareContext() bereitet Daten für das Template auf.
- _renderInner(context, options) gibt ein HTMLElement zurück, das in den DOM eingesetzt wird.

- ApplicationV2 nutzt standardisierte Hooks zur Erweiterung, z. B. getHeaderControlsApplicationV2, um Header-Buttons zu ergänzen.

6.10.2 Keine FormApplication mehr

- FormApplication (v1) ist ab v13 **deprecated**. Neue Implementierungen sollten auf ApplicationV2 basieren und ihren eigenen Formular-Submit-Flow schreiben.
- Falls du Form-ähnliche UIs brauchst (z. B. im Config-Panel), solltest du:
  - normales \<form\> im Template nutzen,
  - Event-Handler in deiner ApplicationV2 definieren (z. B. this._onSubmit(event)),
  - selbst bestimmen, welche Engine-API du aufrufst.

6.11 Abhängigkeiten zu anderen Phasen

Hängt ab von:

- Phase 1 -- Core & State
  - JanusStateCore (Zeit, scoring, social, story),
  - Logger/Fehlerbehandlung für UI-Actions.
- Phase 2 -- Static Academy Data
  - Szenen, Orte, Lektionen, Exams, Events für Anzeige.
- Phase 3 -- DSA5 System Bridge
  - Proben-Buttons im Control Panel, Exam-Auslöser.
- Phase 4 -- Academy Simulation
  - Calendar/Slot-Berechnungen, Lesson/Exam-Logik.
- Phase 5 -- Atmosphere
  - Moods, Playliststeuerung, Auto-Bindings.

Wird genutzt von:

- Phase 7 -- KI Integration
  - KI-Prozesse können UI-Aktionen anregen (z. B. Vorschlag: „In der nächsten Session diese Lektion & dieses Event spielen"),

- aber Phase 7 selbst hat keine eigene UI -- sie nutzt Phase-6-Apps oder erzeugt Daten, die dort angezeigt werden.

6.12 Aufwandsschätzung Phase 6

Sehr grob, 1 Person, mit deinen Skills & Vorarbeiten:

92. JanusBaseApp & UI-State-Grundlagen

- Basisklasse, generische Styles, Settings-Hooks\

  → 1,5--2 PT

93. Control Panel

- Template, Zeit/Slot-Anzeige, Quick Actions, Engine-Calls\

  → 2--3 PT

94. Academy Overview

- Wochenplan-Grid, Slot-Details, Navigation\

  → 2--3 PT

95. Scoring & Social Views

- Leaderboard, Schülerliste, einfaches Relation-UI\

  → 2--3 PT

96. Atmosphere DJ

- Mood-Liste, Set/Fade, Volume-Kontrollen, Auto-Bind-Toggles\

  → 2--3 PT

97. State Inspector & optional Config Panel

- Inspector: read-only, JSON/TreeView
- Config: minimaler Editor für ein/zwei kritische Mappings\

  → 2--3 PT

98. UX-Feinschliff & Tests (Keyboard, Responsiveness, Dark/Light)\

→ 1,5--2,5 PT

Summe: ca. **13--19 Personentage**, je nachdem, wie hübsch & komfortabel du die UIs gestaltest.

6.13 Risiken & Gegenmaßnahmen

99. Risiko: UI-Logik & Business-Logik vermischen sich

- *Problem:* Du fängst an, im UI direkt JanusStateCore zu verändern oder DSA5-Objekte anzufassen.
- Gegenmaßnahmen:
  - Klare Regel: UI darf **nur** Engine-APIs aufrufen (Phase 1--5).
  - Kein direktes game.actors.update o. Ä. in den UI-Klassen.

100. Risiko: ApplicationV2-API ändert sich noch

- *Problem:* V13 hat ApplicationV2 massiv gepusht; Feinheiten können in zukünftigen Versionen verschoben werden.
- Gegenmaßnahmen:
  - Alle ApplicationV2-Spezifika in JanusBaseApp bündeln.
  - Möglichst nah an den offiziellen ApplicationV2-Guides bleiben.

101. Risiko: UI wird überladen

- *Problem:* Control Panel kriegt „alles" -- wird unübersichtlich.
- Gegenmaßnahmen:
  - Strikte Trennung nach Apps (Control Panel, Overview, Scoring, DJ).
  - Für seltene Funktionen lieber separate Unter-Apps statt in den Main-Screen stopfen.

102. Risiko: Performance bei vielen Re-Renders

- *Problem:* Du triggest zu viele render()-Aufrufe bei kleinen State-Änderungen.
- Gegenmaßnahmen:
  - Renders throttlen oder nur selektiv refreshen.
  - Hooks/Hinweise sinnvoll nutzen, z. B. nur bei relevanten State-Events neu rendern.

6.14 Umsetzungsschritte (konkreter Plan)

Schritt 1 -- Basisklasse

- JanusBaseApp implementieren:
  - Extends ApplicationV2,
  - Standard-Options,
  - Template-Loader,

- Header-Controls-Hook.

Schritt 2 -- Minimal Control Panel

- Einfache Version mit:
  - Anzeige currentSlotRef,
  - Buttons „Nächste Phase", „Nächster Tag" (rufen academy.calendar.*),
  - Test-Button „Sinnesschärfe-Probe" (ruft Phase-3-Roll-API).

Schritt 3 -- Academy Overview

- Grid-Ansicht mit Dummy-Daten → danach Anbindung an echte calendar- & lessons-Daten.

Schritt 4 -- Scoring & Social

- Scoring-Tab (Leaderboard),
- einfache Relation-Anzeige.

Schritt 5 -- Atmosphere DJ

- Liste von 5--10 Moods,
- Buttons zum Wechseln, Lautstärke-Slider.

Schritt 6 -- State Inspector & Config

- Inspector: JSON/Tree-View des State (read-only),
- Config: 1--2 exemplarische Mappings, gespeichert als Settings.

Schritt 7 -- Polishing & Doku

- Hotkeys (optional),
- kleine Tooltips,
- kurze Dev-Doku:
  - „So öffnest du Control Panel",
  - „So erweiterst du eine UI mit neuen Buttons."

Wenn du magst, können wir als nächsten Schritt **eine konkrete API-Skizze für JanusControlPanel** oder JanusBaseApp machen -- also quasi das Grundgerüst in Code, das du direkt ins Modul schreiben kannst. Dann wird Phase 6 vom Architekturkapitel zur fast-fertigen Implementationsvorlage.

Phase 7 – KI-Integration (Detailliert)

Phase 7: KI Integration (Roundtrip) -- Der geschlossene Kreis -- Erweiterte Dokumentation

Ziel: Die Etablierung eines robusten, nahtlosen und asynchronen Datenaustauschs mit Large Language Models (LLMs) zur Generierung von dynamischen Kampagneninhalten. Das System soll den manuellen Aufwand von Copy-Paste minimieren, die Fehleranfälligkeit durch menschliche Übertragung eliminieren und der KI einen präzisen, kuratierten Kontext liefern, um „Halluzinationen" (das Erfinden nicht existenter Fakten oder Regeln) effektiv zu verhindern.

Module & Funktionen

7.1 Context Builder (ki/export.js)

Dieses Modul ist der „Regisseur" des Prompts. Es ist dafür verantwortlich, den „Prompt-Kontext" intelligent zusammenzustellen. Da LLMs ein begrenztes Kontext-Fenster (Token-Limit) haben und zu viele irrelevante Informationen die Qualität der Antworten verwässern („Lost in the Middle"-Phänomen), können wir nicht einfach den gesamten Datenbank-Dump senden. Der Context Builder muss als intelligenter Filter agieren.

- Funktion & Logik:

- **State-Snapshot (Der IST-Zustand):** Extrahiert den aktuellen Zustand aus core/state.js. Dies umfasst nicht nur harte Zahlen wie Zeit und Hauspunkte, sondern auch den „atmosphärischen Status" (z.B. aktuelle Bedrohungsstufe/Doom Clock) und aktive Plot-Flags.

  - *Beispiel:* „Woche 3, Praiosstag, Morgen. Haus Salamander führt. Bedrohung: Niedrig."

- **Kalender-Vorschau (Der SOLL-Zustand):** Blick in die Zukunft via academy/calendar.js. Was steht in der nächsten Woche an? Gibt es feste Feiertage, geplante Prüfungen oder Fristen, die die KI in ihre Narration einweben muss?

- **Relevantes Gedächtnis (RAG-Ansatz):** Hier findet eine semantische Filterung statt. Statt das gesamte chronicle zu senden, filtert das Modul basierend auf Relevanz.

  - *Tags:* Wenn der GM als Fokus „Nordturm" angibt, werden nur Logbuch-Einträge mit dem Tag #Nordturm exportiert.

  - *Recency:* Priorisierung von Ereignissen der letzten 3 Spielwochen gegenüber alten Geschehnissen.

  - *NPC-Wissen:* Exportiert spezifische npcMemory-Einträge nur für NPCs, die in der aktuellen Szene wahrscheinlich auftreten werden.

- **Regel-Referenzen (Anti-Halluzination):** Fügt Ausschnitte aus dem dsa5_index.json (Phase 3) hinzu. Statt alle 500 Zauber zu senden, sendet es vielleicht nur die Zauber, die die SCs beherrschen, plus eine Liste relevanter Magier-Akademie-Zauber. Dies zwingt die KI, existierende Foundry-UUIDs zu nutzen.

- **Output:** Erstellt io/outbox/context_current.json. Diese Datei ist ein maschinenlesbares, token-optimiertes Abbild der aktuellen Spielrealität, bereit für den Prompt.

7.2 Import Processor (ki/import.js)

Der Parser und „Sicherheitsbeauftragte" für die KI-Antworten. Da KI-Output unvorhersehbar sein kann (Syntaxfehler, Logikfehler), darf er niemals ungeprüft in die Datenbank geschrieben werden.

- Funktion:

- **Schema-Validierung (Syntax):** Prüft das eingehende JSON (world_build.json) rigoros gegen das JANUS-Schema (via core/validator.js).

  - *Prüfung:* Sind alle Pflichtfelder da? Sind Zeitangaben im korrekten Format? Sind Referenzen auf UUIDs gültig?

  - *Fallback:* Bei leichten Fehlern (z.B. fehlende optionale Felder) wird repariert; bei schweren Fehlern wird der Import abgelehnt und ein Fehlerbericht erstellt.

- **Diffing & Review (Semantik):** Bevor Änderungen angewendet werden, generiert das Modul einen menschenlesbaren „Diff-Report" für den GM. Dies ist ein Schutzmechanismus gegen unerwünschte KI-Entscheidungen.

  - *Beispiel:* „Achtung: Die KI schlägt vor, den NPC ‚Alrik' zu töten und Haus Feuer 50 Punkte abzuziehen. Bestätigen?"

- Execution (Anwendung):

  - **Content:** Erstellt Journal Entries (Szenen, Handouts) und Actors (neue NPCs) über den Director. Dabei werden automatisch Verknüpfungen (Links) zu existierenden Entities gesetzt.

  - **Simulation:** Updated den State direkt über die spezialisierten APIs (z.B. social.modifyRelation, economy.addDebt). Dies stellt sicher, dass Hooks gefeuert werden und die UI sich aktualisiert.

  - **Zeit:** Setzt den Kalender auf den Zeitpunkt des letzten generierten Slots, um die Kontinuität zu wahren.

7.3 Jobs & Watcher (ki/jobs.js)

Die Verwaltungsschicht für asynchrone Aufgaben. Die Generierung von komplexen Wochenplänen durch ein LLM kann mehrere Minuten dauern. Während dieser Zeit darf Foundry nicht blockieren („einfrieren").

- **Job-Definition:** Ein Job kapselt einen Auftrag. Er enthält eine eindeutige ID, den Prompt-Typ (z.B. „Wochenplanung", „NPC-Generierung") und Metadaten (Zeitstempel).
- Dateisystem-Kommunikation:

  - **Request:** JANUS7 schreibt job_\[UUID\]_request.json in die Outbox.

  - **Response:** Das externe KI-Tool schreibt job_\[UUID\]_response.json in die Inbox.

- **Watcher-Mechanismus:** Ein Hintergrundprozess (z.B. via setInterval oder Socket-Event), der den Ordner io/inbox/ überwacht.

  - Sobald eine passende Response-Datei auftaucht, wird der Job-Status von „Wartend" auf „Fertig" gesetzt.

  - **User Feedback:** Benachrichtigt den GM über ein UI-Toast („KI-Daten für Woche 4 bereit zum Import") oder ein blinkendes Icon im Control Panel.

- **Status-Tracking & Persistenz:** Verwaltet den Status von Aufträgen (Pending, Processing, Ready, Failed) und speichert diesen im State. Das ist kritisch, damit ein Job nicht „verloren geht", wenn der GM während der Wartezeit Foundry neu lädt (F5).

🔗 Abhängigkeiten & Schnittstellen

Abhängigkeiten (Benötigt\...)

Das KI-Modul ist ein „Consumer" fast aller anderen Systemteile.

- **Phase 1 (IO):** Für den physischen Dateizugriff (Lesen/Schreiben/Watchen). Die Abstraktion hier ist wichtig, damit das KI-Modul nicht wissen muss, *wie* Dateien gespeichert werden (lokal vs. S3).
- **Phase 1 (State):** Als primäre Datenquelle für den Export und Ziel für Updates. Ohne State ist die KI blind.
- **Phase 4 (Simulation):** Um logische Änderungen anzuwenden.

  - *Wichtig:* Der Import Processor schreibt nicht direkt in state.social.relations, sondern ruft Social.modifyRelation() auf. Nur so werden Nebeneffekte (z.B. Gossip-Trigger) korrekt ausgelöst.

- **Phase 3 (Indexer):** Nutzt den Index als „Wahrheits-Referenz". Wenn die KI sagt „Nutze Zauber X", prüft der Import Processor gegen den Index, ob Zauber X existiert, bevor er den Import zulässt.

Schnittstellen (Bietet an\...)

Das KI-Modul bietet High-Level-Funktionen für die UI (Control Panel).

- Export API:
  - KI.createContextExport(options): Erstellt einen Snapshot.
    - options.includeFullHistory: (Bool) Soll das gesamte Logbuch exportiert werden?
    - options.focusTags: (Array) Welche Themen sind gerade relevant?
- Import API:
  - KI.listPendingJobs(): Gibt eine Liste aller abgeschlossenen, aber noch nicht importierten Jobs zurück.
  - KI.previewWorldBuild(json): Liefert eine Zusammenfassung der Änderungen (Dry-Run / Diff-View) für das Review-Fenster.
  - KI.applyWorldBuild(json): Führt die Änderungen final durch und schreibt in die Datenbank.
- Hooks:
  - janus7NewFileDetected: Feuert, wenn der Watcher etwas findet. Das Control Panel kann darauf reagieren (z.B. Notification Sound, Button blinkt).
  - janus7JobCompleted: Wenn ein Import erfolgreich durchgelaufen ist, um z.B. das UI zu refreshen oder den nächsten Schritt im Tutorial anzuzeigen.

ChatGTP:

Phase 7, der „Brain-Interface"-Teil von JANUS. Jetzt geht's darum, wie dein Akademiesystem vernünftig mit Gemini/ChatGPT/Claude & Co. spricht, ohne dass du es beim nächsten Versionssprung verfluchst. 😄

Ich baue dir Phase 7 wieder so, dass du sie direkt als Kapitel ins Word-Dokument kopieren kannst.

Phase 7 -- KI-Integration (Roundtrip)

JANUS ⇄ LLM ohne Spaghetti

7.1 Ziel & Scope

Ziel von Phase 7

Phase 7 sorgt dafür, dass JANUS:

103. Exportieren kann:
    - Kampagnenzustand (State, Zeit, Punkte, Social etc.).

- Akademiedaten (Stundenpläne, Lektionen, NPCs, Orte).
- Kontexttexte (Lore, Lehrplan, Referenzdateien).

104. KI-freundliche Pakete baut:

- z\. B. JANUS_EXPORT_V2.json oder ein ZIP im bekannten JANUS_KI_KIT-Layout.
- Klar strukturierte Formate zum Füttern von Gemini, ChatGPT, Claude etc. (analog zu Modulen wie „Compendium Exporter", die Compendia als JSON/TXT/ZIP für externe Tools exportieren).

105. Importieren kann:

- Antworten/Änderungsvorschläge einer KI (z. B. JSON mit angepasstem Stundenplan, neuen Events, Texten).
- Diese durch den Validator (Phase 1) jagt und dann kontrolliert in den State/World einspielt.

106. LLM-agnostisch ist:

- Kein hart eingebautes „nur OpenAI" oder „nur Gemini", sondern generische Export-/Importformate.
- Optionale Brücken zu existierenden AI-Integrationsmodulen wie **Integrate AI**, **Phil's AI Assistant**, **Cibola 8**, **RPGX AI Assistant** nur als *Addons*, nicht als Pflicht.

Nicht-Ziele

- Kein eigenes API-Key-Management im Stil von „Mini-OpenAI-Dashboard".
- Kein „LLM im Hintergrund, das dauernd deine Welt umbaut". Alles bleibt **explizit, manuell ausgelöst, transparent**.
- Keine Konkurrenz zu vorhandenen AI-Hilfsmodulen -- JANUS stellt Daten sauber bereit, du entscheidest, wie du die KI nutzt.

7.2 Modulstruktur & Namespaces

Vorschlag:

modules/janus7/

ki/

exporter.js // State + Wissen → JSON/ZIP

importer.js // KI-JSON → Validierung → Änderungen anwenden

prompts.js // Prompt-Templates für Gemini/ChatGPT/Claude usw.

diff.js // (optional) lesbare Änderungs-Diffs erzeugen

Globale Registrierung:

game.janus7.ki = {

exporter: JanusKIExporter,

importer: JanusKIImporter,

prompts: JanusKIPrompts,

diff: JanusKIDiff

};

7.3 Formate & Datenmodell

7.3.1 JANUSIMPORTV1 (Eingang für Foundry)

Das gibt es bei dir schon:

- Schuljahre, Trimester, Wochen, Tage.
- journal, lessons, events, campaign_state_updates.
- optional art-Block (Scene-/NPC-Prompts, Bild-Metadaten).

Phase 7-Regel:\

JANUS_IMPORT_V1 bleibt *das* **kanonische Importformat**. KI-Ergebnisse, die die Akademie erweitern, sollen idealerweise in dieses Format übertragen werden.

7.3.2 JANUS_EXPORT_V2 (Roundtrip-Format)

Neues, reines **Exportformat**, optimiert für:

- Lesbarkeit durch LLMs,
- eindeutige IDs,
- spätere Rückprojektion in JANUS_IMPORT_V1 oder direkt in den State.

Minimaler Kern:

type JanusExportV2 = {

version: „JANUS_EXPORT_V2";

meta: {

exportedAt: string; // ISO

world: string;

janusVersion: string;

```
system: string; // "dsa5"
};
campaign_state: JanusStateSnapshot; // aus Phase 1
academy: {
calendar: \...; // strukturierter Stundenplan
lessons: \...; // Curricula, Inhalte
exams: \...;
events: \...;
scoring: \...; // Hauspunkte/Statistiken
social: \...; // Beziehungs-/Reputationsnetz
};
references: {
npcs: \...;
locations: \...;
items?: \...;
};
knowledge_links?: {
// Verweise auf deine Markdown-Referenzen (Geographie, Geschichte, Zauberlehrplan etc.)
};
art?: {
scenes?: \...;
npcs?: \...;
};
};
```

Die KI soll *nicht* direkt Foundry-Dokumente ändern, sondern dieses abstrahierte Modell.

7.3.3 KI-Antwortformat (JANUSKIRESPONSE_V1)

Damit du Antworten konsistent verarbeiten kannst, definierst du ein KI-Rückgabeformat, z. B.:

type JanusKIResponseV1 = {

version: "JANUS_KI_RESPONSE_V1";

sourceExportMeta: JanusExportV2\["meta"\];

changes: {

calendarUpdates?: \...; // neue/angepasste Slots

lessonUpdates?: \...;

eventUpdates?: \...;

scoringAdjustments?: \...;

socialAdjustments?: \...;

journalEntries?: \...; // neue/aktualisierte Texte

};

notes?: string; // Freitext der KI (Begründung, Kommentare)

};

Das ist das, was importer.js erwartet.

7.4 Exporter (ki/exporter.js)

7.4.1 Aufgaben

- Aktuellen **JANUS-State** (Phase 1) auslesen.
- Akademiedaten aus Phasen 2--4 extrahieren.
- Optional relevante **Foundry-Dokumente** (Journal, Scenes, Compendia) als Kontexte beilegen.
- Das Ganze in ein JANUS_EXPORT_V2-Objekt gießen.
- Dieses Objekt:

  - entweder direkt als **JSON-Datei** herunterladen (saveDataToFile),

  - oder in ein ZIP packen (ähnlicher Usecase wie das Modul „Compendium Exporter": mehrere Formate in ZIP für externe Verarbeitung/AI).

7.4.2 Öffentliche API

- createExportSnapshot(options?): JanusExportV2

- includeLore, includeReferences, includeArtPlaceholders etc.

- downloadExport(options?)

  - nutzt foundry.utils.saveDataToFile(JSON.stringify(export), "application/json", filename).

- downloadKIPackage(options?)

  - baut optional ZIP-Struktur im bekannten JANUS_KI_KIT-Layout:

    - 00_README/, 01_MODULE_CURRENT/, 02_PROMPTS_CURRENT/, 07_KNOWLEDGE_AKADEMIE_DGDZ/ etc.

  - analog zu deinem heute schon bestehenden „Kit".

7.4.3 Datenquellen

- JanusStateCore.snapshot() -- konsistenter State-Dump (Phase 1).
- academy.calendar.exportModel() -- strukturierter Stundenplan (Phase 4).
- academy.lessons/exams/events.exportModel() -- Lehrplan & Szenen (Phase 4).
- academy.scoring/social.exportModel() -- Gamification & Beziehungen (Phase 4).
- npcDatabase, locationDatabase -- aus Phase 2-StaticData.
- Optional Foundry-Dokumente:

  - game.journal, game.scenes, game.actors -- via Dokument-API.

7.5 Importer & Validator (ki/importer.js)

####### 7.5.1 Aufgaben

- Datei (JSON) vom Nutzer annehmen (via FilePicker/DialogV2).
- Prüfen, ob es:

  - ein JANUS_KI_RESPONSE_V1 ist, oder

  - ein vollwertiges JANUS_IMPORT_V1, das direkt in Foundry eingespielt werden soll.

- Über den **Validator** (Phase 1) laufen:

  - Schema-Check,

  - inhaltliche Checks (gibt es die referenzierten Lessons/NPCs/Orte noch?),

  - optional DSA5-Checks (Talente/Zauber gültig? über Phase 3).

- Änderungsvorschläge in *lesbare Patches* übersetzen:

  - „Slot X → Y", „neue Lesson", „Event verschoben" etc.

- Dem SL im UI (ApplicationV2 oder DialogV2) einen **Änderungsplan** zeigen:
  - Häkchen: „Änderung übernehmen / verwerfen".
- Bei Bestätigung:
  - Engine-APIs der Phasen 1--4 aufrufen, um Änderungen sauber einzuspielen.

7.5.2 Öffentliche API

- previewImport(file: File): Promise\<JanusKIPatchPlan\>
  - liest JSON, validiert, erzeugt Patchplan.
- applyPatchPlan(patchPlan, options?): Promise\<void\>
  - ruft nur Engine-Methoden auf:
    - state.applyUpdates(\...),
    - academy.calendar.applyChanges(\...),
    - academy.lessons.upsertLesson(\...),
    - academy.events.upsertEvent(\...),
    - usw.
- importJanusImportV1(file)
  - Sonderfall, wenn eine KI direkt ein JANUS_IMPORT_V1 generiert
  - → durch Validator → dann in deine bestehende Importlogik (Phase 2/4).

####### 7.5.3 UI & Dialoge

Für den SL:

- **DialogV2**-basierter Importdialog:
  - Datei auswählen.
  - Vorschau: welche Bereiche betroffen sind (Kalender, Lessons, Events, Scoring, Social, Journals).
  - Detaillierte Liste pro Bereich (z. B. Liste neuer Events mit Kurzbeschreibung).

######## 7.6 Prompt-Generator (ki/prompts.js)

Das ist der Teil, der in Richtung „Phil's AI Assistant" geht -- nur eben JANUS-spezifisch und ohne API-Key-Handling.

######### 7.6.1 Aufgabe

- Aus JANUS_EXPORT_V2 passende, **vorgekaute Prompts** generieren, z. B.:
  - „Mach mir aus diesem Wochenplan neue Schul-Ereignisse"
  - „Verbessere die Lektionen dieses Trimesters"
  - „Erzeuge 10 optionale Nebenplots passend zu den existierenden NPCs"
- Die Prompts sollen so gestaltet sein, dass:
  - sie das Schema von JANUS_KI_RESPONSE_V1 erklären,
  - die KI gezielt JSON mit klaren changes-Blöcken zurückgibt,
  - kein Ratespiel beim Import nötig ist.

7.6.2 Öffentliche API

- buildPrompt(export: JanusExportV2, mode: "lessons" \| "events" \| "social" \| \...): string
- buildPromptPack(export: JanusExportV2): { title: string; content: string }\[\]
  - ein Paket mit mehreren Anwendungsfällen (Lessons, Events, Exams, Social).

Diese Strings gehen **nicht** in Foundry-Chat, sondern sind für den Menschen zum Copy&Paste in Gemini/ChatGPT/Claude usw. gedacht -- genau wie das neue „Phil's AI Assistant"-Modul es macht: Daten lesen, Prompt vorschlagen, du nutzt deine bevorzugte Web-KI.

######## 7.7 Optionale Integration mit AI-Modulen

Wenn du später doch direkte API-Integration möchtest, bietet sich an:

- **Integrate AI** als generische KI-Schicht, auf die JANUS sich stützt (statt selbst HTTP zu machen).
- Oder Nutzung von Modulen wie **Cibola 8**, **RPGX AI Assistant**, **AI Tools** nur als „Frontend", das deine Exportdaten benutzt.

Phase-7-Designprinzip:

- JANUS kennt deren **Existenz**, ist aber nicht davon abhängig.
- In ki/integrations.js (optional) könntest du später Hooks bauen wie:
  - „Export an IntegrateAI schicken, Antwort als JANUS_KI_RESPONSE_V1 erwarten".

######## 7.8 Abhängigkeiten zu anderen Phasen

Hängt ab von:

- Phase 1 -- Core & State
  - State-Snapshot/Restore, Validator, Logger.
- Phase 2 -- Static Academy Data
  - NPC/Ort/Lehrplan-Referenzen.
- Phase 3 -- DSA5 System Bridge
  - Validierung von Talenten/Zaubern, falls KI Proben oder Prüfungen vorschlägt.
- Phase 4 -- Academy Simulation
  - Kalender-/Lesson-/Event-APIs, auf die Patches angewendet werden.
- Phase 5 -- Atmosphere
  - optional: KI kann neue Moods oder Scene-Bindings vorschlagen → Importer muss das in AtmosphereProfiles/Bindings überführen.
- Phase 6 -- UI
  - Import-/Export-Buttons im Control Panel,
  - ggf. eigenes „KI-Roundtrip"-Panel.

Wird genutzt von:

- nichts „unterhalb" -- Phase 7 sitzt ganz oben als Integrationsschicht.

######## 7.9 Aufwandsschätzung Phase 7

Grob, 1 Person, mit deiner Vorerfahrung:

107. Exportmodell & Snapshot-Logik (createExportSnapshot)\

→ 2--3 PT

108. Download/ZIP-Handling (downloadExport, downloadKIPackage)\

→ 1,5--2,5 PT

109. KI-Antwortformat & Importer (Parsing, Validator-Anbindung, Patchplan)\

→ 3--5 PT (je nach Granularität der Patches)

110. UI für Import (DialogV2 oder eigene App) inkl. Auswahl & Vorschau\

→ 2--3 PT

111. Prompt-Generator (einige solide Templates, nicht 100)\

→ 2--3 PT

112. Optional: Integrations-Hooks zu Drittmodulen\

→ 1--3 PT (je nachdem, wie tief du gehst)

Summe: ca. **11--19 Personentage**, stark abhängig vom Detailgrad des Importers und der Integrationen.

######## 7.10 Risiken & Gegenmaßnahmen

113. Risiko: Schema-Drift zwischen Export, KI-Antwort und Import

- *Problem:* Du änderst Exportstruktur, vergisst Prompts/Importer anzupassen → KI-Antwort passt nicht mehr.
- Gegenmaßnahmen:

  - Versionierung ernst nehmen (version: „JANUS_EXPORT_V2", „JANUS_KI_RESPONSE_V1").

  - Validator prüft version strikt.

  - Prompts enthalten immer das Schema-Snippet als Referenz.

114. Risiko: KI macht Unsinn / zerstört Lore

- *Problem:* KI generiert Events/Änderungen, die nicht zur Welt passen.
- Gegenmaßnahmen:

  - Importer arbeitet ausschließlich mit „Patchplan mit Häkchen", nie „Blind-Automerge".

  - Validator markiert problematische Änderungen (fehlende Referenzen, ungültige Talente etc.).

115. Risiko: Performance bei großen Welten

- *Problem:* Export großer Worlds (viele Journale, NPCs) → große JSON/ZIPs.
- Gegenmaßnahmen:

  - Export-Optionen für „lite", „nur Akademie", „nur aktuelle Woche".

  - Ähnlich wie Compendium Exporter: ggf. Batch-/Teilexporte.

116. Risiko: Abhängigkeit von einzelnen KI-Anbietern

- *Problem:* Du optimierst Prompts auf einen Anbieter → Wechsel schmerzt.
- Gegenmaßnahmen:

- Prompt-Generator generiert **neutrale**, standardkonforme JSON-Aufgaben.
- KI-spezifische Feinheiten (z. B. „Du bist Gemini...") nur in dünnen, leicht austauschbaren Wrappern.

######## 7.11 Umsetzungsschritte (konkreter Plan)

Schritt 1 -- Formate definieren

- JANUS_EXPORT_V2 & JANUS_KI_RESPONSE_V1 in einem schema.md dokumentieren.
- Validator anpassen, damit er beides erkennt.

Schritt 2 -- Snapshot/Export

- exporter.createExportSnapshot() implementieren.
- Erste „Lite"-Downloads (nur State + aktuelle Woche) mit saveDataToFile.

Schritt 3 -- KI-Import-Parsing & Patchplan

- importer.previewImport():
  - JSON einlesen, Schema prüfen, Patchplan generieren.
- JanusKIDiff (optional): lesbare Beschreibung der Änderungen.

Schritt 4 -- Import-UI (DialogV2/App)

- File-Auswahl, Vorschau, Häkchen, „Apply"-Button (ApplicationV2/ DialogV2).

Schritt 5 -- Prompt-Generator

- 2--3 solide Grundprompts:
  - „Verbessere Stundenplan & Lektionen"
  - „Erzeuge neue Akademie-Events"
  - „Passe Social-/Scoring-Daten vorsichtig an"

Schritt 6 -- Tiefer Export (Kit-Struktur)

- ZIP-Export im JANUS_KI_KIT-Layout, inkl. deiner Markdown-Referenzen (Geographie, Kosmologie, Lehrpläne, NPC-Datenbanken etc.).

Schritt 7 -- Optionale AI-Modul-Hooks

- Falls gewünscht: kleine Integrationsschicht, die Integrate AI oder ähnliche Module nutzt, um automatisiert Requests an lokale LLMs zu stellen.

Damit ist Phase 7 jetzt so präzise, dass du sie 1:1 als Kapitel übernehmen kannst -- inklusive Schnittstellen, State-Bereichen, Risiken und Plan.

Als Nächstes wäre dann Phase 8 (Backlog & Extensions) der Parkplatz für alles, was wir bewusst *nicht* in 1--7 gepackt haben (Tablet-UI, vollständiger Editor, Deep-AI-Automatismen, ...), damit die Kernarchitektur sauber bleibt.

Phase 8 – Backlog & Extensions (Detailliert)

Backlog & Extensions -- Zukunftsmusik -- Erweiterte Dokumentation

Diese Features sind konzeptionell vorbereitet und architektonisch eingeplant, aber nicht Teil des MVP (Minimum Viable Product). Sie stellen die langfristige Evolution von JANUS7 dar und erweitern das System um spezifische Fachbereiche und Komfortfunktionen.

Backlog A: Akademie-Vertiefungen (Content & Simulation)

Hier geht es um die Vertiefung der fachspezifischen Simulationen, um das akademische Leben noch detaillierter abzubilden.

- **A1 Bibliothek:** Eine dedizierte UI für die Recherche.

  - **Mechanik:** Spieler geben Suchbegriffe ein (z.B. \"Chimären\", \"Borbarad\"). Das System prüft basierend auf Tags in Journalen und Item-Datenbanken, ob Bücher dazu existieren und ob der Spieler Zugang dazu hat (z.B. \"Verbotene Abteilung\").

  - **Erfolg:** Freischaltung von Handouts (Buchauszüge), Bonus-Würfeln für Wissensproben oder Hinweisen für die Thesis.

- **A2 Alchimie:** Ein Crafting-Interface für das Labor.

  - **Mechanik:** Verknüpfung mit dem economy.stash (Gruppenlager). Spieler wählen ein Rezept aus der Datenbank (alchemy_recipes.json). Das System prüft, ob die Zutaten vorhanden sind, verlangt die entsprechenden Proben (via Bridge) und zieht verbrauchte Materialien automatisch ab.

  - **Ergebnis:** Generierung des Tranks als Item im Inventar oder Explosion bei Patzer.

- **A3 Dämonologie:** Spezialregeln für Beschwörungen (Theorie und Praxis).

  - **Mechanik:** Ein Risiko-Manager für Beschwörungsproben. Trackt \"Verdammnis-Punkte\" (Codex-Verstöße) und berechnet Wahrscheinlichkeiten für Kontrollverlust basierend auf Mondphasen (Kalender) und Ort (Labor vs. freies Feld).

Backlog B: Exkursionen (Die Welt draußen)

Erweiterung des Scopes über die Akademie-Mauern hinaus für Forschungsreisen oder Abenteuer.

- **Reise-Logik:** Ein System für Reisen auf der Aventurien-Karte.

  - **Mechanik:** Berechnet Reisezeiten basierend auf Transportmittel (Fuß, Pferd, Kutsche) und Route. Berücksichtigt Wetter und Terrain.

  - **Encounters:** Generiert Zufallsbegegnungen (Random Encounters) passend zur Region (dsa5-bestiary Integration), die nicht nur Kämpfe, sondern auch soziale oder Natur-Ereignisse sein können.

- **Externe Locations:** Verwaltung von temporären Orten (Zeltlager, Ruinen, Dörfer), die nicht Teil des festen Campus-Plans sind und dynamisch generiert oder geladen werden.

Backlog C: Begegnungen (Encounter Manager)

Tools für den Spielleiter, um schnell spannende Begegnungen zu erstellen.

- **Monster der Woche:** Ein Generator, der basierend auf dem aktuellen Level der Gruppe, der Region (z.B. \"Akademie-Keller\", \"Verbotener Wald\") und dem Thema der Woche passende Gegner aus dem Bestiarium auswählt.

  - **Funktion:** Bereitet den Encounter vor, indem er Tokens auf der Szene platziert, Stats lädt und Loot generiert.

Backlog D: Wizardry (Interaktive Tutorials)

Didaktische Hilfsmittel, um Neulingen die komplexen Regeln von DSA5 spielerisch beizubringen.

- **D1 Zauber-Assistent:** Ein Overlay, das sich öffnet, wenn ein Spieler einen Zauber wirken will.

  - **Funktion:** Führt Schritt für Schritt durch den Prozess:

    1. \"Ziel wählen\" (Reichweite prüfen)
    2. \"Modifikationen ansagen\" (Kraftfokus, Erzwingen, Kosten sparen)
    3. \"Probe würfeln\" (3W20 auf Eigenschaften)
    4. \"Qualität bestimmen\" (FP berechnen)
    5. \"Auswirkung anwenden\" (Schaden/Effekt/Dauer)

  - **Didaktik:** Erklärt dabei die DSA5-Regeln (z.B. \"Warum ist die Probe erschwert? Weil das Ziel in Bewegung ist.\").

- **D2 Auto-Kampf-Setup:** Ein Assistent für den GM, um Kämpfe schnell zu starten.

  - **Funktion:** \"Erstelle Kampf mit 3 Goblins\". Platziert Tokens, würfelt Initiative für alle, öffnet den Combat Tracker und spielt passende Kampfmusik ab.

🔗 Abhängigkeiten der Extensions

- **Alle Backlog-Items** hängen stark von **Phase 3 (Bridge)** (für den Zugriff auf Regeln, Würfelmechaniken und Kompendien) und **Phase 4 (Simulation)** (für Konsequenzen wie Ressourcenverbrauch, Zeitaufwand oder Rufänderungen) ab.
- Sie erweitern primär die **Phase 6 (UI)** um neue, spezialisierte Fenster und Overlays, nutzen aber die bestehende Datenstruktur und Logik des Cores, um konsistent mit dem Rest des Systems zu bleiben.

ChatGTP:
Phase 8 ist im Prinzip: „Alles, was wir geil finden, aber **bewusst nicht** in 7.0 reingestopft haben."\

Also der offizielle Wunschzettel, damit es *kein* Wildwuchs mehr wird. 😄

Ich schreibe es dir wieder so, dass du es direkt als Kapitel „Phase 8" ins Roadmap-Dokument kleben kannst.

Phase 8 -- Backlog & Extensions

Der offizielle Wunschzettel von JANUS7

8.1 Ziel & Scope

Ziel von Phase 8

Phase 8 sammelt **bewusst verschobene Features** und Ideen, die:

- technisch sinnvoll sind,
- aber für JANUS7 „Core" (Phasen 0--7) nicht notwendig sind,
- oft mehr **UX-Komfort**, „Spielzeug" oder Cross-Integration darstellen als Kernfunktionen.

Damit:

- bleibt JANUS7 **überschaubar & wartbar**,
- gehen aber keine Ideen verloren -- jede Idee hat einen Platz, eine grobe Beschreibung, Abhängigkeiten und eine Priorität.

Scope von Phase 8

- Definition von **Backlog-Epics** (große Themenblöcke).
- Pro Epic:
  - Kurzbeschreibung,
  - Nutzen,
  - Dependencies auf Phasen 1--7,
  - grobe Komplexität & Risiko.
- Kein „Pflichtenheft", sondern ein **geordneter Ideen-Parkplatz**.

8.2 Struktur des Backlogs

Wir bündeln die Ideen in fünf Kategorien:

117. **Core & Architektur-Extensions**

118. **Akademie & DSA5-Gameplay-Extensions**
119. **UX, Hybrid Play & Mobilität**
120. **Deep-KI & Automatisierung**
121. **Tooling & Dev-Workflow**

Innerhalb jeder Kategorie definieren wir einzelne **Epics**. Die tatsächliche Umsetzung erfolgt nur, wenn:

- eine Phase 1--7 stabil ist,
- der Nutzen klar ist,
- und Zeit/Bandbreite da ist.

8.3 Backlog-Epics im Detail

A. Core & Architektur-Extensions

Epic A1 -- Multi-Akademie / Multi-Setting-Unterstützung

Idee:\

JANUS kann zukünftig **mehrere Akademien / Schul-Szenarien** parallel verwalten:

- z\. B. Punin, Gareth, Drakonia oder eine komplett andere Magierschule.
- pro Akademie: eigener Kalender, eigener Lehrplan, eigene NPC-Basis.

Nutzen:

- Wiederverwendbarkeit des JANUS7-Kerns für andere DSA5-Settings und ggf. andere Systeme.

Dependencies:

- Phase 1 (State):
  - JanusStateCore muss eine Ebene „academyId" kennen.
- Phase 2 (Static Data):
  - Static-Daten müssen pro Akademie getrennt sein.
- Phase 4 (Simulation):
  - Kalender & Logik müssen Akademie-spezifisch funktionieren.

Risiko/Komplexität:\

Mittel--hoch (Architektur-Auswirkung).\

Empfehlung: **erst nach** stabiler 7.0, dann als 7.1/7.2 Feature.

Epic A2 -- Story-/Thread-Graph & Visualisierung

Idee:\

Visualisierung der laufenden Kampagnen-Threads in einem **Graph**, der:

- Beziehungen zwischen Events, Lektionen, NPCs, Orten zeigt,
- anzeigt, welche Stränge offen, abgeschlossen oder eskaliert sind.

Nutzen:

- Bessere Übersicht für lange Kampagnen,
- direkte Unterstützung bei Session-Prep.

Dependencies:

- Phase 1 (State):
  - Thread-/Story-State muss sauber modelliert sein.
- Phase 4 (Academy Simulation):
  - Events & Lektionen müssen Bezug zu Threads haben.

Risiko/Komplexität:\

Mittel; eher UX-/Visualisierungsaufwand.

Epic A3 -- Versionierte Lore & Layering (Canon vs. Homebrew)

Idee:\

Trennung von:

- **Kanon-Lore** (offizielle DSA5-Weltinfos, Module, deine Referenz-Docs),
- **Homebrew-Erweiterungen** (eigene NSCs, Zusatzorte, alternative Geschichtsstränge).

Mit Möglichkeit, Layer ein- und auszublenden.

Nutzen:

- Mehr Klarheit, was „offiziell" und was „Hausregel/Headcanon" ist.
- Besserer Umgang mit deinen vielen Lore-Dokumenten (Geographie, Kosmologie, Geschichte, Zauberlehrplan usw.).

Dependencies:

- Phase 2 (Static Academy Data):

  - NPC-/Ort-/Lore-Datenbank muss Meta-Infos (Quelle/Layer) unterstützen.
- Phase 7 (KI):
  - Exporter kann gezielt „Canon only" oder „Canon + Homebrew"-Datasets anbieten.

Risiko/Komplexität:\

Mittel; vor allem Datenmodell & UI-Konzepte.

B. Akademie & DSA5-Gameplay-Extensions

Epic B1 -- In-Game Curriculum & Lesson Editor

Idee:\

Eine komfortable UI innerhalb von Foundry, mit der du:

- Lektionen direkt im Spiel ändern/erstellen kannst,
- Stundenplan-Slots umverteilst,
- ohne externe Files bearbeiten zu müssen.

Nutzen:

- Schnelle Anpassung „on the fly" nach Spieleraktionen,
- Beibehaltung des Prinzips: Lehrplan bleibt modifizierbar.

Dependencies:

- Phase 2 (Static Data):
  - sauberes Datenmodell für Lessons/Exams.
- Phase 4 (Simulation):
  - Kalender-Logik muss dynamische Updates verkraften.
- Phase 6 (UI):
  - braucht dedizierte Editor-Oberflächen (Formulare, Validierung).

Risiko/Komplexität:\

Mittel--hoch; Gefahr, komplexen Editor zu bauen statt schlankem Werkzeug.

Epic B2 -- Noten-, Zeugnis- & Berichtssystem

Idee:\

Mechanik & UI für:

- Noten / Qualitätsstufen pro Klasse & Prüfung,
- Zwischenzeugnisse / Abschlusszeugnis,
- optional Export als Journal-PDF/Markdown für die Spieler.

Nutzen:

- „Magierschule" fühlt sich noch mehr nach „echter" Akademie an,
- langfristige Konsequenz: Leistungen haben Gewicht (Social, Scoring, Karriere).

Dependencies:

- Phase 3 (DSA5 Bridge):
  - Zugriff auf Proben-/Ergebnislogik.
- Phase 4 (Exams & Scoring):
  - Prüfungs- & Scoring-System muss Erweiterungen für „Noten" kennen.
- Phase 6 (UI):
  - zusätzliche Views für Zeugnisse/Verläufe.

Risiko/Komplexität:\

Mittel; hauptsächlich Fleißarbeit & UI-Design.

Epic B3 -- Generisches „School Framework" (System-agnostisch)

Idee:\

JANUS als **generischer Schul-Framework**, nicht nur für DSA5:

- definierbare „Häuser", Punkte, Stundenpläne, Unterrichtseinheiten,
- System-spezifische Layer für DSA5, Cthulhu, andere Systeme.

Nutzen:

- Reuse von JANUS bei anderen Kampagnen,
- langfristig: „Janus Academy Engine" statt „nur Punin".

Dependencies:

- Phase 3 (System Bridge):
  - DSA5-Bridge muss ein Modul sein, nicht Teil des Cores.
- Phase 1/2:
  - Core-Modelle müssen systemneutral sein, System-Spezifika in Adapter schieben.

Risiko/Komplexität:\

Hoch; Architektur-Impact, eher mittelfristige Vision (7.x++).

C. UX, Hybrid Play & Mobilität

Epic C1 -- Second-Screen / Companion View (Read-Only)

Idee:\

Ein **reduziertes, read-only UI**, das auf:

- separatem Browserfenster oder
- einem zweiten Client (z. B. Tablet)

läuft und:

- nur Schulkalender, aktuelle Woche, Hauspunkte, vielleicht Mood-Info zeigt.

Wichtig:\

Kein eigener Remote-API-Server -- wir nutzen existierende Mechanismen:

- Foundry-Client auf Tablet + Module wie **TouchVTT**, **Mobile Improvements**, **Mobile Token Control** oder **Vadanx\'s Mobile Controller** für mobile Bedienung.

Nutzen:

- Spielende sehen „Dashboard" (Hauspunkte, Stundenplan) auf dem eigenen Gerät,
- ohne dass du eine komplett eigene WebApp pflegen musst.

Dependencies:

- Phase 6 (UI):
  - kompakte, panelartige UI, die auch auf kleineren Screens gut läuft.
- Phase 5 (Atmosphere):
  - optional Anzeige des aktuellen Moods.

Risiko/Komplexität:\

Mittel; techn. machbar, aber UX-Testaufwand auf verschiedenen Devices.

Epic C2 -- Session-Prep Wizard & Agenda

Idee:\

Assistent innerhalb des Control Panels:

- wählt aus Kalender, Threads & offenen Hooks die relevanten Elemente für die **nächste Session**,
- erstellt eine „Agenda": Szenen, NSCs, Prüfungen, mögliche Events.

Nutzen:

- Vorbereitung verschlankt,
- direkte Verknüpfung von Kalender, Story-Threads und Szenen.

Dependencies:

- Phase 4 (Story/Events/Threads),
- Phase 6 (UI: Control Panel).

Risiko/Komplexität:\

Mittel; Großteil ist Datenaggregation & sinnvolle Darstellung.

Epic C3 -- Analytics & Telemetrie-Light

Idee:\

Ein kleines Analytics-Modul:

- „Wie oft waren wir in welcher Szene?"
- „Welche NSCs wurden am häufigsten gespielt?"
- „Welche Lektionen/Prüfungen wurden bereits durchgeführt?"

Keine Tracking-Hölle, nur **spielrelevante Statistiken**.

Dependencies:

- Phase 1 (State-Historie oder Logging),
- Phase 4 (Slots/Events mit Zeitbezug),
- Phase 6 (UI-Dashboards).

Risiko/Komplexität:\

Mittel; Gefahr des Over-Engineerings (Backlog für später).

D. Deep-KI & Automatisierung

Epic D1 -- Direkte Nutzung von „Integrate AI" / „Phil's AI Assistant" / „Cibola 8"

Idee:\

JANUS nutzt optionale KI-Module als **Transport-Schicht**:

- **Integrate AI**: generischer Layer, um externe/lokale Modelle anzusprechen.

- **Phil's AI Assistant**: liest Foundry-Daten, generiert Prompts und nutzt Web-UIs von Gemini/ChatGPT/Claude ohne eigene API-Keys.
- **Cibola 8**: großer AI-Hub für Text/Bilder/Musik.

JANUS könnte z. B.:

- exportierte Snippets/State gezielt an diese Module übergeben,
- daraus generierte Inhalte zurück in JANUS_KI_RESPONSE_V1 übersetzen (Phase 7).

Dependencies:

- Phase 7 (KI-Roundtrip),
- bestehende AI-Module.

Risiko/Komplexität:\

Mittel; hängt stark vom API-Design der Zielmodule ab.

Epic D2 -- On-the-fly Content: Namen, Hooks, Prüfungen

Idee:\

Schlanke KI-Funktionen, die:

- mit Exportdaten (NPC-Datenbank, Lektionen, Zauberlehrplan) kurze, gut abgegrenzte Aufgaben lösen:
  - Namen generieren (ähnlich „Integrate AI: Gimme a Name"),
  - Plot-Hooks generieren (ähnlich „Gimme a Hook"),
  - Prüfungsszenarien, Klassendynamiken, Mini-Abenteuer.

Dependencies:

- Phase 2 (Referenzdaten),
- Phase 7 (Prompt-Generator & Importpfade).

Risiko/Komplexität:\

Gering--mittel; gut inkrementell baubar.

Epic D3 -- Halbautomatischer „Director Assistant"

Idee:\

Ein optionaler Modus, in dem die KI:

- Vorschläge macht, „was als Nächstes passieren könnte", basierend auf State,

- aber **nichts** automatisch ausführt -- du bestätigst alles im UI.

Dependencies:

- Starker Ausbau von Phase 7 (Kontextaufbereitung),
- Stabile State- und Thread-Modelle (Phase 1/4).

Risiko/Komplexität:\

Hoch; Gefahr, dass es mehr kognitive Last als Hilfe erzeugt. Eindeutig „später/später" Epic.

E. Tooling & Dev-Workflow

Epic E1 -- Build-Pipeline mit Module Maker & CLI

Idee:\

Standardisierte Dev-Pipeline, die:

- den Foundry **Module Maker** nutzt, um Compendia/Manifeste sauber zu pflegen,
- mit dem **foundryvtt-cli** und ggf. Modulen wie **Compendium Exporter** zusammenspielt, um:
  - Compendia in JSON/YAML zu entpacken,
  - KI-Kits zu bauen,
  - Releases als ZIP stabil zu erzeugen.

Nutzen:

- weniger Handarbeit beim Modulpacken,
- reproduzierbare Builds,
- einfacher Transfer zwischen Welten.

Dependencies:

- keine In-World-Phasen; reines Dev-Thema.
- muss aber mit JSON-/Compendium-Strukturen kompatibel sein.

Risiko/Komplexität:\

Mittel; eher Skript- & Tooling-Aufwand als Code im Modul.

Epic E2 -- Test Harness / Simulation Mode

Idee:\

Kleiner „Testmodus", bei dem:

- du beliebige Zeitpunkte/Slots und Events im Schnelldurchlauf simulierst,
- ohne echte Spieler oder produktive World zu brauchen.

Nutzen:

- Regressionstests für Kalender, Scoring, Events, DSA5-Bridge,
- schneller Check nach einem Update (Foundry v14/v15, DSA5-Update etc.).

Dependencies:

- Engine-APIs in Phasen 1--5 müssen „testbar" sein (kein harter UI-Kram in der Logik).

Risiko/Komplexität:\

Mittel; hoher Nutzen, aber initialer Setup-Aufwand.

Epic E3 -- Importer aus offiziellen DSA5-Modulen

Idee:\

Automatisierte/halbautomatisierte Generatoren, die:

- aus deinen vielen DSA5-Modulen (Core, Magic, Bestiary, Library, Study Room usw.)
  - passende Lektionen, Bibliothekslisten, Prüfungsinhalte, NSC-Lexika extrahieren.

Nutzen:

- nutzt deine vorhandenen Module maximal aus,
- reduziert doppelte Dateneingabe.

Dependencies:

- Phase 3 (DSA5 Bridge),
- DSA5-Modulstruktur (Actors, Items, Journals),
- Phase 2 (Static Data-Modelle).

Risiko/Komplexität:\

Mittel--hoch; DSA5-Module sind vielfältig strukturiert.

8.4 Priorisierung & Labels

Du kannst die Epics z. B. mit Labels versehen:

- **P1 -- Nice-to-have bald** (hoher Mehrwert, überschaubarer Aufwand):
  - B1 Curriculum Editor (schlanke Version),

- C2 Session-Prep Wizard,
- D2 On-the-fly Content,
- E1 Build-Pipeline (zumindest Basis).

- **P2 -- Mittelfristige Erweiterung**:
  - B2 Noten & Zeugnisse,
  - C1 Second-Screen View (Read-Only),
  - C3 Analytics-Light,
  - E2 Test Harness.
- **P3 -- Langfristige Vision / Forschungsprojekte**:
  - A1 Multi-Akademie,
  - A3 Lore-Layering (voll ausgebaut),
  - B3 Generisches School Framework,
  - D1 tiefe Integrate-AI-/Cibola-/Phil-Integration,
  - D3 halbautomatischer Director Assistant,
  - E3 automatische DSA5-Importer.

8.5 Risiken & Leitplanken für den Backlog

- **Kein Core-Bloat:**\

  Nichts aus Phase 8 wandert in 7.0, solange Phasen 1--7 nicht stabil und im Alltag erprobt sind.

- **Modularität first:**\

  Jedes Epic wird, wenn umgesetzt, in **eigene Dateien/Namespaces** gepackt -- kein „mal eben" in den Director klatschen.

- **Foundry- & System-Updates im Blick:**\

  Da Foundry mit v13 viel auf ApplicationV2 und neue UX setzt, sollten Erweiterungen eng an den offiziellen Guides alignen.

- **Nicht gegen existierende Module arbeiten:**\

  Insbesondere bei:

- Mobilität/Touch (TouchVTT, Mobile Improvements, Mobile Token Control, Vadanx Mobile Controller),
- Audio/Atmosphäre (Soundscape, Soundscape Adventure, Audio-Tag „audio" im Modul-Hub),
- KI (Integrate AI, Phil's AI Assistant, Cibola 8).

8.6 Umsetzungsempfehlung für Phase 8

Phase 8 ist kein Blocker für 7.0, sondern ein sich ständig bewegender Ideenpool. Praktischer Ansatz:

122. **Backlog als Datei pflegen**

- z\. B. JANUS7_BACKLOG.md mit genau diesen Epics,
- dort kannst du Häkchen, Datumsangaben, Notizen ergänzen.

123. **Bei jedem größeren Release-Zyklus** (z. B. 7.1, 7.2):

- 1--2 P1-Epics auswählen,
- sauber planen, wie in Phasen 1--7,
- nie mehr als ein „großes Architektur-Epic" parallel.

124. **Kill-Switch beibehalten:**\

Wenn ein Epic sich als unnötig/zu teuer erweist → offiziell streichen oder in einen „Archiv"-Abschnitt verschieben, statt es als Zombie-Idee mitzuschleppen.

Damit hast du jetzt:

- einen **klaren, kuratierten Backlog**,
- geclusterte Erweiterungen mit Dependencies und grober Priorität,
- und -- ganz wichtig -- eine Barriere zwischen „JANUS7-Core" und allem, was danach kommt.

Das hält dein Projekt sauber, auch wenn du in ein, zwei Jahren plötzlich auf die Idee kommst, Punin parallel zu einer GURPS-Zauberuni auf dem Mars laufen zu lassen.

Schlusswort

Diese umfassende Dokumentation vereint alle Aspekte von JANUS7 – von der Vision und Architektur über die detaillierten Implementierungsphasen bis hin zum strukturierten Backlog für zukünftige Erweiterungen.

Mit dieser Roadmap ist JANUS7:

- Funktional: Alle Kernfeatures sind durchdacht und planbar
- Wartbar: Klare Architektur und Trennung ermöglichen langfristige Pflege
- Erweiterbar: Modularer Aufbau ermöglicht schrittweise Verbesserungen
- Zukunftssicher: Basiert auf offiziellen Foundry-APIs und Best Practices

Viel Erfolg bei der Umsetzung dieses ambitionierten Projekts!

— Das JANUS7 Entwicklungsteam